CHAPTER 9
Code Review

Written by Tom Manshreck and Caitlin Sadowski
Edited by Lisa Carey

Code review is a process in which code is reviewed by someone other than the author, often before the introduction of that code into a codebase. Although that is a simple definition, implementations of the process of code review vary widely throughout the software industry. Some organizations have a select group of "gatekeepers" across the codebase that review changes. Others delegate code review processes to smaller teams, allowing different teams to require different levels of code review. At Google, essentially every change is reviewed before being committed, and every engineer is responsible for initiating reviews and reviewing changes.

Code reviews generally require a combination of a process and a tool supporting that process. At Google, we use a custom code review tool, Critique, to support our process.[1] Critique is an important enough tool at Google to warrant its own chapter in this book. This chapter focuses on the process of code review as it is practiced at Google rather than the specific tool, both because these foundations are older than the tool and because most of these insights can be adapted to whatever tool you might use for code review.

For more information on Critique, see Chapter 19.

1 We also use Gerrit to review Git code, primarily for our open source projects. However, Critique is the primary tool of a typical software engineer at Google.

Some of the benefits of code review, such as detecting bugs in code before they enter a codebase, are well established[2] and somewhat obvious (if imprecisely measured). Other benefits, however, are more subtle. Because the code review process at Google is so ubiquitous and extensive, we've noticed many of these more subtle effects, including psychological ones, which provide many benefits to an organization over time and scale.

Code Review Flow

Code reviews can happen at many stages of software development. At Google, code reviews take place before a change can be committed to the codebase; this stage is also known as a *precommit review*. The primary end goal of a code review is to get another engineer to consent to the change, which we denote by tagging the change as "looks good to me" (LGTM). We use this LGTM as a necessary permissions "bit" (combined with other bits noted below) to allow the change to be committed.

A typical code review at Google goes through the following steps:

1. A user writes a change to the codebase in their workspace. This *author* then creates a snapshot of the change: a patch and corresponding description that are uploaded to the code review tool. This change produces a *diff* against the codebase, which is used to evaluate what code has changed.
2. The author can use this initial patch to apply automated review comments or do self-review. When the author is satisfied with the diff of the change, they mail the change to one or more reviewers. This process notifies those reviewers, asking them to view and comment on the snapshot.
3. *Reviewers* open the change in the code review tool and post comments on the diff. Some comments request explicit resolution. Some are merely informational.
4. The author modifies the change and uploads new snapshots based on the feedback and then replies back to the reviewers. Steps 3 and 4 may be repeated multiple times.
5. After the reviewers are happy with the latest state of the change, they agree to the change and accept it by marking it as "looks good to me" (LGTM). Only one LGTM is required by default, although convention might request that all reviewers agree to the change.
6. After a change is marked LGTM, the author is allowed to commit the change to the codebase, provided they *resolve all comments* and that the change is *approved*. We'll cover approval in the next section.

2 Steve McConnell, *Code Complete* (Redmond: Microsoft Press, 2004).

We'll go over this process in more detail later in this chapter.

Code Is a Liability

It's important to remember (and accept) that code itself is a liability. It might be a necessary liability, but by itself, code is simply a maintenance task to someone somewhere down the line. Much like the fuel that an airplane carries, it has weight, though it is, of course, necessary for that airplane to fly.

New features are often necessary, of course, but care should be taken before developing code in the first place to ensure that any new feature is warranted. Duplicated code not only is a wasted effort, it can actually cost more in time than not having the code at all; changes that could be easily performed under one code pattern often require more effort when there is duplication in the codebase. Writing entirely new code is so frowned upon that some of us have a saying: "If you're writing it from scratch, you're doing it wrong!"

This is especially true of library or utility code. Chances are, if you are writing a utility, someone else somewhere in a codebase the size of Google's has probably done something similar. Tools such as those discussed in Chapter 17 are therefore critical for both finding such utility code and preventing the introduction of duplicate code. Ideally, this research is done beforehand, and a design for anything new has been communicated to the proper groups before any new code is written.

Of course, new projects happen, new techniques are introduced, new components are needed, and so on. All that said, a code review is not an occasion to rehash or debate previous design decisions. Design decisions often take time, requiring the circulation of design proposals, debate on the design in API reviews or similar meetings, and perhaps the development of prototypes. As much as a code review of entirely new code should not come out of the blue, the code review process itself should also not be viewed as an opportunity to revisit previous decisions.

How Code Review Works at Google

We've pointed out roughly how the typical code review process works, but the devil is in the details. This section outlines in detail how code review works at Google and how these practices allow it to scale properly over time.

There are three aspects of review that require "approval" for any given change at Google:

- A correctness and comprehension check from another engineer that the code is appropriate and does what the author claims it does. This is often a team member, though it does not need to be. This is reflected in the LGTM permissions

"bit," which will be set after a peer reviewer agrees that the code "looks good" to them.

- Approval from one of the code owners that the code is appropriate for this particular part of the codebase (and can be checked into a particular directory). This approval might be implicit if the author is such an owner. Google's codebase is a tree structure with hierarchical owners of particular directories. (See Chapter 16). Owners act as gatekeepers for their particular directories. A change might be proposed by any engineer and LGTM'ed by any other engineer, but an owner of the directory in question must also *approve* this addition to their part of the codebase. Such an owner might be a tech lead or other engineer deemed expert in that particular area of the codebase. It's generally up to each team to decide how broadly or narrowly to assign ownership privileges.
- Approval from someone with language "readability"[3] that the code conforms to the language's style and best practices, checking whether the code is written in the manner we expect. This approval, again, might be implicit if the author has such readability. These engineers are pulled from a company-wide pool of engineers who have been granted readability in that programming language.

Although this level of control sounds onerous—and, admittedly, it sometimes is—most reviews have one person assuming all three roles, which speeds up the process quite a bit. Importantly, the author can also assume the latter two roles, needing only an LGTM from another engineer to check code into their own codebase, provided they already have readability in that language (which owners often do).

These requirements allow the code review process to be quite flexible. A tech lead who is an owner of a project and has that code's language readability can submit a code change with only an LGTM from another engineer. An intern without such authority can submit the same change to the same codebase, provided they get approval from an owner with language readability. The three aforementioned permission "bits" can be combined in any combination. An author can even request more than one LGTM from separate people by explicitly tagging the change as wanting an LGTM from all reviewers.

In practice, most code reviews that require more than one approval usually go through a two-step process: gaining an LGTM from a peer engineer, and then seeking approval from appropriate code owner/readability reviewer(s). This allows the two roles to focus on different aspects of the code review and saves review time. The primary reviewer can focus on code correctness and the general validity of the code change; the code owner can focus on whether this change is appropriate for their part

3 At Google, "readability" does not refer simply to comprehension, but to the set of styles and best practices that allow code to be maintainable to other engineers. See Chapter 3.

of the codebase without having to focus on the details of each line of code. An approver is often looking for something different than a peer reviewer, in other words. After all, someone is trying to check in code to their project/directory. They are more concerned with questions such as: "Will this code be easy or difficult to maintain?" "Does it add to my technical debt?" "Do we have the expertise to maintain it within our team?"

If all three of these types of reviews can be handled by one reviewer, why not just have those types of reviewers handle all code reviews? The short answer is scale. Separating the three roles adds flexibility to the code review process. If you are working with a peer on a new function within a utility library, you can get someone on your team to review the code for code correctness and comprehension. After several rounds (perhaps over several days), your code satisfies your peer reviewer and you get an LGTM. Now, you need only get an *owner* of the library (and owners often have appropriate readability) to approve the change.

Ownership

Hyrum Wright

When working on a small team in a dedicated repository, it's common to grant the entire team access to everything in the repository. After all, you know the other engineers, the domain is narrow enough that each of you can be experts, and small numbers constrain the effect of potential errors.

As the team grows larger, this approach can fail to scale. The result is either a messy repository split or a different approach to recording who has what knowledge and responsibilities in different parts of the repository. At Google, we call this set of knowledge and responsibilities *ownership* and the people to exercise them *owners*. This concept is different than possession of a collection of source code, but rather implies a sense of stewardship to act in the company's best interest with a section of the codebase. (Indeed, "stewards" would almost certainly be a better term if we had it to do over again.)

Specially named OWNERS files list usernames of people who have ownership responsibilities for a directory and its children. These files may also contain references to other OWNERS files or external access control lists, but eventually they resolve to a list of individuals. Each subdirectory may also contain a separate OWNERS file, and the relationship is hierarchically additive: a given file is generally owned by the union of the members of all the OWNERS files above it in the directory tree. OWNERS files may have as many entries as teams like, but we encourage a relatively small and focused list to ensure responsibility is clear.

Ownership of Google's code conveys approval rights for code within one's purview, but these rights also come with a set of responsibilities, such as understanding the code that is owned or knowing how to find somebody who does. Different teams have

different criteria for granting ownership to new members, but we generally encourage them not to use ownership as a rite of initiation and encourage departing members to yield ownership as soon as is practical.

This distributed ownership structure enables many of the other practices we've outlined in this book. For example, the set of people in the root OWNERS file can act as global approvers for large-scale changes (see Chapter 22) without having to bother local teams. Likewise, OWNERS files act as a kind of documentation, making it easy for people and tools to find those responsible for a given piece of code just by walking up the directory tree. When new projects are created, there's no central authority that has to register new ownership privileges: a new OWNERS file is sufficient.

This ownership mechanism is simple, yet powerful, and has scaled well over the past two decades. It is one of the ways that Google ensures that tens of thousands of engineers can operate efficiently on billions of lines of code in a single repository.

Code Review Benefits

Across the industry, code review itself is not controversial, although it is far from a universal practice. Many (maybe even most) other companies and open source projects have some form of code review, and most view the process as important as a sanity check on the introduction of new code into a codebase. Software engineers understand some of the more obvious benefits of code review, even if they might not personally think it applies in all cases. But at Google, this process is generally more thorough and wide spread than at most other companies.

Google's culture, like that of a lot of software companies, is based on giving engineers wide latitude in how they do their jobs. There is a recognition that strict processes tend not to work well for a dynamic company needing to respond quickly to new technologies, and that bureaucratic rules tend not to work well with creative professionals. Code review, however, is a mandate, one of the few blanket processes in which all software engineers at Google must participate. Google requires code review for almost[4] every code change to the codebase, no matter how small. This mandate does have a cost and effect on engineering velocity given that it does slow down the introduction of new code into a codebase and can impact time-to-production for any given code change. (Both of these are common complaints by software engineers of strict code review processes.) Why, then, do we require this process? Why do we believe that this is a long-term benefit?

4 Some changes to documentation and configurations might not require a code review, but it is often still preferable to obtain such a review.

A well-designed code review process and a culture of taking code review seriously provides the following benefits:

- Checks code correctness
- Ensures the code change is comprehensible to other engineers
- Enforces consistency across the codebase
- Psychologically promotes team ownership
- Enables knowledge sharing
- Provides a historical record of the code review itself

Many of these benefits are critical to a software organization over time, and many of them are beneficial to not only the author but also the reviewers. The following sections go into more specifics for each of these items.

Code Correctness

An obvious benefit of code review is that it allows a reviewer to check the "correctness" of the code change. Having another set of eyes look over a change helps ensure that the change does what was intended. Reviewers typically look for whether a change has proper testing, is properly designed, and functions correctly and efficiently. In many cases, checking code correctness is checking whether the particular change can introduce bugs into the codebase.

Many reports point to the efficacy of code review in the prevention of future bugs in software. A study at IBM found that discovering defects earlier in a process, unsurprisingly, led to less time required to fix them later on.[5] The investment in the time for code review saved time otherwise spent in testing, debugging, and performing regressions, provided that the code review process itself was streamlined to keep it lightweight. This latter point is important; code review processes that are heavyweight, or that don't scale properly, become unsustainable.[6] We will get into some best practices for keeping the process lightweight later in this chapter.

To prevent the evaluation of correctness from becoming more subjective than objective, authors are generally given deference to their particular approach, whether it be in the design or the function of the introduced change. A reviewer shouldn't propose

5 "Advances in Software Inspection," *IEEE Transactions on Software Engineering*, SE-12(7): 744–751, July 1986. Granted, this study took place before robust tooling and automated testing had become so important in the software development process, but the results still seem relevant in the modern software age.

6 Rigby, Peter C. and Christian Bird. 2013. "Convergent software peer review practices." ESEC/FSE 2013: *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*, August 2013: 202-212. *https://dl.acm.org/doi/10.1145/2491411.2491444*.

alternatives because of personal opinion. Reviewers can propose alternatives, but only if they improve comprehension (by being less complex, for example) or functionality (by being more efficient, for example). In general, engineers are encouraged to approve changes that improve the codebase rather than wait for consensus on a more "perfect" solution. This focus tends to speed up code reviews.

As tooling becomes stronger, many correctness checks are performed automatically through techniques such as static analysis and automated testing (though tooling might never completely obviate the value for human-based inspection of code—see Chapter 20 for more information). Though this tooling has its limits, it has definitely lessoned the need to rely on human-based code reviews for checking code correctness.

That said, checking for defects during the initial code review process is still an integral part of a general "shift left" strategy, aiming to discover and resolve issues at the earliest possible time so that they don't require escalated costs and resources farther down in the development cycle. A code review is neither a panacea nor the only check for such correctness, but it is an element of a defense-in-depth against such problems in software. As a result, code review does not need to be "perfect" to achieve results.

Surprisingly enough, checking for code correctness is not the primary benefit Google accrues from the process of code review. Checking for code correctness generally ensures that a change works, but more importance is attached to ensuring that a code change is understandable and makes sense over time and as the codebase itself scales. To evaluate those aspects, we need to look at factors other than whether the code is simply logically "correct" or understood.

Comprehension of Code

A code review typically is the first opportunity for someone other than the author to inspect a change. This perspective allows a reviewer to do something that even the best engineer cannot do: provide feedback unbiased by an author's perspective. *A code review is often the first test of whether a given change is understandable to a broader audience*. This perspective is vitally important because code will be read many more times than it is written, and understanding and comprehension are critically important.

It is often useful to find a reviewer who has a different perspective from the author, especially a reviewer who might need, as part of their job, to maintain or use the code being proposed within the change. Unlike the deference reviewers should give authors regarding design decisions, it's often useful to treat questions on code comprehension using the maxim "the customer is always right." In some respect, any questions you get now will be multiplied many-fold over time, so view each question on code comprehension as valid. This doesn't mean that you need to change your

approach or your logic in response to the criticism, but it does mean that you might need to explain it more clearly.

Together, the code correctness and code comprehension checks are the main criteria for an LGTM from another engineer, which is one of the approval bits needed for an approved code review. When an engineer marks a code review as LGTM, they are saying that the code does what it says and that it is understandable. Google, however, also requires that the code be sustainably maintained, so we have additional approvals needed for code in certain cases.

Code Consistency

At scale, code that you write will be depended on, and eventually maintained, by someone else. Many others will need to read your code and understand what you did. Others (including automated tools) might need to refactor your code long after you've moved to another project. Code, therefore, needs to conform to some standards of consistency so that it can be understood and maintained. Code should also avoid being overly complex; simpler code is easier for others to understand and maintain as well. Reviewers can assess how well this code lives up to the standards of the codebase itself during code review. A code review, therefore, should act to ensure *code health*.

It is for maintainability that the LGTM state of a code review (indicating code correctness and comprehension) is separated from that of readability approval. Readability approvals can be granted only by individuals who have successfully gone through the process of code readability training in a particular programming language. For example, Java code requires approval from an engineer who has "Java readability."

A readability approver is tasked with reviewing code to ensure that it follows agreed-on best practices for that particular programming language, is consistent with the codebase for that language within Google's code repository, and avoids being overly complex. Code that is consistent and simple is easier to understand and easier for tools to update when it comes time for refactoring, making it more resilient. If a particular pattern is always done in one fashion in the codebase, it's easier to write a tool to refactor it.

Additionally, code might be written only once, but it will be read dozens, hundreds, or even thousands of times. Having code that is consistent across the codebase improves comprehension for all of engineering, and this consistency even affects the process of code review itself. Consistency sometimes clashes with functionality; a readability reviewer may prefer a less complex change that may not be functionally "better" but is easier to understand.

With a more consistent codebase, it is easier for engineers to step in and review code on someone else's projects. Engineers might occasionally need to look outside the

team for help in a code review. Being able to reach out and ask experts to review the code, knowing they can expect the code itself to be consistent, allows those engineers to focus more properly on code correctness and comprehension.

Psychological and Cultural Benefits

Code review also has important cultural benefits: it reinforces to software engineers that code is not "theirs" but in fact part of a collective enterprise. Such psychological benefits can be subtle but are still important. Without code review, most engineers would naturally gravitate toward personal style and their own approach to software design. The code review process forces an author to not only let others have input, but to compromise for the sake of the greater good.

It is human nature to be proud of one's craft and to be reluctant to open up one's code to criticism by others. It is also natural to be somewhat reticent to welcome critical feedback about code that one writes. The code review process provides a mechanism to mitigate what might otherwise be an emotionally charged interaction. Code review, when it works best, provides not only a challenge to an engineer's assumptions, but also does so in a prescribed, neutral manner, acting to temper any criticism which might otherwise be directed to the author if provided in an unsolicited manner. After all, the process *requires* critical review (we in fact call our code review tool "Critique"), so you can't fault a reviewer for doing their job and being critical. The code review process itself, therefore, can act as the "bad cop," whereas the reviewer can still be seen as the "good cop."

Of course, not all, or even most, engineers need such psychological devices. But buffering such criticism through the process of code review often provides a much gentler introduction for most engineers to the expectations of the team. Many engineers joining Google, or a new team, are intimidated by code review. It is easy to think that any form of critical review reflects negatively on a person's job performance. But over time, almost all engineers come to expect to be challenged when sending a code review and come to value the advice and questions offered through this process (though, admittedly, this sometimes takes a while).

Another psychological benefit of code review is validation. Even the most capable engineers can suffer from imposter syndrome and be too self-critical. A process like code review acts as validation and recognition for one's work. Often, the process involves an exchange of ideas and knowledge sharing (covered in the next section), which benefits both the reviewer and the reviewee. As an engineer grows in their domain knowledge, it's sometimes difficult for them to get positive feedback on how they improve. The process of code review can provide that mechanism.

The process of initiating a code review also forces all authors to take a little extra care with their changes. Many software engineers are not perfectionists; most will admit that code that "gets the job done" is better than code that is perfect but that takes too

long to develop. Without code review, it's natural that many of us would cut corners, even with the full intention of correcting such defects later. "Sure, I don't have all of the unit tests done, but I can do that later." A code review forces an engineer to resolve those issues before sending the change. Collecting the components of a change for code review psychologically forces an engineer to make sure that all of their ducks are in a row. The little moment of reflection that comes before sending off your change is the perfect time to read through your change and make sure you're not missing anything.

Knowledge Sharing

One of the most important, but underrated, benefits of code review is in knowledge sharing. Most authors pick reviewers who are experts, or at least knowledgeable, in the area under review. The review process allows reviewers to impart domain knowledge to the author, allowing the reviewer(s) to offer suggestions, new techniques, or advisory information to the author. (Reviewers can even mark some comments "FYI," requiring no action; they are simply added as an aid to the author.) Authors who become particularly proficient in an area of the codebase will often become owners as well, who then in turn will be able to act as reviewers for other engineers.

Part of the code review process of feedback and confirmation involves asking questions on why the change is done in a particular way. This exchange of information facilitates knowledge sharing. In fact, many code reviews involve an exchange of information both ways: the authors as well as the reviewers can learn new techniques and patterns from code review. At Google, reviewers may even directly share suggested edits with an author within the code review tool itself.

An engineer may not read every email sent to them, but they tend to respond to every code review sent. This knowledge sharing can occur across time zones and projects as well, using Google's scale to disseminate information quickly to engineers in all corners of the codebase. Code review is a perfect time for knowledge transfer: it is timely and actionable. (Many engineers at Google "meet" other engineers first through their code reviews!)

Given the amount of time Google engineers spend in code review, the knowledge accrued is quite significant. A Google engineer's primary task is still programming, of course, but a large chunk of their time is still spent in code review. The code review process provides one of the primary ways that software engineers interact with one another and exchange information about coding techniques. Often, new patterns are advertised within the context of code review, sometimes through refactorings such as large-scale changes.

Moreover, because each change becomes part of the codebase, code review acts as a historical record. Any engineer can inspect the Google codebase and determine when some particular pattern was introduced and bring up the actual code review in

question. Often, that archeology provides insights to many more engineers than the original author and reviewer(s).

Code Review Best Practices

Code review can, admittedly, introduce friction and delay to an organization. Most of these issues are not problems with code review per se, but with their chosen implementation of code review. Keeping the code review process running smoothly at Google is no different, and it requires a number of best practices to ensure that code review is worth the effort put into the process. Most of those practices emphasize keeping the process nimble and quick so that code review can scale properly.

Be Polite and Professional

As pointed out in the Culture section of this book, Google heavily fosters a culture of trust and respect. This filters down into our perspective on code review. A software engineer needs an LGTM from only one other engineer to satisfy our requirement on code comprehension, for example. Many engineers make comments and LGTM a change with the understanding that the change can be submitted after those changes are made, without any additional rounds of review. That said, code reviews can introduce anxiety and stress to even the most capable engineers. It is critically important to keep all feedback and criticism firmly in the professional realm.

In general, reviewers should defer to authors on particular approaches and only point out alternatives if the author's approach is deficient. If an author can demonstrate that several approaches are equally valid, the reviewer should accept the preference of the author. Even in those cases, if defects are found in an approach, consider the review a learning opportunity (for both sides!). All comments should remain strictly professional. Reviewers should be careful about jumping to conclusions based on a code author's particular approach. It's better to ask questions on why something was done the way it was before assuming that approach is wrong.

Reviewers should be prompt with their feedback. At Google, we expect feedback from a code review within 24 (working) hours. If a reviewer is unable to complete a review in that time, it's good practice (and expected) to respond that they've at least seen the change and will get to the review as soon as possible. Reviewers should avoid responding to the code review in piecemeal fashion. Few things annoy an author more than getting feedback from a review, addressing it, and then continuing to get unrelated further feedback in the review process.

As much as we expect professionalism on the part of the reviewer, we expect professionalism on the part of the author as well. Remember that you are not your code, and that this change you propose is not "yours" but the team's. After you check that piece of code into the codebase, it is no longer yours in any case. Be receptive to

questions on your approach, and be prepared to explain why you did things in certain ways. Remember that part of the responsibility of an author is to make sure this code is understandable and maintainable for the future.

It's important to treat each reviewer comment within a code review as a TODO item; a particular comment might not need to be accepted without question, but it should at least be addressed. If you disagree with a reviewer's comment, let them know, and let them know why and don't mark a comment as resolved until each side has had a chance to offer alternatives. One common way to keep such debates civil if an author doesn't agree with a reviewer is to offer an alternative and ask the reviewer to PTAL (please take another look). Remember that code review is a learning opportunity for both the reviewer and the author. That insight often helps to mitigate any chances for disagreement.

By the same token, if you are an owner of code and responding to a code review within your codebase, be amenable to changes from an outside author. As long as the change is an improvement to the codebase, you should still give deference to the author that the change indicates something that could and should be improved.

Write Small Changes

Probably the most important practice to keep the code review process nimble is to keep changes small. A code review should ideally be easy to digest and focus on a single issue, both for the reviewer and the author. Google's code review process discourages massive changes consisting of fully formed projects, and reviewers can rightfully reject such changes as being too large for a single review. Smaller changes also prevent engineers from wasting time waiting for reviews on larger changes, reducing downtime. These small changes have benefits further down in the software development process as well. It is far easier to determine the source of a bug within a change if that particular change is small enough to narrow it down.

That said, it's important to acknowledge that a code review process that relies on small changes is sometimes difficult to reconcile with the introduction of major new features. A set of small, incremental code changes can be easier to digest individually, but more difficult to comprehend within a larger scheme. Some engineers at Google admittedly are not fans of the preference given to small changes. Techniques exist for managing such code changes (development on integration branches, management of changes using a diff base different than HEAD), but those techniques inevitably involve more overhead. Consider the optimization for small changes just that: an optimization, and allow your process to accommodate the occasional larger change.

"Small" changes should generally be limited to about 200 lines of code. A small change should be easy on a reviewer and, almost as important, not be so cumbersome that additional changes are delayed waiting for an extensive review. Most changes at

Google are expected to be reviewed within about a day.[7] (This doesn't necessarily mean that the review is over within a day, but that initial feedback is provided within a day.) About 35% of the changes at Google are to a single file.[8] Being easy on a reviewer allows for quicker changes to the codebase and benefits the author as well. The author wants a quick review; waiting on an extensive review for a week or so would likely impact follow-on changes. A small initial review also can prevent much more expensive wasted effort on an incorrect approach further down the line.

Because code reviews are typically small, it's common for almost all code reviews at Google to be reviewed by one and only one person. Were that not the case—if a team were expected to weigh in on all changes to a common codebase—there is no way the process itself would scale. By keeping the code reviews small, we enable this optimization. It's not uncommon for multiple people to comment on any given change—most code reviews are sent to a team member, but also CC'd to appropriate teams—but the primary reviewer is still the one whose LGTM is desired, and only one LGTM is necessary for any given change. Any other comments, though important, are still optional.

Keeping changes small also allows the "approval" reviewers to more quickly approve any given changes. They can quickly inspect whether the primary code reviewer did due diligence and focus purely on whether this change augments the codebase while maintaining code health over time.

Write Good Change Descriptions

A change description should indicate its type of change on the first line, as a summary. The first line is prime real estate and is used to provide summaries within the code review tool itself, to act as the subject line in any associated emails, and to become the visible line Google engineers see in a history summary within Code Search (see Chapter 17), so that first line is important.

Although the first line should be a summary of the entire change, the description should still go into detail on what is being changed *and why*. A description of "Bug fix" is not helpful to a reviewer or a future code archeologist. If several related modifications were made in the change, enumerate them within a list (while still keeping it on message and small). The description is the historical record for this change, and tools such as Code Search allow you to find who wrote what line in any particular change in the codebase. Drilling down into the original change is often useful when trying to fix a bug.

7 Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, and Alberto Bacchelli, "Modern code review: a case study at Google."

8 Ibid.

Descriptions aren't the only opportunity for adding documentation to a change. When writing a public API, you generally don't want to leak implementation details, but by all means do so within the actual implementation, where you should comment liberally. If a reviewer does not understand why you did something, even if it is correct, it is a good indicator that such code needs better structure or better comments (or both). If, during the code review process, a new decision is reached, update the change description, or add appropriate comments within the implementation. A code review is not just something that you do in the present time; it is something you do to record what you did for posterity.

Keep Reviewers to a Minimum

Most code reviews at Google are reviewed by precisely one reviewer.[9] Because the code review process allows the bits on code correctness, owner acceptance, and language readability to be handled by one individual, the code review process scales quite well across an organization the size of Google.

There is a tendency within the industry, and within individuals, to try to get additional input (and unanimous consent) from a cross-section of engineers. After all, each additional reviewer can add their own particular insight to the code review in question. But we've found that this leads to diminishing returns; the most important LGTM is the first one, and subsequent ones don't add as much as you might think to the equation. The cost of additional reviewers quickly outweighs their value.

The code review process is optimized around the trust we place in our engineers to do the right thing. In certain cases, it can be useful to get a particular change reviewed by multiple people, but even in those cases, those reviewers should focus on different aspects of the same change.

Automate Where Possible

Code review is a human process, and that human input is important, but if there are components of the code process that can be automated, try to do so. Opportunities to automate mechanical human tasks should be explored; investments in proper tooling reap dividends. At Google, our code review tooling allows authors to automatically submit and automatically sync changes to the source control system upon approval (usually used for fairly simple changes).

One of the most important technological improvements regarding automation over the past few years is automatic static analysis of a given code change (see Chapter 20). Rather than require authors to run tests, linters, or formatters, the current Google code review tooling provides most of that utility automatically through what is

9 Ibid.

known as *presubmits*. A presubmit process is run when a change is initially sent to a reviewer. Before that change is sent, the presubmit process can detect a variety of problems with the existing change, reject the current change (and prevent sending an awkward email to a reviewer), and ask the original author to fix the change first. Such automation not only helps out with the code review process itself, it also allows the reviewers to focus on more important concerns than formatting.

Types of Code Reviews

All code reviews are not alike! Different types of code review require different levels of focus on the various aspects of the review process. Code changes at Google generally fall into one of the following buckets (though there is sometimes overlap):

- Greenfield reviews and new feature development
- Behavioral changes, improvements, and optimizations
- Bug fixes and rollbacks
- Refactorings and large-scale changes

Greenfield Code Reviews

The least common type of code review is that of entirely new code, a so-called *greenfield review*. A greenfield review is the most important time to evaluate whether the code will stand the test of time: that it will be easier to maintain as time and scale change the underlying assumptions of the code. Of course, the introduction of entirely new code should not come as a surprise. As mentioned earlier in this chapter, code is a liability, so the introduction of entirely new code should generally solve a real problem rather than simply provide yet another alternative. At Google, we generally require new code and/or projects to undergo an extensive design review, apart from a code review. A code review is not the time to debate design decisions already made in the past (and by the same token, a code review is not the time to introduce the design of a proposed API).

To ensure that code is sustainable, a greenfield review should ensure that an API matches an agreed design (which may require reviewing a design document) and is tested *fully*, with all API endpoints having some form of unit test, and that those tests fail when the code's assumptions change. (See Chapter 11). The code should also have proper owners (one of the first reviews in a new project is often of a single OWNERS file for the new directory), be sufficiently commented, and provide supplemental documentation, if needed. A greenfield review might also necessitate the introduction of a project into the continuous integration system. (See Chapter 23).

Behavioral Changes, Improvements, and Optimizations

Most changes at Google generally fall into the broad category of modifications to existing code within the codebase. These additions may include modifications to API endpoints, improvements to existing implementations, or optimizations for other factors such as performance. Such changes are the bread and butter of most software engineers.

In each of these cases, the guidelines that apply to a greenfield review also apply: is this change necessary, and does this change improve the codebase? Some of the best modifications to a codebase are actually deletions! Getting rid of dead or obsolete code is one of the best ways to improve the overall code health of a codebase.

Any behavioral modifications should necessarily include revisions to appropriate tests for any new API behavior. Augmentations to the implementation should be tested in a Continuous Integration (CI) system to ensure that those modifications don't break any underlying assumptions of the existing tests. As well, optimizations should of course ensure that they don't affect those tests and might need to include performance benchmarks for the reviewers to consult. Some optimizations might also require benchmark tests.

Bug Fixes and Rollbacks

Inevitably, you will need to submit a change for a bug fix to your codebase. *When doing so, avoid the temptation to address other issues*. Not only does this risk increasing the size of the code review, it also makes it more difficult to perform regression testing or for others to roll back your change. A bug fix should focus solely on fixing the indicated bug and (usually) updating associated tests to catch the error that occurred in the first place.

Addressing the bug with a revised test is often necessary. The bug surfaced because existing tests were either inadequate, or the code had certain assumptions that were not met. As a reviewer of a bug fix, it is important to ask for updates to unit tests if applicable.

Sometimes, a code change in a codebase as large as Google's causes some dependency to fail that was either not detected properly by tests or that unearths an untested part of the codebase. In those cases, Google allows such changes to be "rolled back," usually by the affected downstream customers. A rollback consists of a change that essentially undoes the previous change. Such rollbacks can be created in seconds because they just revert the previous change to a known state, but they still require a code review.

It also becomes critically important that any change that could cause a potential rollback (and that includes all changes!) be as small and atomic as possible so that a rollback, if needed, does not cause further breakages on other dependencies that can be

difficult to untangle. At Google, we've seen developers start to depend on new code very quickly after it is submitted, and rollbacks sometimes break these developers as a result. Small changes help to mitigate these concerns, both because of their atomicity, and because reviews of small changes tend to be done quickly.

Refactorings and Large-Scale Changes

Many changes at Google are automatically generated: the author of the change isn't a person, but a machine. We discuss more about the large-scale change (LSC) process in Chapter 22, but even machine-generated changes require review. In cases where the change is considered low risk, it is reviewed by designated reviewers who have approval privileges for our entire codebase. But for cases in which the change might be risky or otherwise requires local domain expertise, individual engineers might be asked to review automatically generated changes as part of their normal workflow.

At first look, a review for an automatically generated change should be handled the same as any other code review: the reviewer should check for correctness and applicability of the change. However, we encourage reviewers to limit comments in the associated change and only flag concerns that are specific to their code, not the underlying tool or LSC generating the changes. While the specific change might be machine generated, the overall process generating these changes has already been reviewed, and individual teams cannot hold a veto over the process, or it would not be possible to scale such changes across the organization. If there is a concern about the underlying tool or process, reviewers can escalate out of band to an LSC oversight group for more information.

We also encourage reviewers of automatic changes to avoid expanding their scope. When reviewing a new feature or a change written by a teammate, it is often reasonable to ask the author to address related concerns within the same change, so long as the request still follows the earlier advice to keep the change small. This does not apply to automatically generated changes because the human running the tool might have hundreds of changes in flight, and even a small percentage of changes with review comments or unrelated questions limits the scale at which the human can effectively operate the tool.

Conclusion

Code review is one of the most important and critical processes at Google. Code review acts as the glue connecting engineers with one another, and the code review process is the primary developer workflow upon which almost all other processes must hang, from testing to static analysis to CI. A code review process must scale appropriately, and for that reason, best practices, including small changes and rapid feedback and iteration, are important to maintain developer satisfaction and appropriate production velocity.

TL;DRs

- Code review has many benefits, including ensuring code correctness, comprehension, and consistency across a codebase.
- Always check your assumptions through someone else; optimize for the reader.
- Provide the opportunity for critical feedback while remaining professional.
- Code review is important for knowledge sharing throughout an organization.
- Automation is critical for scaling the process.
- The code review itself provides a historical record.

CHAPTER 10

Documentation

Written by Tom Manshreck
Edited by Riona MacNamara

Of the complaints most engineers have about writing, using, and maintaining code, a singular common frustration is the lack of quality documentation. "What are the side effects of this method?" "I got an error after step 3." "What does this acronym mean?" "Is this document up to date?" Every software engineer has voiced complaints about the quality, quantity, or sheer lack of documentation throughout their career, and the software engineers at Google are no different.

Technical writers and project managers may help, but software engineers will always need to write most documentation themselves. Engineers, therefore, need the proper tools and incentives to do so effectively. The key to making it easier for them to write quality documentation is to introduce processes and tools that scale with the organization and that tie into their existing workflow.

Overall, the state of engineering documentation in the late 2010s is similar to the state of software testing in the late 1980s. Everyone recognizes that more effort needs to be made to improve it, but there is not yet organizational recognition of its critical benefits. That is changing, if slowly. At Google, our most successful efforts have been when documentation is *treated like code* and incorporated into the traditional engineering workflow, making it easier for engineers to write and maintain simple documents.

What Qualifies as Documentation?

When we refer to "documentation," we're talking about every supplemental text that an engineer needs to write to do their job: not only standalone documents, but code comments as well. (In fact, most of the documentation an engineer at Google writes

comes in the form of code comments.) We'll discuss the various types of engineering documents further in this chapter.

Why Is Documentation Needed?

Quality documentation has tremendous benefits for an engineering organization. Code and APIs become more comprehensible, reducing mistakes. Project teams are more focused when their design goals and team objectives are clearly stated. Manual processes are easier to follow when the steps are clearly outlined. Onboarding new members to a team or code base takes much less effort if the process is clearly documented.

But because documentation's benefits are all necessarily downstream, they generally don't reap immediate benefits to the author. Unlike testing, which (as we'll see) quickly provides benefits to a programmer, documentation generally requires more effort up front and doesn't provide clear benefits to an author until later. But, like investments in testing, the investment made in documentation will pay for itself over time. After all, you might write a document only once,[1] but it will be read hundreds, perhaps thousands of times afterward; its initial cost is amortized across all the future readers. Not only does documentation scale over time, but it is critical for the rest of the organization to scale as well. It helps answer questions like these:

- Why were these design decisions made?
- Why did we implement this code in this manner?
- Why did *I* implement this code in this manner, if you're looking at your own code two years later?

If documentation conveys all these benefits, why is it generally considered "poor" by engineers? One reason, as we've mentioned, is that the benefits aren't *immediate*, especially to the writer. But there are several other reasons:

- Engineers often view writing as a separate skill than that of programming. (We'll try to illustrate that this isn't quite the case, and even where it is, it isn't necessarily a separate skill from that of software engineering.)
- Some engineers don't feel like they are capable writers. But you don't need a robust command of English[2] to produce workable documentation. You just need to step outside yourself a bit and see things from the audience's perspective.

1 OK, you will need to maintain it and revise it occasionally.

2 English is still the primary language for most programmers, and most technical documentation for programmers relies on an understanding of English.

- Writing documentation is often more difficult because of limited tools support or integration into the developer workflow.
- Documentation is viewed as an extra burden—something else to maintain—rather than something that will make maintenance of their existing code easier.

Not every engineering team needs a technical writer (and even if that were the case, there aren't enough of them). This means that engineers will, by and large, write most of the documentation themselves. So, instead of forcing engineers to become technical writers, we should instead think about how to make writing documentation easier for engineers. Deciding how much effort to devote to documentation is a decision your organization will need to make at some point.

Documentation benefits several different groups. Even to the writer, documentation provides the following benefits:

- It helps formulate an API. Writing documentation is one of the surest ways to figure out if your API makes sense. Often, the writing of the documentation itself leads engineers to reevaluate design decisions that otherwise wouldn't be questioned. If you can't explain it and can't define it, you probably haven't designed it well enough.
- It provides a road map for maintenance and a historical record. Tricks in code should be avoided, in any case, but good comments help out a great deal when you're staring at code you wrote two years ago, trying to figure out what's wrong.
- It makes your code look more professional and drive traffic. Developers will naturally assume that a well-documented API is a better-designed API. That's not always the case, but they are often highly correlated. Although this benefit sounds cosmetic, it's not quite so: whether a product has good documentation is usually a pretty good indicator of how well a product will be maintained.
- It will prompt fewer questions from other users. This is probably the biggest benefit over time to someone writing the documentation. If you have to explain something to someone more than once, it usually makes sense to document that process.

As great as these benefits are to the writer of documentation, the lion's share of documentation's benefits will naturally accrue to the reader. Google's C++ Style Guide notes the maxim "optimize for the reader." This maxim applies not just to code, but to the comments around code, or the documentation set attached to an API. Much like testing, the effort you put into writing good documents will reap benefits many times over its lifetime. Documentation is critical over time, and reaps tremendous benefits for especially critical code as an organization scales.

Documentation Is Like Code

Software engineers who write in a single, primary programming language still often reach for different languages to solve specific problems. An engineer might write shell scripts or Python to run command-line tasks, or they might write most of their backend code in C++ but write some middleware code in Java, and so on. Each language is a tool in the toolbox.

Documentation should be no different: it's a tool, written in a different language (usually English) to accomplish a particular task. Writing documentation is not much different than writing code. Like a programming language, it has rules, a particular syntax, and style decisions, often to accomplish a similar purpose as that within code: enforce consistency, improve clarity, and avoid (comprehension) errors. Within technical documentation, grammar is important not because one needs rules, but to standardize the voice and avoid confusing or distracting the reader. Google requires a certain comment style for many of its languages for this reason.

Like code, documents should also have owners. Documents without owners become stale and difficult to maintain. Clear ownership also makes it easier to handle documentation through existing developer workflows: bug tracking systems, code review tooling, and so forth. Of course, documents with different owners can still conflict with one another. In those cases, it is important to designate *canonical* documentation: determine the primary source and consolidate other associated documents into that primary source (or deprecate the duplicates).

The prevalent usage of "go/ links" at Google (see Chapter 3) makes this process easier. Documents with straightforward go/ links often become the canonical source of truth. One other way to promote canonical documents is to associate them directly with the code they document by placing them directly under source control and alongside the source code itself.

Documentation is often so tightly coupled to code that it should, as much as possible, be treated *as code*. That is, your documentation should:

- Have internal policies or rules to be followed
- Be placed under source control
- Have clear ownership responsible for maintaining the docs
- Undergo reviews for changes (and change *with* the code it documents)
- Have issues tracked, as bugs are tracked in code
- Be periodically evaluated (tested, in some respect)
- If possible, be measured for aspects such as accuracy, freshness, etc. (tools have still not caught up here)

The more engineers treat documentation as "one of" the necessary tasks of software development, the less they will resent the upfront costs of writing, and the more they will reap the long-term benefits. In addition, making the task of documentation easier reduces those upfront costs.

Case Study: The Google Wiki

When Google was much smaller and leaner, it had few technical writers. The easiest way to share information was through our own internal wiki (GooWiki). At first, this seemed like a reasonable approach; all engineers shared a single documentation set and could update it as needed.

But as Google scaled, problems with a wiki-style approach became apparent. Because there were no true owners for documents, many became obsolete.[3] Because no process was put in place for adding new documents, duplicate documents and document sets began appearing. GooWiki had a flat namespace, and people were not good at applying any hierarchy to the documentation sets. At one point, there were 7 to 10 documents (depending on how you counted them) on setting up Borg, our production compute environment, only a few of which seemed to be maintained, and most were specific to certain teams with certain permissions and assumptions.

Another problem with GooWiki became apparent over time: the people who could fix the documents were not the people who used them. New users discovering bad documents either couldn't confirm that the documents were wrong or didn't have an easy way to report errors. They knew something was wrong (because the document didn't work), but they couldn't "fix" it. Conversely, the people best able to fix the documents often didn't need to consult them after they were written. The documentation became so poor as Google grew that the quality of documentation became Google's number one developer complaint on our annual developer surveys.

The way to improve the situation was to move important documentation under the same sort of source control that was being used to track code changes. Documents began to have their own owners, canonical locations within the source tree, and processes for identifying bugs and fixing them; the documentation began to dramatically improve. Additionally, the way documentation was written and maintained began to look the same as how code was written and maintained. Errors in the documents could be reported within our bug tracking software. Changes to the documents could be handled using the existing code review process. Eventually, engineers began to fix the documents themselves or send changes to technical writers (who were often the owners).

3 When we deprecated GooWiki, we found that around 90% of the documents had no views or updates in the previous few months.

Moving documentation to source control was initially met with a lot of controversy. Many engineers were convinced that doing away with the GooWiki, that bastion of freedom of information, would lead to poor quality because the bar for documentation (requiring a review, requiring owners for documents, etc.) would be higher. But that wasn't the case. The documents became better.

The introduction of Markdown as a common documentation formatting language also helped because it made it easier for engineers to understand how to edit documents without needing specialized expertise in HTML or CSS. Google eventually introduced its own framework for embedding documentation within code: g3doc. With that framework, documentation improved further, as documents existed side by side with the source code within the engineer's development environment. Now, engineers could update the code and its associated documentation in the same change (a practice for which we're still trying to improve adoption).

The key difference was that maintaining documentation became a similar experience to maintaining code: engineers filed bugs, made changes to documents in changelists, sent changes to reviews by experts, and so on. Leveraging of existing developer workflows, rather than creating new ones, was a key benefit.

Know Your Audience

One of the most important mistakes that engineers make when writing documentation is to write only for themselves. It's natural to do so, and writing for yourself is not without value: after all, you might need to look at this code in a few years and try to figure out what you once meant. You also might be of approximately the same skill set as someone reading your document. But if you write only for yourself, you are going to make certain assumptions, and given that your document might be read by a very wide audience (all of engineering, external developers), even a few lost readers is a large cost. As an organization grows, mistakes in documentation become more prominent, and your assumptions often do not apply.

Instead, before you begin writing, you should (formally or informally) identify the audience(s) your documents need to satisfy. A design document might need to persuade decision makers. A tutorial might need to provide very explicit instructions to someone utterly unfamiliar with your codebase. An API might need to provide complete and accurate reference information for any users of that API, be they experts or novices. Always try to identify a primary audience and write to that audience.

Good documentation need not be polished or "perfect." One mistake engineers make when writing documentation is assuming they need to be much better writers. By that measure, few software engineers would write. Think about writing like you do about testing or any other process you need to do as an engineer. Write to your audience, in the voice and style that they expect. If you can read, you can write. Remember that

your audience is standing where you once stood, but *without your new domain knowledge*. So you don't need to be a great writer; you just need to get someone like you as familiar with the domain as you now are. (And as long as you get a stake in the ground, you can improve this document over time.)

Types of Audiences

We've pointed out that you should write at the skill level and domain knowledge appropriate for your audience. But who precisely is your audience? Chances are, you have multiple audiences based on one or more of the following criteria:

- Experience level (expert programmers, or junior engineers who might not even be familiar—gulp!—with the language).
- Domain knowledge (team members, or other engineers in your organization who are familiar only with API endpoints).
- Purpose (end users who might need your API to do a specific task and need to find that information quickly, or software gurus who are responsible for the guts of a particularly hairy implementation that you hope no one else needs to maintain).

In some cases, different audiences require different writing styles, but in most cases, the trick is to write in a way that applies as broadly to your different audience groups as possible. Often, you will need to explain a complex topic to both an expert and a novice. Writing for the expert with domain knowledge may allow you to cut corners, but you'll confuse the novice; conversely, explaining everything in detail to the novice will doubtless annoy the expert.

Obviously, writing such documents is a balancing act and there's no silver bullet, but one thing we've found is that it helps to keep your documents *short*. Write descriptively enough to explain complex topics to people unfamiliar with the topic, but don't lose or annoy experts. Writing a short document often requires you to write a longer one (getting all the information down) and then doing an edit pass, removing duplicate information where you can. This might sound tedious, but keep in mind that this expense is spread across all the readers of the documentation. As Blaise Pascal once said, "If I had more time, I would have written you a shorter letter." By keeping a document short and clear, you will ensure that it will satisfy both an expert and a novice.

Another important audience distinction is based on how a user encounters a document:

- *Seekers* are engineers who *know what they want* and want to know if what they are looking at fits the bill. A key pedagogical device for this audience is *consistency*. If you are writing reference documentation for this group—within a

code file, for example—you will want to have your comments follow a similar format so that readers can quickly scan a reference and see whether they find what they are looking for.

- *Stumblers* might not know exactly what they want. They might have only a vague idea of how to implement what they are working with. The key for this audience is *clarity*. Provide overviews or introductions (at the top of a file, for example) that explain the purpose of the code they are looking at. It's also useful to identify when a doc is *not* appropriate for an audience. A lot of documents at Google begin with a "TL;DR statement" such as "TL;DR: if you are not interested in C++ compilers at Google, you can stop reading now."

Finally, one important audience distinction is between that of a customer (e.g., a user of an API) and that of a provider (e.g., a member of the project team). As much as possible, documents intended for one should be kept apart from documents intended for the other. Implementation details are important to a team member for maintenance purposes; end users should not need to read such information. Often, engineers denote design decisions within the reference API of a library they publish. Such reasonings belong more appropriately in specific documents (design documents) or, at best, within the implementation details of code hidden behind an interface.

Documentation Types

Engineers write various different types of documentation as part of their work: design documents, code comments, how-to documents, project pages, and more. These all count as "documentation." But it is important to know the different types, and to *not mix types*. A document should have, in general, a singular purpose, and stick to it. Just as an API should do one thing and do it well, avoid trying to do several things within one document. Instead, break out those pieces more logically.

There are several main types of documents that software engineers often need to write:

- Reference documentation, including code comments
- Design documents
- Tutorials
- Conceptual documentation
- Landing pages

It was common in the early days of Google for teams to have monolithic wiki pages with bunches of links (many broken or obsolete), some conceptual information about how the system worked, an API reference, and so on, all sprinkled together. Such documents fail because they don't serve a single purpose (and they also get so long

that no one will read them; some notorious wiki pages scrolled through several dozens of screens). Instead, make sure your document has a singular purpose, and if adding something to that page doesn't make sense, you probably want to find, or even create, another document for that purpose.

Reference Documentation

Reference documentation is the most common type that engineers need to write; indeed, they often need to write some form of reference documents every day. By reference documentation, we mean anything that documents the usage of code within the codebase. Code comments are the most common form of reference documentation that an engineer must maintain. Such comments can be divided into two basic camps: API comments versus implementation comments. Remember the audience differences between these two: API comments don't need to discuss implementation details or design decisions and can't assume a user is as versed in the API as the author. Implementation comments, on the other hand, can assume a lot more domain knowledge of the reader, though be careful in assuming too much: people leave projects, and sometimes it's safer to be methodical about exactly why you wrote this code the way you did.

Most reference documentation, even when provided as separate documentation from the code, is generated from comments within the codebase itself. (As it should; reference documentation should be single-sourced as much as possible.) Some languages such as Java or Python have specific commenting frameworks (Javadoc, PyDoc, GoDoc) meant to make generation of this reference documentation easier. Other languages, such as C++, have no standard "reference documentation" implementation, but because C++ separates out its API surface (in header or *.h* files) from the implementation (*.cc* files), header files are often a natural place to document a C++ API.

Google takes this approach: a C++ API deserves to have its reference documentation live within the header file. Other reference documentation is embedded directly in the Java, Python, and Go source code as well. Because Google's Code Search browser (see Chapter 17) is so robust, we've found little benefit to providing separate generated reference documentation. Users in Code Search not only search code easily, they can usually find the original definition of that code as the top result. Having the documentation alongside the code's definitions also makes the documentation easier to discover and maintain.

We all know that code comments are essential to a well-documented API. But what precisely is a "good" comment? Earlier in this chapter, we identified two major audiences for reference documentation: seekers and stumblers. Seekers know what they want; stumblers don't. The key win for seekers is a consistently commented codebase so that they can quickly scan an API and find what they are looking for. The key win for stumblers is clearly identifying the purpose of an API, often at the top of a file

header. We'll walk through some code comments in the subsections that follow. The code commenting guidelines that follow apply to C++, but similar rules are in place at Google for other languages.

File comments

Almost all code files at Google must contain a file comment. (Some header files that contain only one utility function, etc., might deviate from this standard.) File comments should begin with a header of the following form:

```
// -----------------------------------------------------------------------------
// str_cat.h
// -----------------------------------------------------------------------------
//
// This header file contains functions for efficiently concatenating and appending
// strings: StrCat() and StrAppend(). Most of the work within these routines is
// actually handled through use of a special AlphaNum type, which was designed
// to be used as a parameter type that efficiently manages conversion to
// strings and avoids copies in the above operations.
…
```

Generally, a file comment should begin with an outline of what's contained in the code you are reading. It should identify the code's main use cases and intended audience (in the preceding case, developers who want to concatenate strings). Any API that cannot be succinctly described in the first paragraph or two is usually the sign of an API that is not well thought out. Consider breaking the API into separate components in those cases.

Class comments

Most modern programming languages are object oriented. Class comments are therefore important for defining the API "objects" in use in a codebase. All public classes (and structs) at Google must contain a class comment describing the class/struct, important methods of that class, and the purpose of the class. Generally, class comments should be "nouned" with documentation emphasizing their object aspect. That is, say, "The Foo class contains x, y, z, allows you to do Bar, and has the following Baz aspects," and so on.

Class comments should generally begin with a comment of the following form:

```
// -----------------------------------------------------------------------------
// AlphaNum
// -----------------------------------------------------------------------------
//
// The AlphaNum class acts as the main parameter type for StrCat() and
// StrAppend(), providing efficient conversion of numeric, boolean, and
// hexadecimal values (through the Hex type) into strings.
```

Function comments

All free functions, or public methods of a class, at Google must also contain a function comment describing what the function *does*. Function comments should stress the *active* nature of their use, beginning with an indicative verb describing what the function does and what is returned.

Function comments should generally begin with a comment of the following form:

```
// StrCat()
//
// Merges the given strings or numbers, using no delimiter(s),
// returning the merged result as a string.
…
```

Note that starting a function comment with a declarative verb introduces consistency across a header file. A seeker can quickly scan an API and read just the verb to get an idea of whether the function is appropriate: "Merges, Deletes, Creates," and so on.

Some documentation styles (and some documentation generators) require various forms of boilerplate on function comments, like "Returns:", "Throws:", and so forth, but at Google we haven't found them to be necessary. It is often clearer to present such information in a single prose comment that's not broken up into artificial section boundaries:

```
// Creates a new record for a customer with the given name and address,
// and returns the record ID, or throws `DuplicateEntryError` if a
// record with that name already exists.
int AddCustomer(string name, string address);
```

Notice how the postcondition, parameters, return value, and exceptional cases are naturally documented together (in this case, in a single sentence), because they are not independent of one another. Adding explicit boilerplate sections would make the comment more verbose and repetitive, but no clearer (and arguably less clear).

Design Docs

Most teams at Google require an approved design document before starting work on any major project. A software engineer typically writes the proposed design document using a specific design doc template approved by the team. Such documents are designed to be collaborative, so they are often shared in Google Docs, which has good collaboration tools. Some teams require such design documents to be discussed and debated at specific team meetings, where the finer points of the design can be discussed or critiqued by experts. In some respects, these design discussions act as a form of code review before any code is written.

Because the development of a design document is one of the first processes an engineer undertakes before deploying a new system, it is also a convenient place to ensure

that various concerns are covered. The canonical design document templates at Google require engineers to consider aspects of their design such as security implications, internationalization, storage requirements and privacy concerns, and so on. In most cases, such parts of those design documents are reviewed by experts in those domains.

A good design document should cover the goals of the design, its implementation strategy, and propose key design decisions with an emphasis on their individual trade-offs. The best design documents suggest design goals and cover alternative designs, denoting their strong and weak points.

A good design document, once approved, also acts not only as a historical record, but as a measure of whether the project successfully achieved its goals. Most teams archive their design documents in an appropriate location within their team documents so that they can review them at a later time. It's often useful to review a design document before a product is launched to ensure that the stated goals when the design document was written remain the stated goals at launch (and if they do not, either the document or the product can be adjusted accordingly).

Tutorials

Every software engineer, when they join a new team, will want to get up to speed as quickly as possible. Having a tutorial that walks someone through the setup of a new project is invaluable; "Hello World" has established itself is one of the best ways to ensure that all team members start off on the right foot. This goes for documents as well as code. Most projects deserve a "Hello World" document that assumes nothing and gets the engineer to make something "real" happen.

Often, the best time to write a tutorial, if one does not yet exist, is when you first join a team. (It's also the best time to find bugs in any existing tutorial you are following.) Get a notepad or other way to take notes, and write down everything you need to do along the way, assuming no domain knowledge or special setup constraints; after you're done, you'll likely know what mistakes you made during the process—and why—and can then edit down your steps to get a more streamlined tutorial. Importantly, write *everything* you need to do along the way; try not to assume any particular setup, permissions, or domain knowledge. If you do need to assume some other setup, state that clearly in the beginning of the tutorial as a set of prerequisites.

Most tutorials require you to perform a number of steps, in order. In those cases, number those steps explicitly. If the focus of the tutorial is on the *user* (say, for external developer documentation), then number each action that a user needs to undertake. Don't number actions that the system may take in response to such user actions. It is critical and important to number explicitly every step when doing this. Nothing is more annoying than an error on step 4 because you forget to tell someone to properly authorize their username, for example.

Example: A bad tutorial

1. Download the package from our server at http://example.com
2. Copy the shell script to your home directory
3. Execute the shell script
4. The foobar system will communicate with the authentication system
5. Once authenticated, foobar will bootstrap a new database named "baz"
6. Test "baz" by executing a SQL command on the command line
7. Type: CREATE DATABASE my_foobar_db;

In the preceding procedure, steps 4 and 5 happen on the server end. It's unclear whether the user needs to do anything, but they don't, so those side effects can be mentioned as part of step 3. As well, it's unclear whether step 6 and step 7 are different. (They aren't.) Combine all atomic user operations into single steps so that the user knows they need to do something at each step in the process. Also, if your tutorial has user-visible input or output, denote that on separate lines (often using the convention of a `monospaced bold` font).

Example: A bad tutorial made better

1. Download the package from our server at *http://example.com*:

   ```
   $ curl -I http://example.com
   ```

2. Copy the shell script to your home directory:

   ```
   $ cp foobar.sh ~
   ```

3. Execute the shell script in your home directory:

   ```
   $ cd ~; foobar.sh
   ```

   The foobar system will first communicate with the authentication system. Once authenticated, foobar will bootstrap a new database named "baz" and open an input shell.

4. Test "baz" by executing a SQL command on the command line:

   ```
   baz:$ CREATE DATABASE my_foobar_db;
   ```

Note how each step requires specific user intervention. If, instead, the tutorial had a focus on some other aspect (e.g., a document about the "life of a server"), number those steps from the perspective of that focus (what the server does).

Conceptual Documentation

Some code requires deeper explanations or insights than can be obtained simply by reading the reference documentation. In those cases, we need conceptual documentation to provide overviews of the APIs or systems. Some examples of conceptual documentation might be a library overview for a popular API, a document describing the life cycle of data within a server, and so on. In almost all cases, a conceptual document is meant to augment, not replace, a reference documentation set. Often this leads to duplication of some information, but with a purpose: to promote clarity. In those cases, it is not necessary for a conceptual document to cover all edge cases (though a reference should cover those cases religiously). In this case, sacrificing some accuracy is acceptable for clarity. The main point of a conceptual document is to impart understanding.

"Concept" documents are the most difficult forms of documentation to write. As a result, they are often the most neglected type of document within a software engineer's toolbox. One problem engineers face when writing conceptual documentation is that it often cannot be embedded directly within the source code because there isn't a canonical location to place it. Some APIs have a relatively broad API surface area, in which case, a file comment might be an appropriate place for a "conceptual" explanation of the API. But often, an API works in conjunction with other APIs and/or modules. The only logical place to document such complex behavior is through a separate conceptual document. If comments are the unit tests of documentation, conceptual documents are the integration tests.

Even when an API is appropriately scoped, it often makes sense to provide a separate conceptual document. For example, Abseil's `StrFormat` library covers a variety of concepts that accomplished users of the API should understand. In those cases, both internally and externally, we provide a format concepts document.

A concept document needs to be useful to a broad audience: both experts and novices alike. Moreover, it needs to emphasize *clarity*, so it often needs to sacrifice completeness (something best reserved for a reference) and (sometimes) strict accuracy. That's not to say a conceptual document should intentionally be inaccurate; it just means that it should focus on common usage and leave rare usages or side effects for reference documentation.

Landing Pages

Most engineers are members of a team, and most teams have a "team page" somewhere on their company's intranet. Often, these sites are a bit of a mess: a typical landing page might contain some interesting links, sometimes several documents titled "read this first!", and some information both for the team and for its customers. Such documents start out useful but rapidly turn into disasters; because they become

so cumbersome to maintain, they will eventually get so obsolete that they will be fixed by only the brave or the desperate.

Luckily, such documents look intimidating, but are actually straightforward to fix: ensure that a landing page clearly identifies its purpose, and then include *only* links to other pages for more information. If something on a landing page is doing more than being a traffic cop, it is *not doing its job*. If you have a separate setup document, link to that from the landing page as a separate document. If you have too many links on the landing page (your page should not scroll multiple screens), consider breaking up the pages by taxonomy, under different sections.

Most poorly configured landing pages serve two different purposes: they are the "goto" page for someone who is a user of your product or API, or they are the home page for a team. Don't have the page serve both masters—it will become confusing. Create a separate "team page" as an internal page apart from the main landing page. What the team needs to know is often quite different than what a customer of your API needs to know.

Documentation Reviews

At Google, all code needs to be reviewed, and our code review process is well understood and accepted. In general, documentation also needs review (though this is less universally accepted). If you want to "test" whether your documentation works, you should generally have someone else review it.

A technical document benefits from three different types of reviews, each emphasizing different aspects:

- A technical review, for accuracy. This review is usually done by a subject matter expert, often another member of your team. Often, this is part of a code review itself.
- An audience review, for clarity. This is usually someone unfamiliar with the domain. This might be someone new to your team or a customer of your API.
- A writing review, for consistency. This is often a technical writer or volunteer.

Of course, some of these lines are sometimes blurred, but if your document is high profile or might end up being externally published, you probably want to ensure that it receives more types of reviews. (We've used a similar review process for this book.) Any document tends to benefit from the aforementioned reviews, even if some of those reviews are ad hoc. That said, even getting one reviewer to review your text is preferable to having no one review it.

Importantly, if documentation is tied into the engineering workflow, it will often improve over time. Most documents at Google now implicitly go through an audience review because at some point, their audience will be using them, and hopefully letting you know when they aren't working (via bugs or other forms of feedback).

Case Study: The Developer Guide Library

As mentioned earlier, there were problems associated with having most (almost all) engineering documentation contained within a shared wiki: little ownership of important documentation, competing documentation, obsolete information, and difficulty in filing bugs or issues with documentation. But this problem was not seen in some documents: the Google C++ style guide was owned by a select group of senior engineers (style arbiters) who managed it. The document was kept in good shape because certain people cared about it. They implicitly owned that document. The document was also canonical: there was only one C++ style guide.

As previously mentioned, documentation that sits directly within source code is one way to promote the establishment of canonical documents; if the documentation sits alongside the source code, it should usually be the most applicable (hopefully). At Google, each API usually has a separate *g3doc* directory where such documents live (written as Markdown files and readable within our Code Search browser). Having the documentation exist alongside the source code not only establishes de facto ownership, it makes the documentation seem more wholly "part" of the code.

Some documentation sets, however, cannot exist very logically within source code. A "C++ developer guide" for Googlers, for example, has no obvious place to sit within the source code. There is no master "C++" directory where people will look for such information. In this case (and others that crossed API boundaries), it became useful to create standalone documentation sets in their own depot. Many of these culled together associated existing documents into a common set, with common navigation and look-and-feel. Such documents were noted as "Developer Guides" and, like the code in the codebase, were under source control in a specific documentation depot, with this depot organized by topic rather than API. Often, technical writers managed these developer guides, because they were better at explaining topics across API boundaries.

Over time, these developer guides became canonical. Users who wrote competing or supplementary documents became amenable to adding their documents to the canonical document set after it was established, and then deprecating their competing documents. Eventually, the C++ style guide became part of a larger "C++ Developer Guide." As the documentation set became more comprehensive and more authoritative, its quality also improved. Engineers began logging bugs because they knew someone was maintaining these documents. Because the documents were locked down under source control, with proper owners, engineers also began sending changelists directly to the technical writers.

The introduction of go/ links (see Chapter 3) allowed most documents to, in effect, more easily establish themselves as canonical on any given topic. Our C++ Developer Guide became established at "go/cpp," for example. With better internal search, go/ links, and the integration of multiple documents into a common documentation set, such canonical documentation sets became more authoritative and robust over time.

Documentation Philosophy

Caveat: the following section is more of a treatise on technical writing best practices (and personal opinion) than of "how Google does it." Consider it optional for software engineers to fully grasp, though understanding these concepts will likely allow you to more easily write technical information.

WHO, WHAT, WHEN, WHERE, and WHY

Most technical documentation answers a "HOW" question. How does this work? How do I program to this API? How do I set up this server? As a result, there's a tendency for software engineers to jump straight into the "HOW" on any given document and ignore the other questions associated with it: the WHO, WHAT, WHEN, WHERE, and WHY. It's true that none of those are generally as important as the HOW—a design document is an exception because an equivalent aspect is often the WHY—but without a proper framing of technical documentation, documents end up confusing. Try to address the other questions in the first two paragraphs of any document:

- WHO was discussed previously: that's the audience. But sometimes you also need to explicitly call out and address the audience in a document. Example: "This document is for new engineers on the Secret Wizard project."
- WHAT identifies the purpose of this document: "This document is a tutorial designed to start a Frobber server in a test environment." Sometimes, merely writing the WHAT helps you frame the document appropriately. If you start adding information that isn't applicable to the WHAT, you might want to move that information into a separate document.
- WHEN identifies when this document was created, reviewed, or updated. Documents in source code have this date noted implicitly, and some other publishing schemes automate this as well. But, if not, make sure to note the date on which the document was written (or last revised) on the document itself.
- WHERE is often implicit as well, but decide where the document should live. Usually, the preference should be under some sort of version control, ideally *with the source code it documents*. But other formats work for different purposes as well. At Google, we often use Google Docs for easy collaboration, particularly on

design issues. At some point, however, any shared document becomes less of a discussion and more of a stable historical record. At that point, move it to someplace more permanent, with clear ownership, version control, and responsibility.

- WHY sets up the purpose for the document. Summarize what you expect someone to take away from the document after reading it. A good rule of thumb is to establish the WHY in the introduction to a document. When you write the summary, verify whether you've met your original expectations (and revise accordingly).

The Beginning, Middle, and End

All documents—indeed, all parts of documents—have a beginning, middle, and end. Although it sounds amazingly silly, most documents should often have, at a minimum, those three sections. A document with only one section has only one thing to say, and very few documents have only one thing to say. Don't be afraid to add sections to your document; they break up the flow into logical pieces and provide readers with a roadmap of what the document covers.

Even the simplest document usually has more than one thing to say. Our popular "C++ Tips of the Week" have traditionally been very short, focusing on one small piece of advice. However, even here, having sections helps. Traditionally, the first section denotes the problem, the middle section goes through the recommended solutions, and the conclusion summarizes the takeaways. Had the document consisted of only one section, some readers would doubtless have difficulty teasing out the important points.

Most engineers loathe redundancy, and with good reason. But in documentation, redundancy is often useful. An important point buried within a wall of text can be difficult to remember or tease out. On the other hand, placing that point at a more prominent location early can lose context provided later on. Usually, the solution is to introduce and summarize the point within an introductory paragraph, and then use the rest of the section to make your case in a more detailed fashion. In this case, redundancy helps the reader understand the importance of what is being stated.

The Parameters of Good Documentation

There are usually three aspects of good documentation: completeness, accuracy, and clarity. You rarely get all three within the same document; as you try to make a document more "complete," for example, clarity can begin to suffer. If you try to document every possible use case of an API, you might end up with an incomprehensible mess. For programming languages, being completely accurate in all cases (and documenting all possible side effects) can also affect clarity. For other documents, trying to be clear about a complicated topic can subtly affect the accuracy of the document; you might decide to ignore some rare side effects in a conceptual document, for example,

because the point of the document is to familiarize someone with the usage of an API, not provide a dogmatic overview of all intended behavior.

In each case, a "good document" is defined as the document that is *doing its intended job*. As a result, you rarely want a document doing more than one job. For each document (and for each document type), decide on its focus and adjust the writing appropriately. Writing a conceptual document? You probably don't need to cover every part of the API. Writing a reference? You probably want this complete, but perhaps must sacrifice some clarity. Writing a landing page? Focus on organization and keep discussion to a minimum. All of this adds up to quality, which, admittedly, is stubbornly difficult to accurately measure.

How can you quickly improve the quality of a document? Focus on the needs of the audience. Often, less is more. For example, one mistake engineers often make is adding design decisions or implementation details to an API document. Much like you should ideally separate the interface from an implementation within a well-designed API, you should avoid discussing design decisions in an API document. Users don't need to know this information. Instead, put those decisions in a specialized document for that purpose (usually a design doc).

Deprecating Documents

Just like old code can cause problems, so can old documents. Over time, documents become stale, obsolete, or (often) abandoned. Try as much as possible to avoid abandoned documents, but when a document no longer serves any purpose, either remove it or identify it as obsolete (and, if available, indicate where to go for new information). Even for unowned documents, someone adding a note that "This no longer works!" is more helpful than saying nothing and leaving something that seems authoritative but no longer works.

At Google, we often attach "freshness dates" to documentation. Such documents note the last time a document was reviewed, and metadata in the documentation set will send email reminders when the document hasn't been touched in, for example, three months. Such freshness dates, as shown in the following example—and tracking your documents as bugs—can help make a documentation set easier to maintain over time, which is the main concern for a document:

```
<!--*
# Document freshness: For more information, see go/fresh-source.
freshness: { owner: `username` reviewed: '2019-02-27' }
*-->
```

Users who own such a document have an incentive to keep that freshness date current (and if the document is under source control, that requires a code review). As a result, it's a low-cost means to ensure that a document is looked over from time to time. At Google, we found that including the owner of a document in this freshness

date within the document itself with a byline of "Last reviewed by..." led to increased adoption as well.

When Do You Need Technical Writers?

When Google was young and growing, there weren't enough technical writers in software engineering. (That's still the case.) Those projects deemed important tended to receive a technical writer, regardless of whether that team really needed one. The idea was that the writer could relieve the team of some of the burden of writing and maintaining documents and (theoretically) allow the important project to achieve greater velocity. This turned out to be a bad assumption.

We learned that most engineering teams can write documentation for themselves (their team) perfectly fine; it's only when they are writing documents for another audience that they tend to need help because it's difficult to write to another audience. The feedback loop within your team regarding documents is more immediate, the domain knowledge and assumptions are clearer, and the perceived needs are more obvious. Of course, a technical writer can often do a better job with grammar and organization, but supporting a single team isn't the best use of a limited and specialized resource; it doesn't scale. It introduced a perverse incentive: become an important project and your software engineers won't need to write documents. Discouraging engineers from writing documents turns out to be the opposite of what you want to do.

Because they are a limited resource, technical writers should generally focus on tasks that software engineers *don't* need to do as part of their normal duties. Usually, this involves writing documents that cross API boundaries. Project Foo might clearly know what documentation Project Foo needs, but it probably has a less clear idea what Project Bar needs. A technical writer is better able to stand in as a person unfamiliar with the domain. In fact, it's one of their critical roles: to challenge the assumptions your team makes about the utility of your project. It's one of the reasons why many, if not most, software engineering technical writers tend to focus on this specific type of API documentation.

Conclusion

Google has made good strides in addressing documentation quality over the past decade, but to be frank, documentation at Google is not yet a first-class citizen. For comparison, engineers have gradually accepted that testing is necessary for any code change, no matter how small. As well, testing tooling is robust, varied and plugged into an engineering workflow at various points. Documentation is not ingrained at nearly the same level.

To be fair, there's not necessarily the same need to address documentation as with testing. Tests can be made atomic (unit tests) and can follow prescribed form and function. Documents, for the most part, cannot. Tests can be automated, and schemes to automate documentation are often lacking. Documents are necessarily subjective; the quality of the document is measured not by the writer, but by the reader, and often quite asynchronously. That said, there is a recognition that documentation is important, and processes around document development are improving. In this author's opinion, the quality of documentation at Google is better than in most software engineering shops.

To change the quality of engineering documentation, engineers—and the entire engineering organization—need to accept that they are both the problem and the solution. Rather than throw up their hands at the state of documentation, they need to realize that producing quality documentation is part of their job and saves them time and effort in the long run. For any piece of code that you expect to live more than a few months, the extra cycles you put in documenting that code will not only help others, it will help you maintain that code as well.

TL;DRs

- Documentation is hugely important over time and scale.
- Documentation changes should leverage the existing developer workflow.
- Keep documents focused on one purpose.
- Write for your audience, not yourself.

CHAPTER 11
Testing Overview

Written by Adam Bender
Edited by Tom Manshreck

Testing has always been a part of programming. In fact, the first time you wrote a computer program you almost certainly threw some sample data at it to see whether it performed as you expected. For a long time, the state of the art in software testing resembled a very similar process, largely manual and error prone. However, since the early 2000s, the software industry's approach to testing has evolved dramatically to cope with the size and complexity of modern software systems. Central to that evolution has been the practice of developer-driven, automated testing.

Automated testing can prevent bugs from escaping into the wild and affecting your users. The later in the development cycle a bug is caught, the more expensive it is; exponentially so in many cases.[1] However, "catching bugs" is only part of the motivation. An equally important reason why you want to test your software is to support the ability to change. Whether you're adding new features, doing a refactoring focused on code health, or undertaking a larger redesign, automated testing can quickly catch mistakes, and this makes it possible to change software with confidence.

Companies that can iterate faster can adapt more rapidly to changing technologies, market conditions, and customer tastes. If you have a robust testing practice, you needn't fear change—you can embrace it as an essential quality of developing software. The more and faster you want to change your systems, the more you need a fast way to test them.

1 See "Defect Prevention: Reducing Costs and Enhancing Quality."

The act of writing tests also improves the design of your systems. As the first clients of your code, a test can tell you much about your design choices. Is your system too tightly coupled to a database? Does the API support the required use cases? Does your system handle all of the edge cases? Writing automated tests forces you to confront these issues early on in the development cycle. Doing so generally leads to more modular software that enables greater flexibility later on.

Much ink has been spilled about the subject of testing software, and for good reason: for such an important practice, doing it well still seems to be a mysterious craft to many. At Google, while we have come a long way, we still face difficult problems getting our processes to scale reliably across the company. In this chapter, we'll share what we have learned to help further the conversation.

Why Do We Write Tests?

To better understand how to get the most out of testing, let's start from the beginning. When we talk about automated testing, what are we really talking about?

The simplest test is defined by:

- A single behavior you are testing, usually a method or API that you are calling
- A specific input, some value that you pass to the API
- An observable output or behavior
- A controlled environment such as a single isolated process

When you execute a test like this, passing the input to the system and verifying the output, you will learn whether the system behaves as you expect. Taken in aggregate, hundreds or thousands of simple tests (usually called a *test suite*) can tell you how well your entire product conforms to its intended design and, more important, when it doesn't.

Creating and maintaining a healthy test suite takes real effort. As a codebase grows, so too will the test suite. It will begin to face challenges like instability and slowness. A failure to address these problems will cripple a test suite. Keep in mind that tests derive their value from the trust engineers place in them. If testing becomes a productivity sink, constantly inducing toil and uncertainty, engineers will lose trust and begin to find workarounds. A bad test suite can be worse than no test suite at all.

In addition to empowering companies to build great products quickly, testing is becoming critical to ensuring the safety of important products and services in our lives. Software is more involved in our lives than ever before, and defects can cause

more than a little annoyance: they can cost massive amounts of money, loss of property, or, worst of all, loss of life.[2]

At Google, we have determined that testing cannot be an afterthought. Focusing on quality and testing is part of how we do our jobs. We have learned, sometimes painfully, that failing to build quality into our products and services inevitably leads to bad outcomes. As a result, we have built testing into the heart of our engineering culture.

The Story of Google Web Server

In Google's early days, engineer-driven testing was often assumed to be of little importance. Teams regularly relied on smart people to get the software right. A few systems ran large integration tests, but mostly it was the Wild West. One product in particular seemed to suffer the worst: it was called the Google Web Server, also known as GWS.

GWS is the web server responsible for serving Google Search queries and is as important to Google Search as air traffic control is to an airport. Back in 2005, as the project swelled in size and complexity, productivity had slowed dramatically. Releases were becoming buggier, and it was taking longer and longer to push them out. Team members had little confidence when making changes to the service, and often found out something was wrong only when features stopped working in production. (At one point, more than 80% of production pushes contained user-affecting bugs that had to be rolled back.)

To address these problems, the tech lead (TL) of GWS decided to institute a policy of engineer-driven, automated testing. As part of this policy, all new code changes were required to include tests, and those tests would be run continuously. Within a year of instituting this policy, the number of emergency pushes *dropped by half*. This drop occurred despite the fact that the project was seeing a record number of new changes every quarter. Even in the face of unprecedented growth and change, testing brought renewed productivity and confidence to one of the most critical projects at Google. Today, GWS has tens of thousands of tests, and releases almost every day with relatively few customer-visible failures.

The changes in GWS marked a watershed for testing culture at Google as teams in other parts of the company saw the benefits of testing and moved to adopt similar tactics.

2 See "Failure at Dhahran."

One of the key insights the GWS experience taught us was that you can't rely on programmer ability alone to avoid product defects. Even if each engineer writes only the occasional bug, after you have enough people working on the same project, you will be swamped by the ever-growing list of defects. Imagine a hypothetical 100-person team whose engineers are so good that they each write only a single bug a month. Collectively, this group of amazing engineers still produces five new bugs every workday. Worse yet, in a complex system, fixing one bug can often cause another, as engineers adapt to known bugs and code around them.

The best teams find ways to turn the collective wisdom of its members into a benefit for the entire team. That is exactly what automated testing does. After an engineer on the team writes a test, it is added to the pool of common resources available to others. Everyone else on the team can now run the test and will benefit when it detects an issue. Contrast this with an approach based on debugging, wherein each time a bug occurs, an engineer must pay the cost of digging into it with a debugger. The cost in engineering resources is night and day and was the fundamental reason GWS was able to turn its fortunes around.

Testing at the Speed of Modern Development

Software systems are growing larger and ever more complex. A typical application or service at Google is made up of thousands or millions of lines of code. It uses hundreds of libraries or frameworks and must be delivered via unreliable networks to an increasing number of platforms running with an uncountable number of configurations. To make matters worse, new versions are pushed to users frequently, sometimes multiple times each day. This is a far cry from the world of shrink-wrapped software that saw updates only once or twice a year.

The ability for humans to manually validate every behavior in a system has been unable to keep pace with the explosion of features and platforms in most software. Imagine what it would take to manually test all of the functionality of Google Search, like finding flights, movie times, relevant images, and of course web search results (see Figure 11-1). Even if you can determine how to solve that problem, you then need to multiply that workload by every language, country, and device Google Search must support, and don't forget to check for things like accessibility and security. Attempting to assess product quality by asking humans to manually interact with every feature just doesn't scale. When it comes to testing, there is one clear answer: automation.

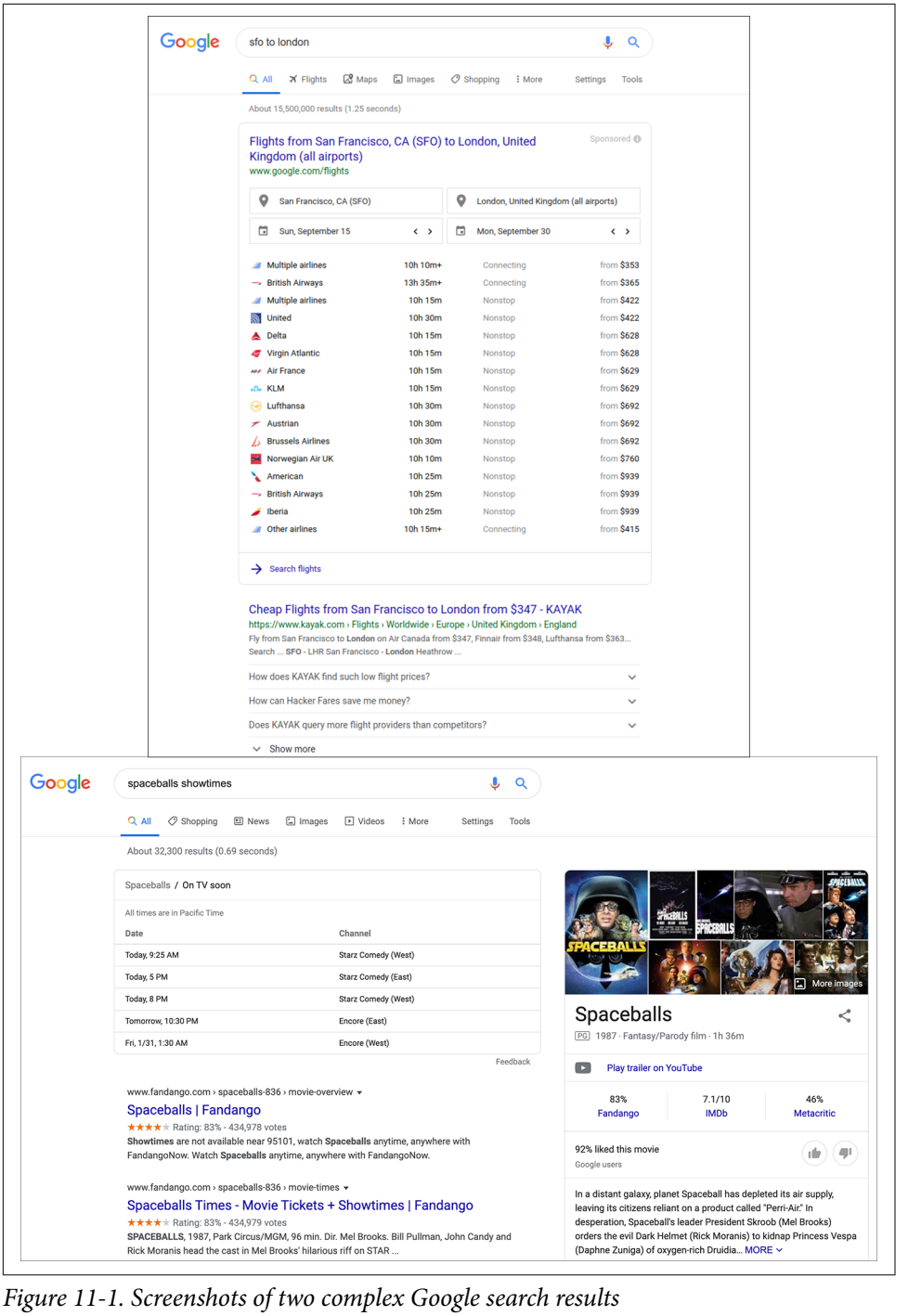

Figure 11-1. Screenshots of two complex Google search results

Write, Run, React

In its purest form, automating testing consists of three activities: writing tests, running tests, and reacting to test failures. An automated test is a small bit of code, usually a single function or method, that calls into an isolated part of a larger system that you want to test. The test code sets up an expected environment, calls into the system, usually with a known input, and verifies the result. Some of the tests are very small, exercising a single code path; others are much larger and can involve entire systems, like a mobile operating system or web browser.

Example 11-1 presents a deliberately simple test in Java using no frameworks or testing libraries. This is not how you would write an entire test suite, but at its core every automated test looks similar to this very simple example.

Example 11-1. An example test

```
// Verifies a Calculator class can handle negative results.
public void main(String[] args) {
   Calculator calculator = new Calculator();
   int expectedResult = -3;
   int actualResult = calculator.subtract(2, 5); // Given 2, Subtracts 5.
   assert(expectedResult == actualResult);
}
```

Unlike the QA processes of yore, in which rooms of dedicated software testers pored over new versions of a system, exercising every possible behavior, the engineers who build systems today play an active and integral role in writing and running automated tests for their own code. Even in companies where QA is a prominent organization, developer-written tests are commonplace. At the speed and scale that today's systems are being developed, the only way to keep up is by sharing the development of tests around the entire engineering staff.

Of course, writing tests is different from writing *good tests*. It can be quite difficult to train tens of thousands of engineers to write good tests. We will discuss what we have learned about writing good tests in the chapters that follow.

Writing tests is only the first step in the process of automated testing. After you have written tests, you need to run them. Frequently. At its core, automated testing consists of repeating the same action over and over, only requiring human attention when something breaks. We will discuss this Continuous Integration (CI) and testing in Chapter 23. By expressing tests as code instead of a manual series of steps, we can run them every time the code changes—easily thousands of times per day. Unlike human testers, machines never grow tired or bored.

Another benefit of having tests expressed as code is that it is easy to modularize them for execution in various environments. Testing the behavior of Gmail in Firefox

requires no more effort than doing so in Chrome, provided you have configurations for both of these systems.[3] Running tests for a user interface (UI) in Japanese or German can be done using the same test code as for English.

Products and services under active development will inevitably experience test failures. What really makes a testing process effective is how it addresses test failures. Allowing failing tests to pile up quickly defeats any value they were providing, so it is imperative not to let that happen. Teams that prioritize fixing a broken test within minutes of a failure are able to keep confidence high and failure isolation fast, and therefore derive more value out of their tests.

In summary, a healthy automated testing culture encourages everyone to share the work of writing tests. Such a culture also ensures that tests are run regularly. Last, and perhaps most important, it places an emphasis on fixing broken tests quickly so as to maintain high confidence in the process.

Benefits of Testing Code

To developers coming from organizations that don't have a strong testing culture, the idea of writing tests as a means of improving productivity and velocity might seem antithetical. After all, the act of writing tests can take just as long (if not longer!) than implementing a feature would take in the first place. On the contrary, at Google, we've found that investing in software tests provides several key benefits to developer productivity:

Less debugging

As you would expect, tested code has fewer defects when it is submitted. Critically, it also has fewer defects throughout its existence; most of them will be caught before the code is submitted. A piece of code at Google is expected to be modified dozens of times in its lifetime. It will be changed by other teams and even automated code maintenance systems. A test written once continues to pay dividends and prevent costly defects and annoying debugging sessions through the lifetime of the project. Changes to a project, or the dependencies of a project, that break a test can be quickly detected by test infrastructure and rolled back before the problem is ever released to production.

Increased confidence in changes

All software changes. Teams with good tests can review and accept changes to their project with confidence because all important behaviors of their project are continuously verified. Such projects encourage refactoring. Changes that refactor

3 Getting the behavior right across different browsers and languages is a different story! But, ideally, the end-user experience should be the same for everyone.

code while preserving existing behavior should (ideally) require no changes to existing tests.

Improved documentation
: Software documentation is notoriously unreliable. From outdated requirements to missing edge cases, it is common for documentation to have a tenuous relationship to the code. Clear, focused tests that exercise one behavior at a time function as executable documentation. If you want to know what the code does in a particular case, look at the test for that case. Even better, when requirements change and new code breaks an existing test, we get a clear signal that the "documentation" is now out of date. Note that tests work best as documentation only if care is taken to keep them clear and concise.

Simpler reviews
: All code at Google is reviewed by at least one other engineer before it can be submitted (see Chapter 9 for more details). A code reviewer spends less effort verifying code works as expected if the code review includes thorough tests that demonstrate code correctness, edge cases, and error conditions. Instead of the tedious effort needed to mentally walk each case through the code, the reviewer can verify that each case has a passing test.

Thoughtful design
: Writing tests for new code is a practical means of exercising the API design of the code itself. If new code is difficult to test, it is often because the code being tested has too many responsibilities or difficult-to-manage dependencies. Well-designed code should be modular, avoiding tight coupling and focusing on specific responsibilities. Fixing design issues early often means less rework later.

Fast, high-quality releases
: With a healthy automated test suite, teams can release new versions of their application with confidence. Many projects at Google release a new version to production every day—even large projects with hundreds of engineers and thousands of code changes submitted every day. This would not be possible without automated testing.

Designing a Test Suite

Today, Google operates at a massive scale, but we haven't always been so large, and the foundations of our approach were laid long ago. Over the years, as our codebase has grown, we have learned a lot about how to approach the design and execution of a test suite, often by making mistakes and cleaning up afterward.

One of the lessons we learned fairly early on is that engineers favored writing larger, system-scale tests, but that these tests were slower, less reliable, and more difficult to debug than smaller tests. Engineers, fed up with debugging the system-scale tests,

asked themselves, "Why can't we just test one server at a time?" or, "Why do we need to test a whole server at once? We could test smaller modules individually." Eventually, the desire to reduce pain led teams to develop smaller and smaller tests, which turned out to be faster, more stable, and generally less painful.

This led to a lot of discussion around the company about the exact meaning of "small." Does small mean unit test? What about integration tests, what size are those? We have come to the conclusion that there are two distinct dimensions for every test case: size and scope. Size refers to the resources that are required to run a test case: things like memory, processes, and time. Scope refers to the specific code paths we are verifying. Note that executing a line of code is different from verifying that it worked as expected. Size and scope are interrelated but distinct concepts.

Test Size

At Google, we classify every one of our tests into a size and encourage engineers to always write the smallest possible test for a given piece of functionality. A test's size is determined not by its number of lines of code, but by how it runs, what it is allowed to do, and how many resources it consumes. In fact, in some cases, our definitions of small, medium, and large are actually encoded as constraints the testing infrastructure can enforce on a test. We go into the details in a moment, but in brief, *small tests* run in a single process, *medium tests* run on a single machine, and *large tests* run wherever they want, as demonstrated in Figure 11-2.[4]

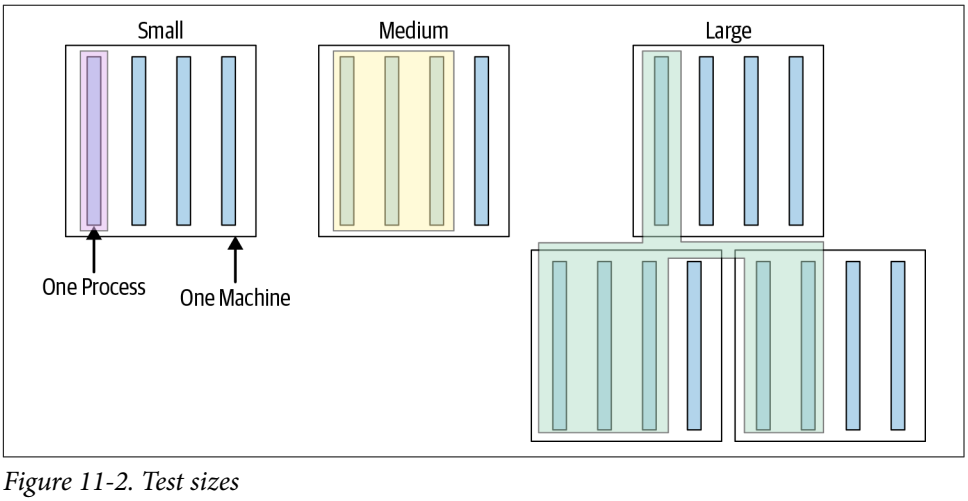

Figure 11-2. Test sizes

4 Technically, we have four sizes of test at Google: small, medium, large, and *enormous*. The internal difference between large and enormous is actually subtle and historical; so, in this book, most descriptions of large actually apply to our notion of enormous.

We make this distinction, as opposed to the more traditional "unit" or "integration," because the most important qualities we want from our test suite are speed and determinism, regardless of the scope of the test. Small tests, regardless of the scope, are almost always faster and more deterministic than tests that involve more infrastructure or consume more resources. Placing restrictions on small tests makes speed and determinism much easier to achieve. As test sizes grow, many of the restrictions are relaxed. Medium tests have more flexibility but also more risk of nondeterminism. Larger tests are saved for only the most complex and difficult testing scenarios. Let's take a closer look at the exact constraints imposed on each type of test.

Small tests

Small tests are the most constrained of the three test sizes. The primary constraint is that small tests must run in a single process. In many languages, we restrict this even further to say that they must run on a single thread. This means that the code performing the test must run in the same process as the code being tested. You can't run a server and have a separate test process connect to it. It also means that you can't run a third-party program such as a database as part of your test.

The other important constraints on small tests are that they aren't allowed to sleep, perform I/O operations,[5] or make any other blocking calls. This means that small tests aren't allowed to access the network or disk. Testing code that relies on these sorts of operations requires the use of test doubles (see Chapter 13) to replace the heavyweight dependency with a lightweight, in-process dependency.

The purpose of these restrictions is to ensure that small tests don't have access to the main sources of test slowness or nondeterminism. A test that runs on a single process and never makes blocking calls can effectively run as fast as the CPU can handle. It's difficult (but certainly not impossible) to accidentally make such a test slow or nondeterministic. The constraints on small tests provide a sandbox that prevents engineers from shooting themselves in the foot.

These restrictions might seem excessive at first, but consider a modest suite of a couple hundred small test cases running throughout the day. If even a few of them fail nondeterministically (often called flaky tests), tracking down the cause becomes a serious drain on productivity. At Google's scale, such a problem could grind our testing infrastructure to a halt.

At Google, we encourage engineers to try to write small tests whenever possible, regardless of the scope of the test, because it keeps the entire test suite running fast and reliably. For more discussion on small versus unit tests, see Chapter 12.

5 There is a little wiggle room in this policy. Tests are allowed to access a filesystem if they use a hermetic, in-memory implementation.

Medium tests

The constraints placed on small tests can be too restrictive for many interesting kinds of tests. The next rung up the ladder of test sizes is the medium test. Medium tests can span multiple processes, use threads, and can make blocking calls, including network calls, to `localhost`. The only remaining restriction is that medium tests aren't allowed to make network calls to any system other than `localhost`. In other words, the test must be contained within a single machine.

The ability to run multiple processes opens up a lot of possibilities. For example, you could run a database instance to validate that the code you're testing integrates correctly in a more realistic setting. Or you could test a combination of web UI and server code. Tests of web applications often involve tools like WebDriver that start a real browser and control it remotely via the test process.

Unfortunately, with increased flexibility comes increased potential for tests to become slow and nondeterministic. Tests that span processes or are allowed to make blocking calls are dependent on the operating system and third-party processes to be fast and deterministic, which isn't something we can guarantee in general. Medium tests still provide a bit of protection by preventing access to remote machines via the network, which is far and away the biggest source of slowness and nondeterminism in most systems. Still, when writing medium tests, the "safety" is off, and engineers need to be much more careful.

Large tests

Finally, we have large tests. Large tests remove the `localhost` restriction imposed on medium tests, allowing the test and the system being tested to span across multiple machines. For example, the test might run against a system in a remote cluster.

As before, increased flexibility comes with increased risk. Having to deal with a system that spans multiple machines and the network connecting them increases the chance of slowness and nondeterminism significantly compared to running on a single machine. We mostly reserve large tests for full-system end-to-end tests that are more about validating configuration than pieces of code, and for tests of legacy components for which it is impossible to use test doubles. We'll talk more about use cases for large tests in Chapter 14. Teams at Google will frequently isolate their large tests from their small or medium tests, running them only during the build and release process so as not to impact developer workflow.

Case Study: Flaky Tests Are Expensive

If you have a few thousand tests, each with a very tiny bit of nondeterminism, running all day, occasionally one will probably fail (flake). As the number of tests grows, statistically so will the number of flakes. If each test has even a 0.1% of failing when it should not, and you run 10,000 tests per day, you will be investigating 10 flakes per day. Each investigation takes time away from something more productive that your team could be doing.

In some cases, you can limit the impact of flaky tests by automatically rerunning them when they fail. This is effectively trading CPU cycles for engineering time. At low levels of flakiness, this trade-off makes sense. Just keep in mind that rerunning a test is only delaying the need to address the root cause of flakiness.

If test flakiness continues to grow, you will experience something much worse than lost productivity: a loss of confidence in the tests. It doesn't take needing to investigate many flakes before a team loses trust in the test suite. After that happens, engineers will stop reacting to test failures, eliminating any value the test suite provided. Our experience suggests that as you approach 1% flakiness, the tests begin to lose value. At Google, our flaky rate hovers around 0.15%, which implies thousands of flakes every day. We fight hard to keep flakes in check, including actively investing engineering hours to fix them.

In most cases, flakes appear because of nondeterministic behavior in the tests themselves. Software provides many sources of nondeterminism: clock time, thread scheduling, network latency, and more. Learning how to isolate and stabilize the effects of randomness is not easy. Sometimes, effects are tied to low-level concerns like hardware interrupts or browser rendering engines. A good automated test infrastructure should help engineers identify and mitigate any nondeterministic behavior.

Properties common to all test sizes

All tests should strive to be hermetic: a test should contain all of the information necessary to set up, execute, and tear down its environment. Tests should assume as little as possible about the outside environment, such as the order in which the tests are run. For example, they should not rely on a shared database. This constraint becomes more challenging with larger tests, but effort should still be made to ensure isolation.

A test should contain *only* the information required to exercise the behavior in question. Keeping tests clear and simple aids reviewers in verifying that the code does what it says it does. Clear code also aids in diagnosing failure when they fail. We like to say that "a test should be obvious upon inspection." Because there are no tests for the tests themselves, they require manual review as an important check on correctness. As a corollary to this, we also strongly discourage the use of control flow state-

ments like conditionals and loops in a test. More complex test flows risk containing bugs themselves and make it more difficult to determine the cause of a test failure.

Remember that tests are often revisited only when something breaks. When you are called to fix a broken test that you have never seen before, you will be thankful someone took the time to make it easy to understand. Code is read far more than it is written, so make sure you write the test you'd like to read!

Test sizes in practice. Having precise definitions of test sizes has allowed us to create tools to enforce them. Enforcement enables us to scale our test suites and still make certain guarantees about speed, resource utilization, and stability. The extent to which these definitions are enforced at Google varies by language. For example, we run all Java tests using a custom security manager that will cause all tests tagged as small to fail if they attempt to do something prohibited, such as establish a network connection.

Test Scope

Though we at Google put a lot of emphasis on test size, another important property to consider is test scope. Test scope refers to how much code is being validated by a given test. Narrow-scoped tests (commonly called "unit tests") are designed to validate the logic in a small, focused part of the codebase, like an individual class or method. Medium-scoped tests (commonly called *integration tests*) are designed to verify interactions between a small number of components; for example, between a server and its database. Large-scoped tests (commonly referred to by names like *functional tests*, *end-to-end* tests, or *system tests*) are designed to validate the interaction of several distinct parts of the system, or emergent behaviors that aren't expressed in a single class or method.

It's important to note that when we talk about unit tests as being narrowly scoped, we're referring to the code that is being *validated*, not the code that is being *executed*. It's quite common for a class to have many dependencies or other classes it refers to, and these dependencies will naturally be invoked while testing the target class. Though some other testing strategies make heavy use of test doubles (fakes or mocks) to avoid executing code outside of the system under test, at Google, we prefer to keep the real dependencies in place when it is feasible to do so. Chapter 13 discusses this issue in more detail.

Narrow-scoped tests tend to be small, and broad-scoped tests tend to be medium or large, but this isn't always the case. For example, it's possible to write a broad-scoped test of a server endpoint that covers all of its normal parsing, request validation, and business logic, which is nevertheless small because it uses doubles to stand in for all out-of-process dependencies like a database or filesystem. Similarly, it's possible to write a narrow-scoped test of a single method that must be medium sized. For

example, modern web frameworks often bundle HTML and JavaScript together, and testing a UI component like a date picker often requires running an entire browser, even to validate a single code path.

Just as we encourage tests of smaller size, at Google, we also encourage engineers to write tests of narrower scope. As a very rough guideline, we tend to aim to have a mix of around 80% of our tests being narrow-scoped unit tests that validate the majority of our business logic; 15% medium-scoped integration tests that validate the interactions between two or more components; and 5% end-to-end tests that validate the entire system. Figure 11-3 depicts how we can visualize this as a pyramid.

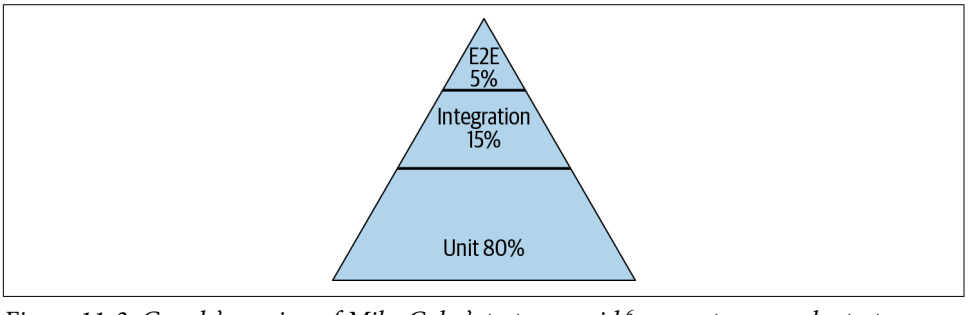

Figure 11-3. Google's version of Mike Cohn's test pyramid;[6] percentages are by test case count, and every team's mix will be a little different

Unit tests form an excellent base because they are fast, stable, and dramatically narrow the scope and reduce the cognitive load required to identify all the possible behaviors a class or function has. Additionally, they make failure diagnosis quick and painless. Two antipatterns to be aware of are the "ice cream cone" and the "hourglass," as illustrated in Figure 11-4.

With the ice cream cone, engineers write many end-to-end tests but few integration or unit tests. Such suites tend to be slow, unreliable, and difficult to work with. This pattern often appears in projects that start as prototypes and are quickly rushed to production, never stopping to address testing debt.

The hourglass involves many end-to-end tests and many unit tests but few integration tests. It isn't quite as bad as the ice cream cone, but it still results in many end-to-end test failures that could have been caught quicker and more easily with a suite of medium-scope tests. The hourglass pattern occurs when tight coupling makes it difficult to instantiate individual dependencies in isolation.

6 Mike Cohn, *Succeeding with Agile: Software Development Using Scrum* (New York: Addison-Wesley Professional, 2009).

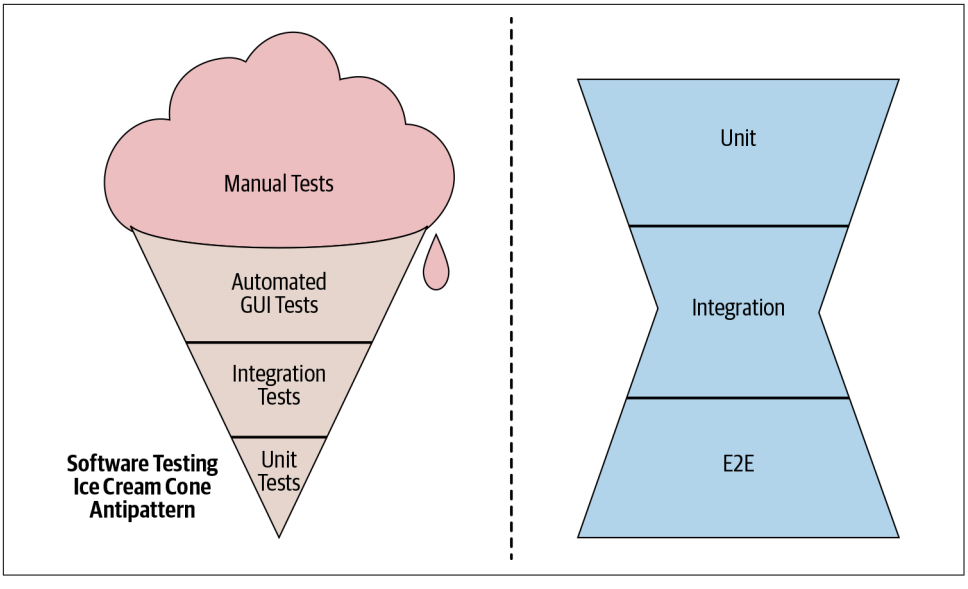

Figure 11-4. Test suite antipatterns

Our recommended mix of tests is determined by our two primary goals: engineering productivity and product confidence. Favoring unit tests gives us high confidence quickly, and early in the development process. Larger tests act as sanity checks as the product develops; they should not be viewed as a primary method for catching bugs.

When considering your own mix, you might want a different balance. If you emphasize integration testing, you might discover that your test suites take longer to run but catch more issues between components. When you emphasize unit tests, your test suites can complete very quickly, and you will catch many common logic bugs. But, unit tests cannot verify the interactions between components, like a contract between two systems developed by different teams. A good test suite contains a blend of different test sizes and scopes that are appropriate to the local architectural and organizational realities.

The Beyoncé Rule

We are often asked, when coaching new hires, which behaviors or properties actually need to be tested? The straightforward answer is: test everything that you don't want to break. In other words, if you want to be confident that a system exhibits a particular behavior, the only way to be sure it will is to write an automated test for it. This includes all of the usual suspects like testing performance, behavioral correctness, accessibility, and security. It also includes less obvious properties like testing how a system handles failure.

We have a name for this general philosophy: we call it the Beyoncé Rule. Succinctly, it can be stated as follows: "If you liked it, then you shoulda put a test on it." The Beyoncé Rule is often invoked by infrastructure teams that are responsible for making changes across the entire codebase. If unrelated infrastructure changes pass all of your tests but still break your team's product, you are on the hook for fixing it and adding the additional tests.

Testing for Failure

One of the most important situations a system must account for is failure. Failure is inevitable, but waiting for an actual catastrophe to find out how well a system responds to a catastrophe is a recipe for pain. Instead of waiting for a failure, write automated tests that simulate common kinds of failures. This includes simulating exceptions or errors in unit tests and injecting Remote Procedure Call (RPC) errors or latency in integration and end-to-end tests. It can also include much larger disruptions that affect the real production network using techniques like Chaos Engineering. A predictable and controlled response to adverse conditions is a hallmark of a reliable system.

A Note on Code Coverage

Code coverage is a measure of which lines of feature code are exercised by which tests. If you have 100 lines of code and your tests execute 90 of them, you have 90% code coverage.[7] Code coverage is often held up as the gold standard metric for understanding test quality, and that is somewhat unfortunate. It is possible to exercise a lot of lines of code with a few tests, never checking that each line is doing anything useful. That's because code coverage only measures that a line was invoked, not what happened as a result. (We recommend only measuring coverage from small tests to avoid coverage inflation that occurs when executing larger tests.)

An even more insidious problem with code coverage is that, like other metrics, it quickly becomes a goal unto itself. It is common for teams to establish a bar for expected code coverage—for instance, 80%. At first, that sounds eminently reasonable; surely you want to have at least that much coverage. In practice, what happens is that instead of treating 80% like a floor, engineers treat it like a ceiling. Soon, changes begin landing with no more than 80% coverage. After all, why do more work than the metric requires?

7 Keep in mind that there are different kinds of coverage (line, path, branch, etc.), and each says something different about which code has been tested. In this simple example, line coverage is being used.

A better way to approach the quality of your test suite is to think about the behaviors that are tested. Do you have confidence that everything your customers expect to work will work? Do you feel confident you can catch breaking changes in your dependencies? Are your tests stable and reliable? Questions like these are a more holistic way to think about a test suite. Every product and team is going to be different; some will have difficult-to-test interactions with hardware, some involve massive datasets. Trying to answer the question "do we have enough tests?" with a single number ignores a lot of context and is unlikely to be useful. Code coverage can provide some insight into untested code, but it is not a substitute for thinking critically about how well your system is tested.

Testing at Google Scale

Much of the guidance to this point can be applied to codebases of almost any size. However, we should spend some time on what we have learned testing at our very large scale. To understand how testing works at Google, you need an understanding of our development environment, the most important fact about which is that most of Google's code is kept in a single, monolithic repository (monorepo). Almost every line of code for every product and service we operate is all stored in one place. We have more than two billion lines of code in the repository today.

Google's codebase experiences close to 25 million lines of change every week. Roughly half of them are made by the tens of thousands of engineers working in our monorepo, and the other half by our automated systems, in the form of configuration updates or large-scale changes (Chapter 22). Many of those changes are initiated from outside the immediate project. We don't place many limitations on the ability of engineers to reuse code.

The openness of our codebase encourages a level of co-ownership that lets everyone take responsibility for the codebase. One benefit of such openness is the ability to directly fix bugs in a product or service you use (subject to approval, of course) instead of complaining about it. This also implies that many people will make changes in a part of the codebase owned by someone else.

Another thing that makes Google a little different is that almost no teams use repository branching. All changes are committed to the repository head and are immediately visible for everyone to see. Furthermore, all software builds are performed using the last committed change that our testing infrastructure has validated. When a product or service is built, almost every dependency required to run it is also built from source, also from the head of the repository. Google manages testing at this scale by use of a CI system. One of the key components of our CI system is our Test Automated Platform (TAP).

For more information on TAP and our CI philosophy, see Chapter 23.

Whether you are considering our size, our monorepo, or the number of products we offer, Google's engineering environment is complex. Every week it experiences millions of changing lines, billions of test cases being run, tens of thousands of binaries being built, and hundreds of products being updated—talk about complicated!

The Pitfalls of a Large Test Suite

As a codebase grows, you will inevitably need to make changes to existing code. When poorly written, automated tests can make it more difficult to make those changes. Brittle tests—those that over-specify expected outcomes or rely on extensive and complicated boilerplate—can actually resist change. These poorly written tests can fail even when unrelated changes are made.

If you have ever made a five-line change to a feature only to find dozens of unrelated, broken tests, you have felt the friction of brittle tests. Over time, this friction can make a team reticent to perform necessary refactoring to keep a codebase healthy. The subsequent chapters will cover strategies that you can use to improve the robustness and quality of your tests.

Some of the worst offenders of brittle tests come from the misuse of mock objects. Google's codebase has suffered so badly from an abuse of mocking frameworks that it has led some engineers to declare "no more mocks!" Although that is a strong statement, understanding the limitations of mock objects can help you avoid misusing them.

For more information on working effectively with mock objects, see Chapter 13.

In addition to the friction caused by brittle tests, a larger suite of tests will be slower to run. The slower a test suite, the less frequently it will be run, and the less benefit it provides. We use a number of techniques to speed up our test suite, including parallelizing execution and using faster hardware. However, these kinds of tricks are eventually swamped by a large number of individually slow test cases.

Tests can become slow for many reasons, like booting significant portions of a system, firing up an emulator before execution, processing large datasets, or waiting for disparate systems to synchronize. Tests often start fast enough but slow down as the

system grows. For example, maybe you have an integration test exercising a single dependency that takes five seconds to respond, but over the years you grow to depend on a dozen services, and now the same tests take five minutes.

Tests can also become slow due to unnecessary speed limits introduced by functions like `sleep()` and `setTimeout()`. Calls to these functions are often used as naive heuristics before checking the result of nondeterministic behavior. Sleeping for half a second here or there doesn't seem too dangerous at first; however, if a "wait-and-check" is embedded in a widely used utility, pretty soon you have added minutes of idle time to every run of your test suite. A better solution is to actively poll for a state transition with a frequency closer to microseconds. You can combine this with a timeout value in case a test fails to reach a stable state.

Failing to keep a test suite deterministic and fast ensures it will become roadblock to productivity. At Google, engineers who encounter these tests have found ways to work around slowdowns, with some going as far as to skip the tests entirely when submitting changes. Obviously, this is a risky practice and should be discouraged, but if a test suite is causing more harm than good, eventually engineers will find a way to get their job done, tests or no tests.

The secret to living with a large test suite is to treat it with respect. Incentivize engineers to care about their tests; reward them as much for having rock-solid tests as you would for having a great feature launch. Set appropriate performance goals and refactor slow or marginal tests. Basically, treat your tests like production code. When simple changes begin taking nontrivial time, spend effort making your tests less brittle.

In addition to developing the proper culture, invest in your testing infrastructure by developing linters, documentation, or other assistance that makes it more difficult to write bad tests. Reduce the number of frameworks and tools you need to support to increase the efficiency of the time you invest to improve things.[8] If you don't invest in making it easy to manage your tests, eventually engineers will decide it isn't worth having them at all.

History of Testing at Google

Now that we've discussed how Google approaches testing, it might be enlightening to learn how we got here. As mentioned previously, Google's engineers didn't always embrace the value of automated testing. In fact, until 2005, testing was closer to a curiosity than a disciplined practice. Most of the testing was done manually, if it was done at all. However, from 2005 to 2006, a testing revolution occurred and changed

8 Each supported language at Google has one standard test framework and one standard mocking/stubbing library. One set of infrastructure runs most tests in all languages across the entire codebase.

the way we approach software engineering. Its effects continue to reverberate within the company to this day.

The experience of the GWS project, which we discussed at the opening of this chapter, acted as a catalyst. It made it clear how powerful automated testing could be. Following the improvements to GWS in 2005, the practices began spreading across the entire company. The tooling was primitive. However, the volunteers, who came to be known as the Testing Grouplet, didn't let that slow them down.

Three key initiatives helped usher automated testing into the company's consciousness: Orientation Classes, the Test Certified program, and Testing on the Toilet. Each one had influence in a completely different way, and together they reshaped Google's engineering culture.

Orientation Classes

Even though much of the early engineering staff at Google eschewed testing, the pioneers of automated testing at Google knew that at the rate the company was growing, new engineers would quickly outnumber existing team members. If they could reach all the new hires in the company, it could be an extremely effective avenue for introducing cultural change. Fortunately, there was, and still is, a single choke point that all new engineering hires pass through: orientation.

Most of Google's early orientation program concerned things like medical benefits and how Google Search worked, but starting in 2005 it also began including an hour-long discussion of the value of automated testing.[9] The class covered the various benefits of testing, such as increased productivity, better documentation, and support for refactoring. It also covered how to write a good test. For many Nooglers (new Googlers) at the time, such a class was their first exposure to this material. Most important, all of these ideas were presented as though they were standard practice at the company. The new hires had no idea that they were being used as trojan horses to sneak this idea into their unsuspecting teams.

As Nooglers joined their teams following orientation, they began writing tests and questioning those on the team who didn't. Within only a year or two, the population of engineers who had been taught testing outnumbered the pretesting culture engineers. As a result, many new projects started off on the right foot.

Testing has now become more widely practiced in the industry, so most new hires arrive with the expectations of automated testing firmly in place. Nonetheless, orientation classes continue to set expectations about testing and connect what Nooglers

9 This class was so successful that an updated version is still taught today. In fact, it is one of the longest-running orientation classes in the company's history.

know about testing outside of Google to the challenges of doing so in our very large and very complex codebase.

Test Certified

Initially, the larger and more complex parts of our codebase appeared resistant to good testing practices. Some projects had such poor code quality that they were almost impossible to test. To give projects a clear path forward, the Testing Grouplet devised a certification program that they called Test Certified. Test Certified aimed to give teams a way to understand the maturity of their testing processes and, more critically, cookbook instructions on how to improve it.

The program was organized into five levels, and each level required some concrete actions to improve the test hygiene on the team. The levels were designed in such a way that each step up could be accomplished within a quarter, which made it a convenient fit for Google's internal planning cadence.

Test Certified Level 1 covered the basics: set up a continuous build; start tracking code coverage; classify all your tests as small, medium, or large; identify (but don't necessarily fix) flaky tests; and create a set of fast (not necessarily comprehensive) tests that can be run quickly. Each subsequent level added more challenges like "no releases with broken tests" or "remove all nondeterministic tests." By Level 5, all tests were automated, fast tests were running before every commit, all nondeterminism had been removed, and every behavior was covered. An internal dashboard applied social pressure by showing the level of every team. It wasn't long before teams were competing with one another to climb the ladder.

By the time the Test Certified program was replaced by an automated approach in 2015 (more on pH later), it had helped more than 1,500 projects improve their testing culture.

Testing on the Toilet

Of all the methods the Testing Grouplet used to try to improve testing at Google, perhaps none was more off-beat than Testing on the Toilet (TotT). The goal of TotT was fairly simple: actively raise awareness about testing across the entire company. The question is, what's the best way to do that in a company with employees scattered around the world?

The Testing Grouplet considered the idea of a regular email newsletter, but given the heavy volume of email everyone deals with at Google, it was likely to become lost in the noise. After a little bit of brainstorming, someone proposed the idea of posting flyers in the restroom stalls as a joke. We quickly recognized the genius in it: the bathroom is one place that everyone must visit at least once each day, no matter what. Joke or not, the idea was cheap enough to implement that it had to be tried.

In April 2006, a short writeup covering how to improve testing in Python appeared in restroom stalls across Google. This first episode was posted by a small band of volunteers. To say the reaction was polarized is an understatement; some saw it as an invasion of personal space, and they objected strongly. Mailing lists lit up with complaints, but the TotT creators were content: the people complaining were still talking about testing.

Ultimately, the uproar subsided and TotT quickly became a staple of Google culture. To date, engineers from across the company have produced several hundred episodes, covering almost every aspect of testing imaginable (in addition to a variety of other technical topics). New episodes are eagerly anticipated and some engineers even volunteer to post the episodes around their own buildings. We intentionally limit each episode to exactly one page, challenging authors to focus on the most important and actionable advice. A good episode contains something an engineer can take back to the desk immediately and try.

Ironically for a publication that appears in one of the more private locations, TotT has had an outsized public impact. Most external visitors see an episode at some point in their visit, and such encounters often lead to funny conversations about how Googlers always seem to be thinking about code. Additionally, TotT episodes make great blog posts, something the original TotT authors recognized early on. They began publishing lightly edited versions publicly, helping to share our experience with the industry at large.

Despite starting as a joke, TotT has had the longest run and the most profound impact of any of the testing initiatives started by the Testing Grouplet.

Testing Culture Today

Testing culture at Google today has come a long way from 2005. Nooglers still attend orientation classes on testing, and TotT continues to be distributed almost weekly. However, the expectations of testing have more deeply embedded themselves in the daily developer workflow.

Every code change at Google is required to go through code review. And every change is expected to include both the feature code and tests. Reviewers are expected to review the quality and correctness of both. In fact, it is perfectly reasonable to block a change if it is missing tests.

As a replacement for Test Certified, one of our engineering productivity teams recently launched a tool called Project Health (pH). The pH tool continuously gathers dozens of metrics on the health of a project, including test coverage and test latency, and makes them available internally. pH is measured on a scale of one (worst) to five (best). A pH-1 project is seen as a problem for the team to address. Almost every team that runs a continuous build automatically gets a pH score.

Over time, testing has become an integral part of Google's engineering culture. We have myriad ways to reinforce its value to engineers across the company. Through a combination of training, gentle nudges, mentorship, and, yes, even a little friendly competition, we have created the clear expectation that testing is everyone's job.

Why didn't we start by mandating the writing of tests?

The Testing Grouplet had considered asking for a testing mandate from senior leadership but quickly decided against it. Any mandate on how to develop code would be seriously counter to Google culture and likely slow the progress, independent of the idea being mandated. The belief was that successful ideas would spread, so the focus became demonstrating success.

If engineers were deciding to write tests on their own, it meant that they had fully accepted the idea and were likely to keep doing the right thing—even if no one was compelling them to.

The Limits of Automated Testing

Automated testing is not suitable for all testing tasks. For example, testing the quality of search results often involves human judgment. We conduct targeted, internal studies using Search Quality Raters who execute real queries and record their impressions. Similarly, it is difficult to capture the nuances of audio and video quality in an automated test, so we often use human judgment to evaluate the performance of telephony or video-calling systems.

In addition to qualitative judgements, there are certain creative assessments at which humans excel. For example, searching for complex security vulnerabilities is something that humans do better than automated systems. After a human has discovered and understood a flaw, it can be added to an automated security testing system like Google's Cloud Security Scanner where it can be run continuously and at scale.

A more generalized term for this technique is Exploratory Testing. Exploratory Testing is a fundamentally creative endeavor in which someone treats the application under test as a puzzle to be broken, maybe by executing an unexpected set of steps or by inserting unexpected data. When conducting an exploratory test, the specific problems to be found are unknown at the start. They are gradually uncovered by probing commonly overlooked code paths or unusual responses from the application. As with the detection of security vulnerabilities, as soon as an exploratory test discovers an issue, an automated test should be added to prevent future regressions.

Using automated testing to cover well-understood behaviors enables the expensive and qualitative efforts of human testers to focus on the parts of your products for which they can provide the most value—and avoid boring them to tears in the process.

Conclusion

The adoption of developer-driven automated testing has been one of the most transformational software engineering practices at Google. It has enabled us to build larger systems with larger teams, faster than we ever thought possible. It has helped us keep up with the increasing pace of technological change. Over the past 15 years, we have successfully transformed our engineering culture to elevate testing into a cultural norm. Despite the company growing by a factor of almost 100 times since the journey began, our commitment to quality and testing is stronger today than it has ever been.

This chapter has been written to help orient you to how Google thinks about testing. In the next few chapters, we are going to dive even deeper into some key topics that have helped shape our understanding of what it means to write good, stable, and reliable tests. We will discuss the what, why, and how of unit tests, the most common kind of test at Google. We will wade into the debate on how to effectively use test doubles in tests through techniques such as faking, stubbing, and interaction testing. Finally, we will discuss the challenges with testing larger and more complex systems, like many of those we have at Google.

At the conclusion of these three chapters, you should have a much deeper and clearer picture of the testing strategies we use and, more important, why we use them.

TL;DRs

- Automated testing is foundational to enabling software to change.
- For tests to scale, they must be automated.
- A balanced test suite is necessary for maintaining healthy test coverage.
- "If you liked it, you should have put a test on it."
- Changing the testing culture in organizations takes time.

CHAPTER 12
Unit Testing

Written by Erik Kuefler
Edited by Tom Manshreck

The previous chapter introduced two of the main axes along which Google classifies tests: *size* and *scope*. To recap, size refers to the resources consumed by a test and what it is allowed to do, and scope refers to how much code a test is intended to validate. Though Google has clear definitions for test size, scope tends to be a little fuzzier. We use the term *unit test* to refer to tests of relatively narrow scope, such as of a single class or method. Unit tests are usually small in size, but this isn't always the case.

After preventing bugs, the most important purpose of a test is to improve engineers' productivity. Compared to broader-scoped tests, unit tests have many properties that make them an excellent way to optimize productivity:

- They tend to be small according to Google's definitions of test size. Small tests are fast and deterministic, allowing developers to run them frequently as part of their workflow and get immediate feedback.
- They tend to be easy to write at the same time as the code they're testing, allowing engineers to focus their tests on the code they're working on without having to set up and understand a larger system.
- They promote high levels of test coverage because they are quick and easy to write. High test coverage allows engineers to make changes with confidence that they aren't breaking anything.
- They tend to make it easy to understand what's wrong when they fail because each test is conceptually simple and focused on a particular part of the system.
- They can serve as documentation and examples, showing engineers how to use the part of the system being tested and how that system is intended to work.

Due to their many advantages, most tests written at Google are unit tests, and as a rule of thumb, we encourage engineers to aim for a mix of about 80% unit tests and 20% broader-scoped tests. This advice, coupled with the ease of writing unit tests and the speed with which they run, means that engineers run a *lot* of unit tests—it's not at all unusual for an engineer to execute thousands of unit tests (directly or indirectly) during the average workday.

Because they make up such a big part of engineers' lives, Google puts a lot of focus on *test maintainability*. Maintainable tests are ones that "just work": after writing them, engineers don't need to think about them again until they fail, and those failures indicate real bugs with clear causes. The bulk of this chapter focuses on exploring the idea of maintainability and techniques for achieving it.

The Importance of Maintainability

Imagine this scenario: Mary wants to add a simple new feature to the product and is able to implement it quickly, perhaps requiring only a couple dozen lines of code. But when she goes to check in her change, she gets a screen full of errors back from the automated testing system. She spends the rest of the day going through those failures one by one. In each case, the change introduced no actual bug, but broke some of the assumptions that the test made about the internal structure of the code, requiring those tests to be updated. Often, she has difficulty figuring out what the tests were trying to do in the first place, and the hacks she adds to fix them make those tests even more difficult to understand in the future. Ultimately, what should have been a quick job ends up taking hours or even days of busywork, killing Mary's productivity and sapping her morale.

Here, testing had the opposite of its intended effect by draining productivity rather than improving it while not meaningfully increasing the quality of the code under test. This scenario is far too common, and Google engineers struggle with it every day. There's no magic bullet, but many engineers at Google have been working to develop sets of patterns and practices to alleviate these problems, which we encourage the rest of the company to follow.

The problems Mary ran into weren't her fault, and there was nothing she could have done to avoid them: bad tests must be fixed before they are checked in, lest they impose a drag on future engineers. Broadly speaking, the issues she encountered fall into two categories. First, the tests she was working with were *brittle*: they broke in response to a harmless and unrelated change that introduced no real bugs. Second, the tests were *unclear*: after they were failing, it was difficult to determine what was wrong, how to fix it, and what those tests were supposed to be doing in the first place.

Preventing Brittle Tests

As just defined, a brittle test is one that fails in the face of an unrelated change to production code that does not introduce any real bugs.[1] Such tests must be diagnosed and fixed by engineers as part of their work. In small codebases with only a few engineers, having to tweak a few tests for every change might not be a big problem. But if a team regularly writes brittle tests, test maintenance will inevitably consume a larger and larger proportion of the team's time as they are forced to comb through an increasing number of failures in an ever-growing test suite. If a set of tests needs to be manually tweaked by engineers for each change, calling it an "automated test suite" is a bit of a stretch!

Brittle tests cause pain in codebases of any size, but they become particularly acute at Google's scale. An individual engineer might easily run thousands of tests in a single day during the course of their work, and a single large-scale change (see Chapter 22) can trigger hundreds of thousands of tests. At this scale, spurious breakages that affect even a small percentage of tests can waste huge amounts of engineering time. Teams at Google vary quite a bit in terms of how brittle their test suites are, but we've identified a few practices and patterns that tend to make tests more robust to change.

Strive for Unchanging Tests

Before talking about patterns for avoiding brittle tests, we need to answer a question: just how often should we expect to need to change a test after writing it? Any time spent updating old tests is time that can't be spent on more valuable work. Therefore, *the ideal test is unchanging*: after it's written, it never needs to change unless the requirements of the system under test change.

What does this look like in practice? We need to think about the kinds of changes that engineers make to production code and how we should expect tests to respond to those changes. Fundamentally, there are four kinds of changes:

Pure refactorings

When an engineer refactors the internals of a system without modifying its interface, whether for performance, clarity, or any other reason, the system's tests shouldn't need to change. The role of tests in this case is to ensure that the refactoring didn't change the system's behavior. Tests that need to be changed during a refactoring indicate that either the change is affecting the system's behavior and isn't a pure refactoring, or that the tests were not written at an appropriate level

1 Note that this is slightly different from a *flaky test*, which fails nondeterministically without any change to production code.

of abstraction. Google's reliance on large-scale changes (described in Chapter 22) to do such refactorings makes this case particularly important for us.

New features
: When an engineer adds new features or behaviors to an existing system, the system's existing behaviors should remain unaffected. The engineer must write new tests to cover the new behaviors, but they shouldn't need to change any existing tests. As with refactorings, a change to existing tests when adding new features suggest unintended consequences of that feature or inappropriate tests.

Bug fixes
: Fixing a bug is much like adding a new feature: the presence of the bug suggests that a case was missing from the initial test suite, and the bug fix should include that missing test case. Again, bug fixes typically shouldn't require updates to existing tests.

Behavior changes
: Changing a system's existing behavior is the one case when we expect to have to make updates to the system's existing tests. Note that such changes tend to be significantly more expensive than the other three types. A system's users are likely to rely on its current behavior, and changes to that behavior require coordination with those users to avoid confusion or breakages. Changing a test in this case indicates that we're breaking an explicit contract of the system, whereas changes in the previous cases indicate that we're breaking an unintended contract. Low-level libraries will often invest significant effort in avoiding the need to ever make a behavior change so as not to break their users.

The takeaway is that after you write a test, you shouldn't need to touch that test again as you refactor the system, fix bugs, or add new features. This understanding is what makes it possible to work with a system at scale: expanding it requires writing only a small number of new tests related to the change you're making rather than potentially having to touch every test that has ever been written against the system. Only breaking changes in a system's behavior should require going back to change its tests, and in such situations, the cost of updating those tests tends to be small relative to the cost of updating all of the system's users.

Test via Public APIs

Now that we understand our goal, let's look at some practices for making sure that tests don't need to change unless the requirements of the system being tested change. By far the most important way to ensure this is to write tests that invoke the system being tested in the same way its users would; that is, make calls against its public API rather than its implementation details. If tests work the same way as the system's users, by definition, change that breaks a test might also break a user. As an additional bonus, such tests can serve as useful examples and documentation for users.

Consider Example 12-1, which validates a transaction and saves it to a database.

Example 12-1. A transaction API

```
public void processTransaction(Transaction transaction) {
  if (isValid(transaction)) {
    saveToDatabase(transaction);
  }
}

private boolean isValid(Transaction t) {
  return t.getAmount() < t.getSender().getBalance();
}

private void saveToDatabase(Transaction t) {
  String s = t.getSender() + "," + t.getRecipient() + "," + t.getAmount();
  database.put(t.getId(), s);
}

public void setAccountBalance(String accountName, int balance) {
  // Write the balance to the database directly
}

public void getAccountBalance(String accountName) {
  // Read transactions from the database to determine the account balance
}
```

A tempting way to test this code would be to remove the "private" visibility modifiers and test the implementation logic directly, as demonstrated in Example 12-2.

Example 12-2. A naive test of a transaction API's implementation

```
@Test
public void emptyAccountShouldNotBeValid() {
  assertThat(processor.isValid(newTransaction().setSender(EMPTY_ACCOUNT)))
      .isFalse();
}

@Test
public void shouldSaveSerializedData() {
  processor.saveToDatabase(newTransaction()
      .setId(123)
      .setSender("me")
      .setRecipient("you")
      .setAmount(100));
  assertThat(database.get(123)).isEqualTo("me,you,100");
}
```

This test interacts with the transaction processor in a much different way than its real users would: it peers into the system's internal state and calls methods that aren't pub-

licly exposed as part of the system's API. As a result, the test is brittle, and almost any refactoring of the system under test (such as renaming its methods, factoring them out into a helper class, or changing the serialization format) would cause the test to break, even if such a change would be invisible to the class's real users.

Instead, the same test coverage can be achieved by testing only against the class's public API, as shown in Example 12-3.[2]

Example 12-3. Testing the public API

```
@Test
public void shouldTransferFunds() {
  processor.setAccountBalance("me", 150);
  processor.setAccountBalance("you", 20);

  processor.processTransaction(newTransaction()
      .setSender("me")
      .setRecipient("you")
      .setAmount(100));

  assertThat(processor.getAccountBalance("me")).isEqualTo(50);
  assertThat(processor.getAccountBalance("you")).isEqualTo(120);
}

@Test
public void shouldNotPerformInvalidTransactions() {
  processor.setAccountBalance("me", 50);
  processor.setAccountBalance("you", 20);

  processor.processTransaction(newTransaction()
      .setSender("me")
      .setRecipient("you")
      .setAmount(100));

  assertThat(processor.getAccountBalance("me")).isEqualTo(50);
  assertThat(processor.getAccountBalance("you")).isEqualTo(20);
}
```

Tests using only public APIs are, by definition, accessing the system under test in the same manner that its users would. Such tests are more realistic and less brittle because they form explicit contracts: if such a test breaks, it implies that an existing user of the system will also be broken. Testing only these contracts means that you're free to do whatever internal refactoring of the system you want without having to worry about making tedious changes to tests.

2 This is sometimes called the "Use the front door first principle."

It's not always clear what constitutes a "public API," and the question really gets to the heart of what a "unit" is in unit testing. Units can be as small as an individual function or as broad as a set of several related packages/modules. When we say "public API" in this context, we're really talking about the API exposed by that unit to third parties outside of the team that owns the code. This doesn't always align with the notion of visibility provided by some programming languages; for example, classes in Java might define themselves as "public" to be accessible by other packages in the same unit but are not intended for use by other parties outside of the unit. Some languages like Python have no built-in notion of visibility (often relying on conventions like prefixing private method names with underscores), and build systems like Bazel can further restrict who is allowed to depend on APIs declared public by the programming language.

Defining an appropriate scope for a unit and hence what should be considered the public API is more art than science, but here are some rules of thumb:

- If a method or class exists only to support one or two other classes (i.e., it is a "helper class"), it probably shouldn't be considered its own unit, and its functionality should be tested through those classes instead of directly.
- If a package or class is designed to be accessible by anyone without having to consult with its owners, it almost certainly constitutes a unit that should be tested directly, where its tests access the unit in the same way that the users would.
- If a package or class can be accessed only by the people who own it, but it is designed to provide a general piece of functionality useful in a range of contexts (i.e., it is a "support library"), it should also be considered a unit and tested directly. This will usually create some redundancy in testing given that the support library's code will be covered both by its own tests and the tests of its users. However, such redundancy can be valuable: without it, a gap in test coverage could be introduced if one of the library's users (and its tests) were ever removed.

At Google, we've found that engineers sometimes need to be persuaded that testing via public APIs is better than testing against implementation details. The reluctance is understandable because it's often much easier to write tests focused on the piece of code you just wrote rather than figuring out how that code affects the system as a whole. Nevertheless, we have found it valuable to encourage such practices, as the extra upfront effort pays for itself many times over in reduced maintenance burden. Testing against public APIs won't completely prevent brittleness, but it's the most important thing you can do to ensure that your tests fail only in the event of meaningful changes to your system.

Test State, Not Interactions

Another way that tests commonly depend on implementation details involves not which methods of the system the test calls, but how the results of those calls are verified. In general, there are two ways to verify that a system under test behaves as expected. With *state testing*, you observe the system itself to see what it looks like after invoking with it. With *interaction testing*, you instead check that the system took an expected sequence of actions on its collaborators in response to invoking it. Many tests will perform a combination of state and interaction validation.

Interaction tests tend to be more brittle than state tests for the same reason that it's more brittle to test a private method than to test a public method: interaction tests check *how* a system arrived at its result, whereas usually you should care only *what* the result is. Example 12-4 illustrates a test that uses a test double (explained further in Chapter 13) to verify how a system interacts with a database.

Example 12-4. A brittle interaction test

```
@Test
public void shouldWriteToDatabase() {
  accounts.createUser("foobar");
  verify(database).put("foobar");
}
```

The test verifies that a specific call was made against a database API, but there are a couple different ways it could go wrong:

- If a bug in the system under test causes the record to be deleted from the database shortly after it was written, the test will pass even though we would have wanted it to fail.
- If the system under test is refactored to call a slightly different API to write an equivalent record, the test will fail even though we would have wanted it to pass.

It's much less brittle to directly test against the state of the system, as demonstrated in Example 12-5.

Example 12-5. Testing against state

```
@Test
public void shouldCreateUsers() {
  accounts.createUser("foobar");
  assertThat(accounts.getUser("foobar")).isNotNull();
}
```

This test more accurately expresses what we care about: the state of the system under test after interacting with it.

The most common reason for problematic interaction tests is an over reliance on mocking frameworks. These frameworks make it easy to create test doubles that record and verify every call made against them, and to use those doubles in place of real objects in tests. This strategy leads directly to brittle interaction tests, and so we tend to prefer the use of real objects in favor of mocked objects, as long as the real objects are fast and deterministic.

For a more extensive discussion of test doubles and mocking frameworks, when they should be used, and safer alternatives, see Chapter 13.

Writing Clear Tests

Sooner or later, even if we've completely avoided brittleness, our tests will fail. Failure is a good thing—test failures provide useful signals to engineers, and are one of the main ways that a unit test provides value.

Test failures happen for one of two reasons:[3]

- The system under test has a problem or is incomplete. This result is exactly what tests are designed for: alerting you to bugs so that you can fix them.
- The test itself is flawed. In this case, nothing is wrong with the system under test, but the test was specified incorrectly. If this was an existing test rather than one that you just wrote, this means that the test is brittle. The previous section discussed how to avoid brittle tests, but it's rarely possible to eliminate them entirely.

When a test fails, an engineer's first job is to identify which of these cases the failure falls into and then to diagnose the actual problem. The speed at which the engineer can do so depends on the test's *clarity*. A clear test is one whose purpose for existing and reason for failing is immediately clear to the engineer diagnosing a failure. Tests fail to achieve clarity when their reasons for failure aren't obvious or when it's difficult to figure out why they were originally written. Clear tests also bring other benefits, such as documenting the system under test and more easily serving as a basis for new tests.

Test clarity becomes significant over time. Tests will often outlast the engineers who wrote them, and the requirements and understanding of a system will shift subtly as it ages. It's entirely possible that a failing test might have been written years ago by an

3 These are also the same two reasons that a test can be "flaky." Either the system under test has a nondeterministic fault, or the test is flawed such that it sometimes fails when it should pass.

engineer no longer on the team, leaving no way to figure out its purpose or how to fix it. This stands in contrast with unclear production code, whose purpose you can usually determine with enough effort by looking at what calls it and what breaks when it's removed. With an unclear test, you might never understand its purpose, since removing the test will have no effect other than (potentially) introducing a subtle hole in test coverage.

In the worst case, these obscure tests just end up getting deleted when engineers can't figure out how to fix them. Not only does removing such tests introduce a hole in test coverage, but it also indicates that the test has been providing zero value for perhaps the entire period it has existed (which could have been years).

For a test suite to scale and be useful over time, it's important that each individual test in that suite be as clear as possible. This section explores techniques and ways of thinking about tests to achieve clarity.

Make Your Tests Complete and Concise

Two high-level properties that help tests achieve clarity are completeness and conciseness. A test is *complete* when its body contains all of the information a reader needs in order to understand how it arrives at its result. A test is *concise* when it contains no other distracting or irrelevant information. Example 12-6 shows a test that is neither complete nor concise:

Example 12-6. An incomplete and cluttered test

```
@Test
public void shouldPerformAddition() {
  Calculator calculator = new Calculator(new RoundingStrategy(),
      "unused", ENABLE_COSINE_FEATURE, 0.01, calculusEngine, false);
  int result = calculator.calculate(newTestCalculation());
  assertThat(result).isEqualTo(5); // Where did this number come from?
}
```

The test is passing a lot of irrelevant information into the constructor, and the actual important parts of the test are hidden inside of a helper method. The test can be made more complete by clarifying the inputs of the helper method, and more concise by using another helper to hide the irrelevant details of constructing the calculator, as illustrated in Example 12-7.

Example 12-7. A complete, concise test

```
@Test
public void shouldPerformAddition() {
  Calculator calculator = newCalculator();
  int result = calculator.calculate(newCalculation(2, Operation.PLUS, 3));
```

```
  assertThat(result).isEqualTo(5);
}
```

Ideas we discuss later, especially around code sharing, will tie back to completeness and conciseness. In particular, it can often be worth violating the DRY (Don't Repeat Yourself) principle if it leads to clearer tests. Remember: a *test's body should contain all of the information needed to understand it without containing any irrelevant or distracting information*.

Test Behaviors, Not Methods

The first instinct of many engineers is to try to match the structure of their tests to the structure of their code such that every production method has a corresponding test method. This pattern can be convenient at first, but over time it leads to problems: as the method being tested grows more complex, its test also grows in complexity and becomes more difficult to reason about. For example, consider the snippet of code in Example 12-8, which displays the results of a transaction.

Example 12-8. A transaction snippet

```
public void displayTransactionResults(User user, Transaction transaction) {
  ui.showMessage("You bought a " + transaction.getItemName());
  if (user.getBalance() < LOW_BALANCE_THRESHOLD) {
    ui.showMessage("Warning: your balance is low!");
  }
}
```

It wouldn't be uncommon to find a test covering both of the messages that might be shown by the method, as presented in Example 12-9.

Example 12-9. A method-driven test

```
@Test
public void testDisplayTransactionResults() {
  transactionProcessor.displayTransactionResults(
      newUserWithBalance(
          LOW_BALANCE_THRESHOLD.plus(dollars(2))),
      new Transaction("Some Item", dollars(3)));

  assertThat(ui.getText()).contains("You bought a Some Item");
  assertThat(ui.getText()).contains("your balance is low");
}
```

With such tests, it's likely that the test started out covering only the first method. Later, an engineer expanded the test when the second message was added (violating the idea of unchanging tests that we discussed earlier). This modification sets a bad precedent: as the method under test becomes more complex and implements more

functionality, its unit test will become increasingly convoluted and grow more and more difficult to work with.

The problem is that framing tests around methods can naturally encourage unclear tests because a single method often does a few different things under the hood and might have several tricky edge and corner cases. There's a better way: rather than writing a test for each method, write a test for each *behavior*.[4] A behavior is any guarantee that a system makes about how it will respond to a series of inputs while in a particular state.[5] Behaviors can often be expressed using the words "given," "when," and "then": "*Given* that a bank account is empty, *when* attempting to withdraw money from it, *then* the transaction is rejected." The mapping between methods and behaviors is many-to-many: most nontrivial methods implement multiple behaviors, and some behaviors rely on the interaction of multiple methods. The previous example can be rewritten using behavior-driven tests, as presented in Example 12-10.

Example 12-10. A behavior-driven test

```
@Test
public void displayTransactionResults_showsItemName() {
  transactionProcessor.displayTransactionResults(
      new User(), new Transaction("Some Item"));
  assertThat(ui.getText()).contains("You bought a Some Item");
}

@Test
public void displayTransactionResults_showsLowBalanceWarning() {
  transactionProcessor.displayTransactionResults(
      newUserWithBalance(
          LOW_BALANCE_THRESHOLD.plus(dollars(2))),
      new Transaction("Some Item", dollars(3)));
  assertThat(ui.getText()).contains("your balance is low");
}
```

The extra boilerplate required to split apart the single test is more than worth it, and the resulting tests are much clearer than the original test. Behavior-driven tests tend to be clearer than method-oriented tests for several reasons. First, they read more like natural language, allowing them to be naturally understood rather than requiring laborious mental parsing. Second, they more clearly express cause and effect because each test is more limited in scope. Finally, the fact that each test is short and descriptive makes it easier to see what functionality is already tested and encourages engineers to add new streamlined test methods instead of piling onto existing methods.

4 See *https://testing.googleblog.com/2014/04/testing-on-toilet-test-behaviors-not.html* and *https://dannorth.net/introducing-bdd*.

5 Furthermore, a *feature* (in the product sense of the word) can be expressed as a collection of behaviors.

Structure tests to emphasize behaviors

Thinking about tests as being coupled to behaviors instead of methods significantly affects how they should be structured. Remember that every behavior has three parts: a "given" component that defines how the system is set up, a "when" component that defines the action to be taken on the system, and a "then" component that validates the result.[6] Tests are clearest when this structure is explicit. Some frameworks like Cucumber and Spock directly bake in given/when/then. Other languages can use whitespace and optional comments to make the structure stand out, such as that shown in Example 12-11.

Example 12-11. A well-structured test

```
@Test
public void transferFundsShouldMoveMoneyBetweenAccounts() {
  // Given two accounts with initial balances of $150 and $20
  Account account1 = newAccountWithBalance(usd(150));
  Account account2 = newAccountWithBalance(usd(20));

  // When transferring $100 from the first to the second account
  bank.transferFunds(account1, account2, usd(100));

  // Then the new account balances should reflect the transfer
  assertThat(account1.getBalance()).isEqualTo(usd(50));
  assertThat(account2.getBalance()).isEqualTo(usd(120));
}
```

This level of description isn't always necessary in trivial tests, and it's usually sufficient to omit the comments and rely on whitespace to make the sections clear. However, explicit comments can make more sophisticated tests easier to understand. This pattern makes it possible to read tests at three levels of granularity:

1. A reader can start by looking at the test method name (discussed below) to get a rough description of the behavior being tested.
2. If that's not enough, the reader can look at the given/when/then comments for a formal description of the behavior.
3. Finally, a reader can look at the actual code to see precisely how that behavior is expressed.

This pattern is most commonly violated by interspersing assertions among multiple calls to the system under test (i.e., combining the "when" and "then" blocks). Merging

6 These components are sometimes referred to as "arrange," "act," and "assert."

the "then" and "when" blocks in this way can make the test less clear because it makes it difficult to distinguish the action being performed from the expected result.

When a test does want to validate each step in a multistep process, it's acceptable to define alternating sequences of when/then blocks. Long blocks can also be made more descriptive by splitting them up with the word "and." Example 12-12 shows what a relatively complex, behavior-driven test might look like.

Example 12-12. Alternating when/then blocks within a test

```
@Test
public void shouldTimeOutConnections() {
  // Given two users
  User user1 = newUser();
  User user2 = newUser();

  // And an empty connection pool with a 10-minute timeout
  Pool pool = newPool(Duration.minutes(10));

  // When connecting both users to the pool
  pool.connect(user1);
  pool.connect(user2);

  // Then the pool should have two connections
  assertThat(pool.getConnections()).hasSize(2);

  // When waiting for 20 minutes
  clock.advance(Duration.minutes(20));

  // Then the pool should have no connections
  assertThat(pool.getConnections()).isEmpty();

  // And each user should be disconnected
  assertThat(user1.isConnected()).isFalse();
  assertThat(user2.isConnected()).isFalse();
}
```

When writing such tests, be careful to ensure that you're not inadvertently testing multiple behaviors at the same time. Each test should cover only a single behavior, and the vast majority of unit tests require only one "when" and one "then" block.

Name tests after the behavior being tested

Method-oriented tests are usually named after the method being tested (e.g., a test for the `updateBalance` method is usually called `testUpdateBalance`). With more focused behavior-driven tests, we have a lot more flexibility and the chance to convey useful information in the test's name. The test name is very important: it will often be the first or only token visible in failure reports, so it's your best opportunity to communi-

cate the problem when the test breaks. It's also the most straightforward way to express the intent of the test.

A test's name should summarize the behavior it is testing. A good name describes both the actions that are being taken on a system and the expected outcome. Test names will sometimes include additional information like the state of the system or its environment before taking action on it. Some languages and frameworks make this easier than others by allowing tests to be nested within one another and named using strings, such as in Example 12-13, which uses Jasmine.

Example 12-13. Some sample nested naming patterns

```
describe("multiplication", function() {
  describe("with a positive number", function() {
    var positiveNumber = 10;
    it("is positive with another positive number", function() {
      expect(positiveNumber * 10).toBeGreaterThan(0);
    });
    it("is negative with a negative number", function() {
      expect(positiveNumber * -10).toBeLessThan(0);
    });
  });
  describe("with a negative number", function() {
    var negativeNumber = 10;
    it("is negative with a positive number", function() {
      expect(negativeNumber * 10).toBeLessThan(0);
    });
    it("is positive with another negative number", function() {
      expect(negativeNumber * -10).toBeGreaterThan(0);
    });
  });
});
```

Other languages require us to encode all of this information in a method name, leading to method naming patterns like that shown in Example 12-14.

Example 12-14. Some sample method naming patterns

```
multiplyingTwoPositiveNumbersShouldReturnAPositiveNumber
multiply_postiveAndNegative_returnsNegative
divide_byZero_throwsException
```

Names like this are much more verbose than we'd normally want to write for methods in production code, but the use case is different: we never need to write code that calls these, and their names frequently need to be read by humans in reports. Hence, the extra verbosity is warranted.

Many different naming strategies are acceptable so long as they're used consistently within a single test class. A good trick if you're stuck is to try starting the test name with the word "should." When taken with the name of the class being tested, this naming scheme allows the test name to be read as a sentence. For example, a test of a `BankAccount` class named `shouldNotAllowWithdrawalsWhenBalanceIsEmpty` can be read as "BankAccount should not allow withdrawals when balance is empty." By reading the names of all the test methods in a suite, you should get a good sense of the behaviors implemented by the system under test. Such names also help ensure that the test stays focused on a single behavior: if you need to use the word "and" in a test name, there's a good chance that you're actually testing multiple behaviors and should be writing multiple tests!

Don't Put Logic in Tests

Clear tests are trivially correct upon inspection; that is, it is obvious that a test is doing the correct thing just from glancing at it. This is possible in test code because each test needs to handle only a particular set of inputs, whereas production code must be generalized to handle any input. For production code, we're able to write tests that ensure complex logic is correct. But test code doesn't have that luxury—if you feel like you need to write a test to verify your test, something has gone wrong!

Complexity is most often introduced in the form of *logic*. Logic is defined via the imperative parts of programming languages such as operators, loops, and conditionals. When a piece of code contains logic, you need to do a bit of mental computation to determine its result instead of just reading it off of the screen. It doesn't take much logic to make a test more difficult to reason about. For example, does the test in Example 12-15 look correct to you?

Example 12-15. Logic concealing a bug

```
@Test
public void shouldNavigateToAlbumsPage() {
  String baseUrl = "http://photos.google.com/";
  Navigator nav = new Navigator(baseUrl);
  nav.goToAlbumPage();
  assertThat(nav.getCurrentUrl()).isEqualTo(baseUrl + "/albums");
}
```

There's not much logic here: really just one string concatenation. But if we simplify the test by removing that one bit of logic, a bug immediately becomes clear, as demonstrated in Example 12-16.

Example 12-16. A test without logic reveals the bug

```
@Test
public void shouldNavigateToPhotosPage() {
  Navigator nav = new Navigator("http://photos.google.com/");
  nav.goToPhotosPage();
  assertThat(nav.getCurrentUrl()))
      .isEqualTo("http://photos.google.com//albums"); // Oops!
}
```

When the whole string is written out, we can see right away that we're expecting two slashes in the URL instead of just one. If the production code made a similar mistake, this test would fail to detect a bug. Duplicating the base URL was a small price to pay for making the test more descriptive and meaningful (see the discussion of DAMP versus DRY tests later in this chapter).

If humans are bad at spotting bugs from string concatenation, we're even worse at spotting bugs that come from more sophisticated programming constructs like loops and conditionals. The lesson is clear: in test code, stick to straight-line code over clever logic, and consider tolerating some duplication when it makes the test more descriptive and meaningful. We'll discuss ideas around duplication and code sharing later in this chapter.

Write Clear Failure Messages

One last aspect of clarity has to do not with how a test is written, but with what an engineer sees when it fails. In an ideal world, an engineer could diagnose a problem just from reading its failure message in a log or report without ever having to look at the test itself. A good failure message contains much the same information as the test's name: it should clearly express the desired outcome, the actual outcome, and any relevant parameters.

Here's an example of a bad failure message:

```
Test failed: account is closed
```

Did the test fail because the account was closed, or was the account expected to be closed and the test failed because it wasn't? A better failure message clearly distinguishes the expected from the actual state and gives more context about the result:

```
Expected an account in state CLOSED, but got account:
    <{name: "my-account", state: "OPEN"}
```

Good libraries can help make it easier to write useful failure messages. Consider the assertions in Example 12-17 in a Java test, the first of which uses classical JUnit asserts, and the second of which uses Truth, an assertion library developed by Google:

Example 12-17. An assertion using the Truth library

```
Set<String> colors = ImmutableSet.of("red", "green", "blue");
assertTrue(colors.contains("orange"));  // JUnit
assertThat(colors).contains("orange");  // Truth
```

Because the first assertion only receives a Boolean value, it is only able to give a generic error message like "expected <true> but was <false>," which isn't very informative in a failing test output. Because the second assertion explicitly receives the subject of the assertion, it is able to give a much more useful error message: AssertionError: <[red, green, blue]> should have contained <orange>."

Not all languages have such helpers available, but it should always be possible to manually specify the important information in the failure message. For example, test assertions in Go conventionally look like Example 12-18.

Example 12-18. A test assertion in Go

```
result := Add(2, 3)
if result != 5 {
  t.Errorf("Add(2, 3) = %v, want %v", result, 5)
}
```

Tests and Code Sharing: DAMP, Not DRY

One final aspect of writing clear tests and avoiding brittleness has to do with code sharing. Most software attempts to achieve a principle called DRY—"Don't Repeat Yourself." DRY states that software is easier to maintain if every concept is canonically represented in one place and code duplication is kept to a minimum. This approach is especially valuable in making changes easier because an engineer needs to update only one piece of code rather than tracking down multiple references. The downside

to such consolidation is that it can make code unclear, requiring readers to follow chains of references to understand what the code is doing.

In normal production code, that downside is usually a small price to pay for making code easier to change and work with. But this cost/benefit analysis plays out a little differently in the context of test code. Good tests are designed to be stable, and in fact you usually *want* them to break when the system being tested changes. So DRY doesn't have quite as much benefit when it comes to test code. At the same time, the costs of complexity are greater for tests: production code has the benefit of a test suite to ensure that it keeps working as it becomes complex, whereas tests must stand by themselves, risking bugs if they aren't self-evidently correct. As mentioned earlier, something has gone wrong if tests start becoming complex enough that it feels like they need their own tests to ensure that they're working properly.

Instead of being completely DRY, test code should often strive to be DAMP—that is, to promote "Descriptive And Meaningful Phrases." A little bit of duplication is OK in tests so long as that duplication makes the test simpler and clearer. To illustrate, Example 12-19 presents some tests that are far too DRY.

Example 12-19. A test that is too DRY

```
@Test
public void shouldAllowMultipleUsers() {
  List<User> users = createUsers(false, false);
  Forum forum = createForumAndRegisterUsers(users);
  validateForumAndUsers(forum, users);
}

@Test
public void shouldNotAllowBannedUsers() {
  List<User> users = createUsers(true);
  Forum forum = createForumAndRegisterUsers(users);
  validateForumAndUsers(forum, users);
}

// Lots more tests...

private static List<User> createUsers(boolean... banned) {
  List<User> users = new ArrayList<>();
  for (boolean isBanned : banned) {
    users.add(newUser()
        .setState(isBanned ? State.BANNED : State.NORMAL)
        .build());
  }
  return users;
}

private static Forum createForumAndRegisterUsers(List<User> users) {
  Forum forum = new Forum();
```

```
  for (User user : users) {
    try {
      forum.register(user);
    } catch(BannedUserException ignored) {}
  }
  return forum;
}

private static void validateForumAndUsers(Forum forum, List<User> users) {
  assertThat(forum.isReachable()).isTrue();
  for (User user : users) {
    assertThat(forum.hasRegisteredUser(user))
        .isEqualTo(user.getState() == State.BANNED);
  }
}
```

The problems in this code should be apparent based on the previous discussion of clarity. For one, although the test bodies are very concise, they are not complete: important details are hidden away in helper methods that the reader can't see without having to scroll to a completely different part of the file. Those helpers are also full of logic that makes them more difficult to verify at a glance (did you spot the bug?). The test becomes much clearer when it's rewritten to use DAMP, as shown in Example 12-20.

Example 12-20. Tests should be DAMP

```
@Test
public void shouldAllowMultipleUsers() {
  User user1 = newUser().setState(State.NORMAL).build();
  User user2 = newUser().setState(State.NORMAL).build();

  Forum forum = new Forum();
  forum.register(user1);
  forum.register(user2);

  assertThat(forum.hasRegisteredUser(user1)).isTrue();
  assertThat(forum.hasRegisteredUser(user2)).isTrue();
}

@Test
public void shouldNotRegisterBannedUsers() {
  User user = newUser().setState(State.BANNED).build();

  Forum forum = new Forum();
  try {
    forum.register(user);
  } catch(BannedUserException ignored) {}

  assertThat(forum.hasRegisteredUser(user)).isFalse();
}
```

These tests have more duplication, and the test bodies are a bit longer, but the extra verbosity is worth it. Each individual test is far more meaningful and can be understood entirely without leaving the test body. A reader of these tests can feel confident that the tests do what they claim to do and aren't hiding any bugs.

DAMP is not a replacement for DRY; it is complementary to it. Helper methods and test infrastructure can still help make tests clearer by making them more concise, factoring out repetitive steps whose details aren't relevant to the particular behavior being tested. The important point is that such refactoring should be done with an eye toward making tests more descriptive and meaningful, and not solely in the name of reducing repetition. The rest of this section will explore common patterns for sharing code across tests.

Shared Values

Many tests are structured by defining a set of shared values to be used by tests and then by defining the tests that cover various cases for how these values interact. Example 12-21 illustrates what such tests look like.

Example 12-21. Shared values with ambiguous names

```
private static final Account ACCOUNT_1 = Account.newBuilder()
    .setState(AccountState.OPEN).setBalance(50).build();

private static final Account ACCOUNT_2 = Account.newBuilder()
    .setState(AccountState.CLOSED).setBalance(0).build();

private static final Item ITEM = Item.newBuilder()
    .setName("Cheeseburger").setPrice(100).build();

// Hundreds of lines of other tests...

@Test
public void canBuyItem_returnsFalseForClosedAccounts() {
  assertThat(store.canBuyItem(ITEM, ACCOUNT_1)).isFalse();
}

@Test
public void canBuyItem_returnsFalseWhenBalanceInsufficient() {
  assertThat(store.canBuyItem(ITEM, ACCOUNT_2)).isFalse();
}
```

This strategy can make tests very concise, but it causes problems as the test suite grows. For one, it can be difficult to understand why a particular value was chosen for a test. In Example 12-21, the test names fortunately clarify which scenarios are being tested, but you still need to scroll up to the definitions to confirm that `ACCOUNT_1` and `ACCOUNT_2` are appropriate for those scenarios. More descriptive constant names (e.g.,

CLOSED_ACCOUNT and ACCOUNT_WITH_LOW_BALANCE) help a bit, but they still make it more difficult to see the exact details of the value being tested, and the ease of reusing these values can encourage engineers to do so even when the name doesn't exactly describe what the test needs.

Engineers are usually drawn to using shared constants because constructing individual values in each test can be verbose. A better way to accomplish this goal is to construct data using helper methods (see Example 12-22) that require the test author to specify only values they care about, and setting reasonable defaults[7] for all other values. This construction is trivial to do in languages that support named parameters, but languages without named parameters can use constructs such as the *Builder* pattern to emulate them (often with the assistance of tools such as AutoValue):

Example 12-22. Shared values using helper methods

```
# A helper method wraps a constructor by defining arbitrary defaults for
# each of its parameters.
def newContact(
    firstName="Grace", lastName="Hopper", phoneNumber="555-123-4567"):
  return Contact(firstName, lastName, phoneNumber)

# Tests call the helper, specifying values for only the parameters that they
# care about.
def test_fullNameShouldCombineFirstAndLastNames(self):
  def contact = newContact(firstName="Ada", lastName="Lovelace")
  self.assertEqual(contact.fullName(), "Ada Lovelace")

// Languages like Java that don't support named parameters can emulate them
// by returning a mutable "builder" object that represents the value under
// construction.
private static Contact.Builder newContact() {
  return Contact.newBuilder()
    .setFirstName("Grace")
    .setLastName("Hopper")
    .setPhoneNumber("555-123-4567");
}

// Tests then call methods on the builder to overwrite only the parameters
// that they care about, then call build() to get a real value out of the
// builder.
@Test
public void fullNameShouldCombineFirstAndLastNames() {
  Contact contact = newContact()
      .setFirstName("Ada")
```

7 In many cases, it can even be useful to slightly randomize the default values returned for fields that aren't explicitly set. This helps to ensure that two different instances won't accidentally compare as equal, and makes it more difficult for engineers to hardcode dependencies on the defaults.

```
      .setLastName("Lovelace")
      .build();
  assertThat(contact.getFullName()).isEqualTo("Ada Lovelace");
}
```

Using helper methods to construct these values allows each test to create the exact values it needs without having to worry about specifying irrelevant information or conflicting with other tests.

Shared Setup

A related way that tests shared code is via setup/initialization logic. Many test frameworks allow engineers to define methods to execute before each test in a suite is run. Used appropriately, these methods can make tests clearer and more concise by obviating the repetition of tedious and irrelevant initialization logic. Used inappropriately, these methods can harm a test's completeness by hiding important details in a separate initialization method.

The best use case for setup methods is to construct the object under tests and its collaborators. This is useful when the majority of tests don't care about the specific arguments used to construct those objects and can let them stay in their default states. The same idea also applies to stubbing return values for test doubles, which is a concept that we explore in more detail in Chapter 13.

One risk in using setup methods is that they can lead to unclear tests if those tests begin to depend on the particular values used in setup. For example, the test in Example 12-23 seems incomplete because a reader of the test needs to go hunting to discover where the string "Donald Knuth" came from.

Example 12-23. Dependencies on values in setup methods

```
private NameService nameService;
private UserStore userStore;

@Before
public void setUp() {
  nameService = new NameService();
  nameService.set("user1", "Donald Knuth");
  userStore = new UserStore(nameService);
}

// [... hundreds of lines of tests ...]

@Test
public void shouldReturnNameFromService() {
  UserDetails user = userStore.get("user1");
  assertThat(user.getName()).isEqualTo("Donald Knuth");
}
```

Tests like these that explicitly care about particular values should state those values directly, overriding the default defined in the setup method if need be. The resulting test contains slightly more repetition, as shown in Example 12-24, but the result is far more descriptive and meaningful.

Example 12-24. Overriding values in setup mMethods

```
private NameService nameService;
private UserStore userStore;

@Before
public void setUp() {
  nameService = new NameService();
  nameService.set("user1", "Donald Knuth");
  userStore = new UserStore(nameService);
}

@Test
public void shouldReturnNameFromService() {
  nameService.set("user1", "Margaret Hamilton");
  UserDetails user = userStore.get("user1");
  assertThat(user.getName()).isEqualTo("Margaret Hamilton");
}
```

Shared Helpers and Validation

The last common way that code is shared across tests is via "helper methods" called from the body of the test methods. We already discussed how helper methods can be a useful way for concisely constructing test values—this usage is warranted, but other types of helper methods can be dangerous.

One common type of helper is a method that performs a common set of assertions against a system under test. The extreme example is a `validate` method called at the end of every test method, which performs a set of fixed checks against the system under test. Such a validation strategy can be a bad habit to get into because tests using this approach are less behavior driven. With such tests, it is much more difficult to determine the intent of any particular test and to infer what exact case the author had in mind when writing it. When bugs are introduced, this strategy can also make them more difficult to localize because they will frequently cause a large number of tests to start failing.

More focused validation methods can still be useful, however. The best validation helper methods assert a *single conceptual fact* about their inputs, in contrast to general-purpose validation methods that cover a range of conditions. Such methods can be particularly helpful when the condition that they are validating is conceptually simple but requires looping or conditional logic to implement that would reduce clarity were it included in the body of a test method. For example, the helper method in Example 12-25 might be useful in a test covering several different cases around account access.

Example 12-25. A conceptually simple test

```
private void assertUserHasAccessToAccount(User user, Account account) {
  for (long userId : account.getUsersWithAccess()) {
    if (user.getId() == userId) {
      return;
    }
  }
  fail(user.getName() + " cannot access " + account.getName());
}
```

Defining Test Infrastructure

The techniques we've discussed so far cover sharing code across methods in a single test class or suite. Sometimes, it can also be valuable to share code across multiple test suites. We refer to this sort of code as *test infrastructure*. Though it is usually more valuable in integration or end-to-end tests, carefully designed test infrastructure can make unit tests much easier to write in some circumstances.

Custom test infrastructure must be approached more carefully than the code sharing that happens within a single test suite. In many ways, test infrastructure code is more similar to production code than it is to other test code given that it can have many callers that depend on it and can be difficult to change without introducing breakages. Most engineers aren't expected to make changes to the common test infrastructure while testing their own features. Test infrastructure needs to be treated as its own separate product, and accordingly, *test infrastructure must always have its own tests*.

Of course, most of the test infrastructure that most engineers use comes in the form of well-known third-party libraries like JUnit. A huge number of such libraries are available, and standardizing on them within an organization should happen as early and universally as possible. For example, Google many years ago mandated Mockito as the only mocking framework that should be used in new Java tests and banned new tests from using other mocking frameworks. This edict produced some grumbling at the time from people comfortable with other frameworks, but today, it's universally seen as a good move that made our tests easier to understand and work with.

Conclusion

Unit tests are one of the most powerful tools that we as software engineers have to make sure that our systems keep working over time in the face of unanticipated changes. But with great power comes great responsibility, and careless use of unit testing can result in a system that requires much more effort to maintain and takes much more effort to change without actually improving our confidence in said system.

Unit tests at Google are far from perfect, but we've found tests that follow the practices outlined in this chapter to be orders of magnitude more valuable than those that don't. We hope they'll help you to improve the quality of your own tests!

TL;DRs

- Strive for unchanging tests.
- Test via public APIs.
- Test state, not interactions.
- Make your tests complete and concise.
- Test behaviors, not methods.
- Structure tests to emphasize behaviors.
- Name tests after the behavior being tested.
- Don't put logic in tests.
- Write clear failure messages.
- Follow DAMP over DRY when sharing code for tests.

CHAPTER 13
Test Doubles

Written by Andrew Trenk and Dillon Bly
Edited by Tom Manshreck

Unit tests are a critical tool for keeping developers productive and reducing defects in code. Although they can be easy to write for simple code, writing them becomes difficult as code becomes more complex.

For example, imagine trying to write a test for a function that sends a request to an external server and then stores the response in a database. Writing a handful of tests might be doable with some effort. But if you need to write hundreds or thousands of tests like this, your test suite will likely take hours to run, and could become flaky due to issues like random network failures or tests overwriting one another's data.

Test doubles come in handy in such cases. A *test double* is an object or function that can stand in for a real implementation in a test, similar to how a stunt double can stand in for an actor in a movie. The use of test doubles is often referred to as *mocking*, but we avoid that term in this chapter because, as we'll see, that term is also used to refer to more specific aspects of test doubles.

Perhaps the most obvious type of test double is a simpler implementation of an object that behaves similarly to the real implementation, such as an in-memory database. Other types of test doubles can make it possible to validate specific details of your system, such as by making it easy to trigger a rare error condition, or ensuring a heavyweight function is called without actually executing the function's implementation.

The previous two chapters introduced the concept of *small tests* and discussed why they should comprise the majority of tests in a test suite. However, production code often doesn't fit within the constraints of small tests due to communication across multiple processes or machines. Test doubles can be much more lightweight than real

implementations, allowing you to write many small tests that execute quickly and are not flaky.

The Impact of Test Doubles on Software Development

The use of test doubles introduces a few complications to software development that require some trade-offs to be made. The concepts introduced here are discussed in more depth throughout this chapter:

Testability
: To use test doubles, a codebase needs to be designed to be *testable*—it should be possible for tests to swap out real implementations with test doubles. For example, code that calls a database needs to be flexible enough to be able to use a test double in place of a real database. If the codebase isn't designed with testing in mind and you later decide that tests are needed, it can require a major commitment to refactor the code to support the use of test doubles.

Applicability
: Although proper application of test doubles can provide a powerful boost to engineering velocity, their improper use can lead to tests that are brittle, complex, and less effective. These downsides are magnified when test doubles are used improperly across a large codebase, potentially resulting in major losses in productivity for engineers. In many cases, test doubles are not suitable and engineers should prefer to use real implementations instead.

Fidelity
: *Fidelity* refers to how closely the behavior of a test double resembles the behavior of the real implementation that it's replacing. If the behavior of a test double significantly differs from the real implementation, tests that use the test double likely wouldn't provide much value—for example, imagine trying to write a test with a test double for a database that ignores any data added to the database and always returns empty results. But perfect fidelity might not be feasible; test doubles often need to be vastly simpler than the real implementation in order to be suitable for use in tests. In many situations, it is appropriate to use a test double even without perfect fidelity. Unit tests that use test doubles often need to be supplemented by larger-scope tests that exercise the real implementation.

Test Doubles at Google

At Google, we've seen countless examples of the benefits to productivity and software quality that test doubles can bring to a codebase, as well as the negative impact they can cause when used improperly. The practices we follow at Google have evolved over time based on these experiences. Historically, we had few guidelines on how to

effectively use test doubles, but best practices evolved as we saw common patterns and antipatterns arise in many teams' codebases.

One lesson we learned the hard way is the danger of overusing mocking frameworks, which allow you to easily create test doubles (we will discuss mocking frameworks in more detail later in this chapter). When mocking frameworks first came into use at Google, they seemed like a hammer fit for every nail—they made it very easy to write highly focused tests against isolated pieces of code without having to worry about how to construct the dependencies of that code. It wasn't until several years and countless tests later that we began to realize the cost of such tests: though these tests were easy to write, we suffered greatly given that they required constant effort to maintain while rarely finding bugs. The pendulum at Google has now begun swinging in the other direction, with many engineers avoiding mocking frameworks in favor of writing more realistic tests.

Even though the practices discussed in this chapter are generally agreed upon at Google, the actual application of them varies widely from team to team. This variance stems from engineers having inconsistent knowledge of these practices, inertia in an existing codebase that doesn't conform to these practices, or teams doing what is easiest for the short term without thinking about the long-term implications.

Basic Concepts

Before we dive into how to effectively use test doubles, let's cover some of the basic concepts related to them. These build the foundation for best practices that we will discuss later in this chapter.

An Example Test Double

Imagine an ecommerce site that needs to process credit card payments. At its core, it might have something like the code shown in Example 13-1.

Example 13-1. A credit card service

```
class PaymentProcessor {
  private CreditCardService creditCardService;
  ...
  boolean makePayment(CreditCard creditCard, Money amount) {
    if (creditCard.isExpired()) { return false; }
    boolean success =
        creditCardService.chargeCreditCard(creditCard, amount);
    return success;
  }
}
```

It would be infeasible to use a real credit card service in a test (imagine all the transaction fees from running the test!), but a test double could be used in its place to *simulate* the behavior of the real system. The code in Example 13-2 shows an extremely simple test double.

Example 13-2. A trivial test double

```
class TestDoubleCreditCardService implements CreditCardService {
 @Override
 public boolean chargeCreditCard(CreditCard creditCard, Money amount) {
   return true;
 }
}
```

Although this test double doesn't look very useful, using it in a test still allows us to test some of the logic in the `makePayment()` method. For example, in Example 13-3, we can validate that the method behaves properly when the credit card is expired because the code path that the test exercises doesn't rely on the behavior of the credit card service.

Example 13-3. Using the test double

```
@Test public void cardIsExpired_returnFalse() {
  boolean success = paymentProcessor.makePayment(EXPIRED_CARD, AMOUNT);
  assertThat(success).isFalse();
}
```

The following sections in this chapter will discuss how to make use of test doubles in more complex situations than this one.

Seams

Code is said to be *testable* if it is written in a way that makes it possible to write unit tests for the code. A *seam* is a way to make code testable by allowing for the use of test doubles—it makes it possible to use different dependencies for the system under test rather than the dependencies used in a production environment.

Dependency injection is a common technique for introducing seams. In short, when a class utilizes dependency injection, any classes it needs to use (i.e., the class's *dependencies*) are passed to it rather than instantiated directly, making it possible for these dependencies to be substituted in tests.

Example 13-4 shows an example of dependency injection. Rather than the constructor creating an instance of `CreditCardService`, it accepts an instance as a parameter.

Example 13-4. Dependency injection

```
class PaymentProcessor {
  private CreditCardService creditCardService;

  PaymentProcessor(CreditCardService creditCardService) {
    this.creditCardService = creditCardService;
  }
  ...
}
```

The code that calls this constructor is responsible for creating an appropriate `CreditCardService` instance. Whereas the production code can pass in an implementation of `CreditCardService` that communicates with an external server, the test can pass in a test double, as demonstrated in Example 13-5.

Example 13-5. Passing in a test double

```
PaymentProcessor paymentProcessor =
    new PaymentProcessor(new TestDoubleCreditCardService());
```

To reduce boilerplate associated with manually specifying constructors, automated dependency injection frameworks can be used for constructing object graphs automatically. At Google, Guice and Dagger are automated dependency injection frameworks that are commonly used for Java code.

With dynamically typed languages such as Python or JavaScript, it is possible to dynamically replace individual functions or object methods. Dependency injection is less important in these languages because this capability makes it possible to use real implementations of dependencies in tests while only overriding functions or methods of the dependency that are unsuitable for tests.

Writing testable code requires an upfront investment. It is especially critical early in the lifetime of a codebase because the later testability is taken into account, the more difficult it is to apply to a codebase. Code written without testing in mind typically needs to be refactored or rewritten before you can add appropriate tests.

Mocking Frameworks

A *mocking framework* is a software library that makes it easier to create test doubles within tests; it allows you to replace an object with a *mock*, which is a test double whose behavior is specified inline in a test. The use of mocking frameworks reduces boilerplate because you don't need to define a new class each time you need a test double.

Example 13-6 demonstrates the use of Mockito, a mocking framework for Java. Mockito creates a test double for `CreditCardService` and instructs it to return a specific value.

Example 13-6. Mocking frameworks

```
class PaymentProcessorTest {
  ...
  PaymentProcessor paymentProcessor;

  // Create a test double of CreditCardService with just one line of code.
  @Mock CreditCardService mockCreditCardService;
  @Before public void setUp() {
    // Pass in the test double to the system under test.
    paymentProcessor = new PaymentProcessor(mockCreditCardService);
  }
  @Test public void chargeCreditCardFails_returnFalse() {
    // Give some behavior to the test double: it will return false
    // anytime the chargeCreditCard() method is called. The usage of
    // "any()" for the method's arguments tells the test double to
    // return false regardless of which arguments are passed.
    when(mockCreditCardService.chargeCreditCard(any(), any())
       .thenReturn(false);
    boolean success = paymentProcessor.makePayment(CREDIT_CARD, AMOUNT);
    assertThat(success).isFalse();
  }
}
```

Mocking frameworks exist for most major programming languages. At Google, we use Mockito for Java, the googlemock component of Googletest for C++, and unittest.mock for Python.

Although mocking frameworks facilitate easier usage of test doubles, they come with some significant caveats given that their overuse will often make a codebase more difficult to maintain. We cover some of these problems later in this chapter.

Techniques for Using Test Doubles

There are three primary techniques for using test doubles. This section presents a brief introduction to these techniques to give you a quick overview of what they are and how they differ. Later sections in this chapter go into more details on how to effectively apply them.

An engineer who is aware of the distinctions between these techniques is more likely to know the appropriate technique to use when faced with the need to use a test double.

Faking

A *fake* is a lightweight implementation of an API that behaves similar to the real implementation but isn't suitable for production; for example, an in-memory database. Example 13-7 presents an example of faking.

Example 13-7. A simple fake

```
// Creating the fake is fast and easy.
AuthorizationService fakeAuthorizationService =
    new FakeAuthorizationService();
AccessManager accessManager = new AccessManager(fakeAuthorizationService):

// Unknown user IDs shouldn't have access.
assertFalse(accessManager.userHasAccess(USER_ID));

// The user ID should have access after it is added to
// the authorization service.
fakeAuthorizationService.addAuthorizedUser(new User(USER_ID));
assertThat(accessManager.userHasAccess(USER_ID)).isTrue();
```

Using a fake is often the ideal technique when you need to use a test double, but a fake might not exist for an object you need to use in a test, and writing one can be challenging because you need to ensure that it has similar behavior to the real implementation, now and in the future.

Stubbing

Stubbing is the process of giving behavior to a function that otherwise has no behavior on its own—you specify to the function exactly what values to return (that is, you *stub* the return values).

Example 13-8 illustrates stubbing. The `when(...).thenReturn(...)` method calls from the Mockito mocking framework specify the behavior of the `lookupUser()` method.

Example 13-8. Stubbing

```
// Pass in a test double that was created by a mocking framework.
AccessManager accessManager = new AccessManager(mockAuthorizationService):

// The user ID shouldn't have access if null is returned.
when(mockAuthorizationService.lookupUser(USER_ID)).thenReturn(null);
assertThat(accessManager.userHasAccess(USER_ID)).isFalse();

// The user ID should have access if a non-null value is returned.
when(mockAuthorizationService.lookupUser(USER_ID)).thenReturn(USER);
assertThat(accessManager.userHasAccess(USER_ID)).isTrue();
```

Stubbing is typically done through mocking frameworks to reduce boilerplate that would otherwise be needed for manually creating new classes that hardcode return values.

Although stubbing can be a quick and simple technique to apply, it has limitations, which we'll discuss later in this chapter.

Interaction Testing

Interaction testing is a way to validate *how* a function is called without actually calling the implementation of the function. A test should fail if a function isn't called the correct way—for example, if the function isn't called at all, it's called too many times, or it's called with the wrong arguments.

Example 13-9 presents an instance of interaction testing. The `verify(...)` method from the Mockito mocking framework is used to validate that `lookupUser()` is called as expected.

Example 13-9. Interaction testing

```
// Pass in a test double that was created by a mocking framework.
AccessManager accessManager = new AccessManager(mockAuthorizationService);
accessManager.userHasAccess(USER_ID);

// The test will fail if accessManager.userHasAccess(USER_ID) didn't call
// mockAuthorizationService.lookupUser(USER_ID).
verify(mockAuthorizationService).lookupUser(USER_ID);
```

Similar to stubbing, interaction testing is typically done through mocking frameworks. This reduces boilerplate compared to manually creating new classes that contain code to keep track of how often a function is called and which arguments were passed in.

Interaction testing is sometimes called *mocking*. We avoid this terminology in this chapter because it can be confused with mocking frameworks, which can be used for stubbing as well as for interaction testing.

As discussed later in this chapter, interaction testing is useful in certain situations but should be avoided when possible because overuse can easily result in brittle tests.

Real Implementations

Although test doubles can be invaluable testing tools, our first choice for tests is to use the real implementations of the system under test's dependencies; that is, the same implementations that are used in production code. Tests have higher fidelity when

they execute code as it will be executed in production, and using real implementations helps accomplish this.

At Google, the preference for real implementations developed over time as we saw that overuse of mocking frameworks had a tendency to pollute tests with repetitive code that got out of sync with the real implementation and made refactoring difficult. We'll look at this topic in more detail later in this chapter.

Preferring real implementations in tests is known as *classical testing*. There is also a style of testing known as *mockist testing*, in which the preference is to use mocking frameworks instead of real implementations. Even though some people in the software industry practice mockist testing (including the creators of the first mocking frameworks), at Google, we have found that this style of testing is difficult to scale. It requires engineers to follow strict guidelines when designing the system under test, and the default behavior of most engineers at Google has been to write code in a way that is more suitable for the classical testing style.

Prefer Realism Over Isolation

Using real implementations for dependencies makes the system under test more realistic given that all code in these real implementations will be executed in the test. In contrast, a test that utilizes test doubles isolates the system under test from its dependencies so that the test does not execute code in the dependencies of the system under test.

We prefer realistic tests because they give more confidence that the system under test is working properly. If unit tests rely too much on test doubles, an engineer might need to run integration tests or manually verify that their feature is working as expected in order to gain this same level of confidence. Carrying out these extra tasks can slow down development and can even allow bugs to slip through if engineers skip these tasks entirely when they are too time consuming to carry out compared to running unit tests.

Replacing all dependencies of a class with test doubles arbitrarily isolates the system under test to the implementation that the author happens to put directly into the class and excludes implementation that happens to be in different classes. However, a good test should be independent of implementation—it should be written in terms of the API being tested rather than in terms of how the implementation is structured.

Using real implementations can cause your test to fail if there is a bug in the real implementation. This is good! You *want* your tests to fail in such cases because it indicates that your code won't work properly in production. Sometimes, a bug in a real implementation can cause a cascade of test failures because other tests that use the real implementation might fail, too. But with good developer tools, such as a

Continuous Integration (CI) system, it is usually easy to track down the change that caused the failure.

Case Study: @DoNotMock

At Google, we've seen enough tests that over-rely on mocking frameworks to motivate the creation of the `@DoNotMock` annotation in Java, which is available as part of the ErrorProne static analysis tool. This annotation is a way for API owners to declare, "this type should not be mocked because better alternatives exist."

If an engineer attempts to use a mocking framework to create an instance of a class or interface that has been annotated as `@DoNotMock`, as demonstrated in Example 13-10, they will see an error directing them to use a more suitable test strategy, such as a real implementation or a fake. This annotation is most commonly used for value objects that are simple enough to use as-is, as well as for APIs that have well-engineered fakes available.

Example 13-10. The @DoNotMock annotation

```
@DoNotMock("Use SimpleQuery.create() instead of mocking.")
public abstract class Query {
  public abstract String getQueryValue();
}
```

Why would an API owner care? In short, it severely constrains the API owner's ability to make changes to their implementation over time. As we'll explore later in the chapter, every time a mocking framework is used for stubbing or interaction testing, it duplicates behavior provided by the API.

When the API owner wants to change their API, they might find that it has been mocked thousands or even tens of thousands of times throughout Google's codebase! These test doubles are very likely to exhibit behavior that violates the API contract of the type being mocked—for instance, returning null for a method that can never return null. Had the tests used the real implementation or a fake, the API owner could make changes to their implementation without first fixing thousands of flawed tests.

How to Decide When to Use a Real Implementation

A real implementation is preferred if it is fast, deterministic, and has simple dependencies. For example, a real implementation should be used for a *value object*. Examples include an amount of money, a date, a geographical address, or a collection class such as a list or a map.

However, for more complex code, using a real implementation often isn't feasible. There might not be an exact answer on when to use a real implementation or a test double given that there are trade-offs to be made, so you need to take the following considerations into account.

Execution time

One of the most important qualities of unit tests is that they should be fast—you want to be able to continually run them during development so that you can get quick feedback on whether your code is working (and you also want them to finish quickly when run in a CI system). As a result, a test double can be very useful when the real implementation is slow.

How slow is too slow for a unit test? If a real implementation added one millisecond to the running time of each individual test case, few people would classify it as slow. But what if it added 10 milliseconds, 100 milliseconds, 1 second, and so on?

There is no exact answer here—it can depend on whether engineers feel a loss in productivity, and how many tests are using the real implementation (one second extra per test case may be reasonable if there are five test cases, but not if there are 500). For borderline situations, it is often simpler to use a real implementation until it becomes too slow to use, at which point the tests can be updated to use a test double instead.

Parellelization of tests can also help reduce execution time. At Google, our test infrastructure makes it trivial to split up tests in a test suite to be executed across multiple servers. This increases the cost of CPU time, but it can provide a large savings in developer time. We discuss this more in Chapter 18.

Another trade-off to be aware of: using a real implementation can result in increased build times given that the tests need to build the real implementation as well as all of its dependencies. Using a highly scalable build system like Bazel can help because it caches unchanged build artifacts.

Determinism

A test is *deterministic* if, for a given version of the system under test, running the test always results in the same outcome; that is, the test either always passes or always fails. In contrast, a test is *nondeterministic* if its outcome can change, even if the system under test remains unchanged.

Nondeterminism in tests can lead to flakiness—tests can occasionally fail even when there are no changes to the system under test. As discussed in Chapter 11, flakiness harms the health of a test suite if developers start to distrust the results of the test and ignore failures. If use of a real implementation rarely causes flakiness, it might not warrant a response, because there is little disruption to engineers. But if flakiness hap-

pens often, it might be time to replace a real implementation with a test double because doing so will improve the fidelity of the test.

A real implementation can be much more complex compared to a test double, which increases the likelihood that it will be nondeterministic. For example, a real implementation that utilizes multithreading might occasionally cause a test to fail if the output of the system under test differs depending on the order in which the threads are executed.

A common cause of nondeterminism is code that is not hermetic; that is, it has dependencies on external services that are outside the control of a test. For example, a test that tries to read the contents of a web page from an HTTP server might fail if the server is overloaded or if the web page contents change. Instead, a test double should be used to prevent the test from depending on an external server. If using a test double is not feasible, another option is to use a hermetic instance of a server, which has its life cycle controlled by the test. Hermetic instances are discussed in more detail in the next chapter.

Another example of nondeterminism is code that relies on the system clock given that the output of the system under test can differ depending on the current time. Instead of relying on the system clock, a test can use a test double that hardcodes a specific time.

Dependency construction

When using a real implementation, you need to construct all of its dependencies. For example, an object needs its entire dependency tree to be constructed: all objects that it depends on, all objects that these dependent objects depend on, and so on. A test double often has no dependencies, so constructing a test double can be much simpler compared to constructing a real implementation.

As an extreme example, imagine trying to create the object in the code snippet that follows in a test. It would be time consuming to determine how to construct each individual object. Tests will also require constant maintenance because they need to be updated when the signature of these objects' constructors is modified:

```
Foo foo = new Foo(new A(new B(new C()), new D()), new E(), ..., new Z());
```

It can be tempting to instead use a test double because constructing one can be trivial. For example, this is all it takes to construct a test double when using the Mockito mocking framework:

```
@Mock Foo mockFoo;
```

Although creating this test double is much simpler, there are significant benefits to using the real implementation, as discussed earlier in this section. There are also often significant downsides to overusing test doubles in this way, which we look at later in this chapter. So, a trade-off needs to be made when considering whether to use a real implementation or a test double.

Rather than manually constructing the object in tests, the ideal solution is to use the same object construction code that is used in the production code, such as a factory method or automated dependency injection. To support the use case for tests, the object construction code needs to be flexible enough to be able to use test doubles rather than hardcoding the implementations that will be used for production.

Faking

If using a real implementation is not feasible within a test, the best option is often to use a fake in its place. A fake is preferred over other test double techniques because it behaves similarly to the real implementation: the system under test shouldn't even be able to tell whether it is interacting with a real implementation or a fake. Example 13-11 illustrates a fake file system.

Example 13-11. A fake file system

```
// This fake implements the FileSystem interface. This interface is also
// used by the real implementation.
public class FakeFileSystem implements FileSystem {
  // Stores a map of file name to file contents. The files are stored in
  // memory instead of on disk since tests shouldn't need to do disk I/O.
  private Map<String, String> files = new HashMap<>();
  @Override
  public void writeFile(String fileName, String contents) {
    // Add the file name and contents to the map.
    files.add(fileName, contents);
  }
  @Override
  public String readFile(String fileName) {
    String contents = files.get(fileName);
    // The real implementation will throw this exception if the
    // file isn't found, so the fake must throw it too.
    if (contents == null) { throw new FileNotFoundException(fileName); }
    return contents;
  }
}
```

Why Are Fakes Important?

Fakes can be a powerful tool for testing: they execute quickly and allow you to effectively test your code without the drawbacks of using real implementations.

A single fake has the power to radically improve the testing experience of an API. If you scale that to a large number of fakes for all sorts of APIs, fakes can provide an enormous boost to engineering velocity across a software organization.

At the other end of the spectrum, in a software organization where fakes are rare, velocity will be slower because engineers can end up struggling with using real implementations that lead to slow and flaky tests. Or engineers might resort to other test double techniques such as stubbing or interaction testing, which, as we'll examine later in this chapter, can result in tests that are unclear, brittle, and less effective.

When Should Fakes Be Written?

A fake requires more effort and more domain experience to create because it needs to behave similarly to the real implementation. A fake also requires maintenance: whenever the behavior of the real implementation changes, the fake must also be updated to match this behavior. Because of this, the team that owns the real implementation should write and maintain a fake.

If a team is considering writing a fake, a trade-off needs to be made on whether the productivity improvements that will result from the use of the fake outweigh the costs of writing and maintaining it. If there are only a handful of users, it might not be worth their time, whereas if there are hundreds of users, it can result in an obvious productivity improvement.

To reduce the number of fakes that need to be maintained, a fake should typically be created only at the root of the code that isn't feasible for use in tests. For example, if a database can't be used in tests, a fake should exist for the database API itself rather than for each class that calls the database API.

Maintaining a fake can be burdensome if its implementation needs to be duplicated across programming languages, such as for a service that has client libraries that allow the service to be invoked from different languages. One solution for this case is to create a single fake service implementation and have tests configure the client libraries to send requests to this fake service. This approach is more heavyweight compared to having the fake written entirely in memory because it requires the test to communicate across processes. However, it can be a reasonable trade-off to make, as long as the tests can still execute quickly.

The Fidelity of Fakes

Perhaps the most important concept surrounding the creation of fakes is *fidelity*; in other words, how closely the behavior of a fake matches the behavior of the real implementation. If the behavior of a fake doesn't match the behavior of the real implementation, a test using that fake is not useful—a test might pass when the fake is used, but this same code path might not work properly in the real implementation.

Perfect fidelity is not always feasible. After all, the fake was necessary because the real implementation wasn't suitable in one way or another. For example, a fake database would usually not have fidelity to a real database in terms of hard drive storage because the fake would store everything in memory.

Primarily, however, a fake should maintain fidelity to the API contracts of the real implementation. For any given input to an API, a fake should return the same output and perform the same state changes of its corresponding real implementation. For example, for a real implementation of `database.save(itemId)`, if an item is successfully saved when its ID does not yet exist but an error is produced when the ID already exists, the fake must conform to this same behavior.

One way to think about this is that the fake must have perfect fidelity to the real implementation, but *only from the perspective of the test*. For example, a fake for a hashing API doesn't need to guarantee that the hash value for a given input is exactly the same as the hash value that is generated by the real implementation—tests likely don't care about the specific hash value, only that the hash value is unique for a given input. If the contract of the hashing API doesn't make guarantees of what specific hash values will be returned, the fake is still conforming to the contract even if it doesn't have perfect fidelity to the real implementation.

Other examples where perfect fidelity typically might not be useful for fakes include latency and resource consumption. However, a fake cannot be used if you need to explicitly test for these constraints (e.g., a performance test that verifies the latency of a function call), so you would need to resort to other mechanisms, such as by using a real implementation instead of a fake.

A fake might not need to have 100% of the functionality of its corresponding real implementation, especially if such behavior is not needed by most tests (e.g., error handling code for rare edge cases). It is best to have the fake fail fast in this case; for example, raise an error if an unsupported code path is executed. This failure communicates to the engineer that the fake is not appropriate in this situation.

Fakes Should Be Tested

A fake must have its *own* tests to ensure that it conforms to the API of its corresponding real implementation. A fake without tests might initially provide realistic behavior, but without tests, this behavior can diverge over time as the real implementation evolves.

One approach to writing tests for fakes involves writing tests against the API's public interface and running those tests against both the real implementation and the fake (these are known as *contract tests*). The tests that run against the real implementation will likely be slower, but their downside is minimized because they need to be run only by the owners of the fake.

What to Do If a Fake Is Not Available

If a fake is not available, first ask the owners of the API to create one. The owners might not be familiar with the concept of fakes, or they might not realize the benefit they provide to users of an API.

If the owners of an API are unwilling or unable to create a fake, you might be able to write your own. One way to do this is to wrap all calls to the API in a single class and then create a fake version of the class that doesn't talk to the API. Doing this can also be much simpler than creating a fake for the entire API because often you'll need to use only a subset of the API's behavior anyway. At Google, some teams have even contributed their fake to the owners of the API, which has allowed other teams to benefit from the fake.

Finally, you could decide to settle on using a real implementation (and deal with the trade-offs of real implementations that are mentioned earlier in this chapter), or resort to other test double techniques (and deal with the trade-offs that we will mention later in this chapter).

In some cases, you can think of a fake as an optimization: if tests are too slow using a real implementation, you can create a fake to make them run faster. But if the speedup from a fake doesn't outweigh the work it would take to create and maintain the fake, it would be better to stick with using the real implementation.

Stubbing

As discussed earlier in this chapter, stubbing is a way for a test to hardcode behavior for a function that otherwise has no behavior on its own. It is often a quick and easy way to replace a real implementation in a test. For example, the code in Example 13-12 uses stubbing to simulate the response from a credit card server.

Example 13-12. Using stubbing to simulate responses

```
@Test public void getTransactionCount() {
  transactionCounter = new TransactionCounter(mockCreditCardServer);
  // Use stubbing to return three transactions.
  when(mockCreditCardServer.getTransactions()).thenReturn(
      newList(TRANSACTION_1, TRANSACTION_2, TRANSACTION_3));
  assertThat(transactionCounter.getTransactionCount()).isEqualTo(3);
}
```

The Dangers of Overusing Stubbing

Because stubbing is so easy to apply in tests, it can be tempting to use this technique anytime it's not trivial to use a real implementation. However, overuse of stubbing can result in major losses in productivity for engineers who need to maintain these tests.

Tests become unclear

Stubbing involves writing extra code to define the behavior of the functions being stubbed. Having this extra code detracts from the intent of the test, and this code can be difficult to understand if you're not familiar with the implementation of the system under test.

A key sign that stubbing isn't appropriate for a test is if you find yourself mentally stepping through the system under test in order to understand why certain functions in the test are stubbed.

Tests become brittle

Stubbing leaks implementation details of your code into your test. When implementation details in your production code change, you'll need to update your tests to reflect these changes. Ideally, a good test should need to change only if user-facing behavior of an API changes; it should remain unaffected by changes to the API's implementation.

Tests become less effective

With stubbing, there is no way to ensure the function being stubbed behaves like the real implementation, such as in a statement like that shown in the following snippet that hardcodes part of the contract of the `add()` method (*"If 1 and 2 are passed in, 3 will be returned"*):

```
when(stubCalculator.add(1, 2)).thenReturn(3);
```

Stubbing is a poor choice if the system under test depends on the real implementation's contract because you will be forced to duplicate the details of the contract, and there is no way to guarantee that the contract is correct (i.e., that the stubbed function has fidelity to the real implementation).

Additionally, with stubbing there is no way to store state, which can make it difficult to test certain aspects of your code. For example, if you call `database.save(item)` on either a real implementation or a fake, you might be able to retrieve the item by calling `database.get(item.id())` given that both of these calls are accessing internal state, but with stubbing, there is no way to do this.

An example of overusing stubbing

Example 13-13 illustrates a test that overuses stubbing.

Example 13-13. Overuse of stubbing

```
@Test public void creditCardIsCharged() {
  // Pass in test doubles that were created by a mocking framework.
  paymentProcessor =
      new PaymentProcessor(mockCreditCardServer, mockTransactionProcessor);
  // Set up stubbing for these test doubles.
  when(mockCreditCardServer.isServerAvailable()).thenReturn(true);
  when(mockTransactionProcessor.beginTransaction()).thenReturn(transaction);
  when(mockCreditCardServer.initTransaction(transaction)).thenReturn(true);
  when(mockCreditCardServer.pay(transaction, creditCard, 500))
      .thenReturn(false);
  when(mockTransactionProcessor.endTransaction()).thenReturn(true);
  // Call the system under test.
  paymentProcessor.processPayment(creditCard, Money.dollars(500));
  // There is no way to tell if the pay() method actually carried out the
  // transaction, so the only thing the test can do is verify that the
  // pay() method was called.
  verify(mockCreditCardServer).pay(transaction, creditCard, 500);
}
```

Example 13-14 rewrites the same test but avoids using stubbing. Notice how the test is shorter and that implementation details (such as how the transaction processor is used) are not exposed in the test. No special setup is needed because the credit card server knows how to behave.

Example 13-14. Refactoring a test to avoid stubbing

```
@Test public void creditCardIsCharged() {
  paymentProcessor =
      new PaymentProcessor(creditCardServer, transactionProcessor);
  // Call the system under test.
  paymentProcessor.processPayment(creditCard, Money.dollars(500));
  // Query the credit card server state to see if the payment went through.
  assertThat(creditCardServer.getMostRecentCharge(creditCard))
      .isEqualTo(500);
}
```

We obviously don't want such a test to talk to an external credit card server, so a fake credit card server would be more suitable. If a fake isn't available, another option is to use a real implementation that talks to a hermetic credit card server, although this will increase the execution time of the tests. (We explore hermetic servers in the next chapter.)

When Is Stubbing Appropriate?

Rather than a catch-all replacement for a real implementation, stubbing is appropriate when you need a function to return a specific value to get the system under test into a certain state, such as Example 13-12 that requires the system under test to return a non-empty list of transactions. Because a function's behavior is defined inline in the test, stubbing can simulate a wide variety of return values or errors that might not be possible to trigger from a real implementation or a fake.

To ensure its purpose is clear, each stubbed function should have a direct relationship with the test's assertions. As a result, a test typically should stub out a small number of functions because stubbing out many functions can lead to tests that are less clear. A test that requires many functions to be stubbed can be a sign that stubbing is being overused, or that the system under test is too complex and should be refactored.

Note that even when stubbing is appropriate, real implementations or fakes are still preferred because they don't expose implementation details and they give you more guarantees about the correctness of the code compared to stubbing. But stubbing can be a reasonable technique to use, as long as its usage is constrained so that tests don't become overly complex.

Interaction Testing

As discussed earlier in this chapter, interaction testing is a way to validate how a function is called without actually calling the implementation of the function.

Mocking frameworks make it easy to perform interaction testing. However, to keep tests useful, readable, and resilient to change, it's important to perform interaction testing only when necessary.

Prefer State Testing Over Interaction Testing

In contrast to interaction testing, it is preferred to test code through *state testing*.

With state testing, you call the system under test and validate that either the correct value was returned or that some other state in the system under test was properly changed. Example 13-15 presents an example of state testing.

Example 13-15. State testing

```
@Test public void sortNumbers() {
  NumberSorter numberSorter = new NumberSorter(quicksort, bubbleSort);
  // Call the system under test.
  List sortedList = numberSorter.sortNumbers(newList(3, 1, 2));
  // Validate that the returned list is sorted. It doesn't matter which
  // sorting algorithm is used, as long as the right result was returned.
  assertThat(sortedList).isEqualTo(newList(1, 2, 3));
}
```

Example 13-16 illustrates a similar test scenario but instead uses interaction testing. Note how it's impossible for this test to determine that the numbers are actually sorted, because the test doubles don't know how to sort the numbers—all it can tell you is that the system under test tried to sort the numbers.

Example 13-16. Interaction testing

```
@Test public void sortNumbers_quicksortIsUsed() {
  // Pass in test doubles that were created by a mocking framework.
  NumberSorter numberSorter =
      new NumberSorter(mockQuicksort, mockBubbleSort);

  // Call the system under test.
  numberSorter.sortNumbers(newList(3, 1, 2));

  // Validate that numberSorter.sortNumbers() used quicksort. The test
  // will fail if mockQuicksort.sort() is never called (e.g., if
  // mockBubbleSort is used) or if it's called with the wrong arguments.
  verify(mockQuicksort).sort(newList(3, 1, 2));
}
```

At Google, we've found that emphasizing state testing is more scalable; it reduces test brittleness, making it easier to change and maintain code over time.

The primary issue with interaction testing is that it can't tell you that the system under test is working properly; it can only validate that certain functions are called as expected. It requires you to make an assumption about the behavior of the code; for example, "*If `database.save(item)` is called, we assume the item will be saved to the database.*" State testing is preferred because it actually validates this assumption (such as by saving an item to a database and then querying the database to validate that the item exists).

Another downside of interaction testing is that it utilizes implementation details of the system under test—to validate that a function was called, you are exposing to the test that the system under test calls this function. Similar to stubbing, this extra code makes tests brittle because it leaks implementation details of your production code into tests. Some people at Google jokingly refer to tests that overuse interaction

testing as *change-detector tests* because they fail in response to any change to the production code, even if the behavior of the system under test remains unchanged.

When Is Interaction Testing Appropriate?

There are some cases for which interaction testing is warranted:

- You cannot perform state testing because you are unable to use a real implementation or a fake (e.g., if the real implementation is too slow and no fake exists). As a fallback, you can perform interaction testing to validate that certain functions are called. Although not ideal, this does provide some basic level of confidence that the system under test is working as expected.
- Differences in the number or order of calls to a function would cause undesired behavior. Interaction testing is useful because it could be difficult to validate this behavior with state testing. For example, if you expect a caching feature to reduce the number of calls to a database, you can verify that the database object is not accessed more times than expected. Using Mockito, the code might look similar to this:

  ```
  verify(databaseReader, atMostOnce()).selectRecords();
  ```

Interaction testing is not a complete replacement for state testing. If you are not able to perform state testing in a unit test, strongly consider supplementing your test suite with larger-scoped tests that do perform state testing. For instance, if you have a unit test that validates usage of a database through interaction testing, consider adding an integration test that can perform state testing against a real database. Larger-scope testing is an important strategy for risk mitigation, and we discuss it in the next chapter.

Best Practices for Interaction Testing

When performing interaction testing, following these practices can reduce some of the impact of the aforementioned downsides.

Prefer to perform interaction testing only for state-changing functions

When a system under test calls a function on a dependency, that call falls into one of two categories:

State-changing
: Functions that have side effects on the world outside the system under test. Examples: `sendEmail()`, `saveRecord()`, `logAccess()`.

Non-state-changing
: Functions that don't have side effects; they return information about the world outside the system under test and don't modify anything. Examples: `getUser()`, `findResults()`, `readFile()`.

In general, you should perform interaction testing only for functions that are state-changing. Performing interaction testing for non-state-changing functions is usually redundant given that the system under test will use the return value of the function to do other work that you can assert. The interaction itself is not an important detail for correctness, because it has no side effects.

Performing interaction testing for non-state-changing functions makes your test brittle because you'll need to update the test anytime the pattern of interactions changes. It also makes the test less readable given that the additional assertions make it more difficult to determine which assertions are important for ensuring correctness of the code. By contrast, state-changing interactions represent something useful that your code is doing to change state somewhere else.

Example 13-17 demonstrates interaction testing on both state-changing and non-state-changing functions.

Example 13-17. State-changing and non-state-changing interactions

```
@Test public void grantUserPermission() {
  UserAuthorizer userAuthorizer =
      new UserAuthorizer(mockUserService, mockPermissionDatabase);
  when(mockPermissionService.getPermission(FAKE_USER)).thenReturn(EMPTY);

  // Call the system under test.
  userAuthorizer.grantPermission(USER_ACCESS);

  // addPermission() is state-changing, so it is reasonable to perform
  // interaction testing to validate that it was called.
  verify(mockPermissionDatabase).addPermission(FAKE_USER, USER_ACCESS);

  // getPermission() is non-state-changing, so this line of code isn't
  // needed. One clue that interaction testing may not be needed:
  // getPermission() was already stubbed earlier in this test.
  verify(mockPermissionDatabase).getPermission(FAKE_USER);
}
```

Avoid overspecification

In Chapter 12, we discuss why it is useful to test behaviors rather than methods. This means that a test method should focus on verifying one behavior of a method or class rather than trying to verify multiple behaviors in a single test.

When performing interaction testing, we should aim to apply the same principle by avoiding overspecifying which functions and arguments are validated. This leads to tests that are clearer and more concise. It also leads to tests that are resilient to changes made to behaviors that are outside the scope of each test, so fewer tests will fail if a change is made to a way a function is called.

Example 13-18 illustrates interaction testing with overspecification. The intention of the test is to validate that the user's name is included in the greeting prompt, but the test will fail if unrelated behavior is changed.

Example 13-18. Overspecified interaction tests

```
@Test public void displayGreeting_renderUserName() {
  when(mockUserService.getUserName()).thenReturn("Fake User");
  userGreeter.displayGreeting(); // Call the system under test.

  // The test will fail if any of the arguments to setText() are changed.
  verify(userPrompt).setText("Fake User", "Good morning!", "Version 2.1");

  // The test will fail if setIcon() is not called, even though this
  // behavior is incidental to the test since it is not related to
  // validating the user name.
  verify(userPrompt).setIcon(IMAGE_SUNSHINE);
}
```

Example 13-19 illustrates interaction testing with more care in specifying relevant arguments and functions. The behaviors being tested are split into separate tests, and each test validates the minimum amount necessary for ensuring the behavior it is testing is correct.

Example 13-19. Well-specified interaction tests

```
@Test public void displayGreeting_renderUserName() {
  when(mockUserService.getUserName()).thenReturn("Fake User");
  userGreeter.displayGreeting(); // Call the system under test.
  verify(userPrompter).setText(eq("Fake User"), any(), any());
}
@Test public void displayGreeting_timeIsMorning_useMorningSettings() {
  setTimeOfDay(TIME_MORNING);
  userGreeter.displayGreeting(); // Call the system under test.
  verify(userPrompt).setText(any(), eq("Good morning!"), any());
  verify(userPrompt).setIcon(IMAGE_SUNSHINE);
}
```

Conclusion

We've learned that test doubles are crucial to engineering velocity because they can help comprehensively test your code and ensure that your tests run fast. On the other hand, misusing them can be a major drain on productivity because they can lead to tests that are unclear, brittle, and less effective. This is why it's important for engineers to understand the best practices for how to effectively apply test doubles.

There is often no exact answer regarding whether to use a real implementation or a test double, or which test double technique to use. An engineer might need to make some trade-offs when deciding the proper approach for their use case.

Although test doubles are great for working around dependencies that are difficult to use in tests, if you want to maximize confidence in your code, at some point you still want to exercise these dependencies in tests. The next chapter will cover larger-scope testing, for which these dependencies are used regardless of their suitability for unit tests; for example, even if they are slow or nondeterministic.

TL;DRs

- A real implementation should be preferred over a test double.
- A fake is often the ideal solution if a real implementation can't be used in a test.
- Overuse of stubbing leads to tests that are unclear and brittle.
- Interaction testing should be avoided when possible: it leads to tests that are brittle because it exposes implementation details of the system under test.

CHAPTER 14
Larger Testing

Written by Joseph Graves
Edited by Tom Manshreck

In previous chapters, we have recounted how a testing culture was established at Google and how small unit tests became a fundamental part of the developer workflow. But what about other kinds of tests? It turns out that Google does indeed use many larger tests, and these comprise a significant part of the risk mitigation strategy necessary for healthy software engineering. But these tests present additional challenges to ensure that they are valuable assets and not resource sinks. In this chapter, we'll discuss what we mean by "larger tests," when we execute them, and best practices for keeping them effective.

What Are Larger Tests?

As mentioned previously, Google has specific notions of test size. Small tests are restricted to one thread, one process, one machine. Larger tests do not have the same restrictions. But Google also has notions of test *scope*. A unit test necessarily is of smaller scope than an integration test. And the largest-scoped tests (sometimes called end-to-end or system tests) typically involve several real dependencies and fewer test doubles.

Larger tests are many things that small tests are not. They are not bound by the same constraints; thus, they can exhibit the following characteristics:

- They may be slow. Our large tests have a default timeout of 15 minutes or 1 hour, but we also have tests that run for multiple hours or even days.
- They may be nonhermetic. Large tests may share resources with other tests and traffic.

- They may be nondeterministic. If a large test is nonhermetic, it is almost impossible to guarantee determinism: other tests or user state may interfere with it.

So why have larger tests? Reflect back on your coding process. How do you confirm that the programs you write actually work? You might be writing and running unit tests as you go, but do you find yourself running the actual binary and trying it out yourself? And when you share this code with others, how do they test it? By running your unit tests, or by trying it out themselves?

Also, how do you know that your code continues to work during upgrades? Suppose that you have a site that uses the Google Maps API and there's a new API version. Your unit tests likely won't help you to know whether there are any compatibility issues. You'd probably run it and try it out to see whether anything broke.

Unit tests can give you confidence about individual functions, objects, and modules, but large tests provide more confidence that the overall system works as intended. And having actual automated tests scales in ways that manual testing does not.

Fidelity

The primary reason larger tests exist is to address *fidelity*. Fidelity is the property by which a test is reflective of the real behavior of the system under test (SUT).

One way of envisioning fidelity is in terms of the environment. As Figure 14-1 illustrates, unit tests bundle a test and a small portion of code together as a runnable unit, which ensures the code is tested but is very different from how production code runs. Production itself is, naturally, the environment of highest fidelity in testing. There is also a spectrum of interim options. A key for larger tests is to find the proper fit, because increasing fidelity also comes with increasing costs and (in the case of production) increasing risk of failure.

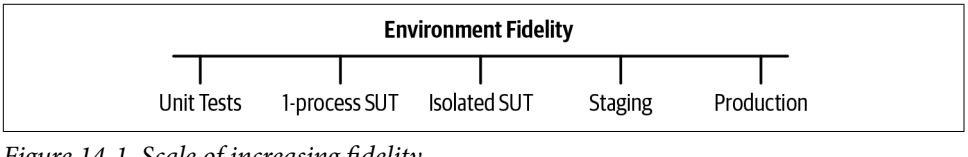

Figure 14-1. Scale of increasing fidelity

Tests can also be measured in terms of how faithful the test content is to reality. Many handcrafted, large tests are dismissed by engineers if the test data itself looks unrealistic. Test data copied from production is much more faithful to reality (having been captured that way), but a big challenge is how to create realistic test traffic *before* launching the new code. This is particularly a problem in artificial intelligence (AI), for which the "seed" data often suffers from intrinsic bias. And, because most data for unit tests is handcrafted, it covers a narrow range of cases and tends to conform to

the biases of the author. The uncovered scenarios missed by the data represent a fidelity gap in the tests.

Common Gaps in Unit Tests

Larger tests might also be necessary where smaller tests fail. The subsections that follow present some particular areas where unit tests do not provide good risk mitigation coverage.

Unfaithful doubles

A single unit test typically covers one class or module. Test doubles (as discussed in Chapter 13) are frequently used to eliminate heavyweight or hard-to-test dependencies. But when those dependencies are replaced, it becomes possible that the replacement and the doubled thing do not agree.

Almost all unit tests at Google are written by the same engineer who is writing the unit under test. When those unit tests need doubles and when the doubles used are mocks, it is the engineer writing the unit test defining the mock and its intended behavior. But that engineer usually did *not* write the thing being mocked and can be misinformed about its actual behavior. The relationship between the unit under test and a given peer is a behavioral contract, and if the engineer is mistaken about the actual behavior, the understanding of the contract is invalid.

Moreover, mocks become stale. If this mock-based unit test is not visible to the author of the real implementation and the real implementation changes, there is no signal that the test (and the code being tested) should be updated to keep up with the changes.

Note that, as mentioned in Chapter 13, if teams provide fakes for their own services, this concern is mostly alleviated.

Configuration issues

Unit tests cover code within a given binary. But that binary is typically not completely self-sufficient in terms of how it is executed. Usually a binary has some kind of deployment configuration or starter script. Additionally, real end-user-serving production instances have their own configuration files or configuration databases.

If there are issues with these files or the compatibility between the state defined by these stores and the binary in question, these can lead to major user issues. Unit tests alone cannot verify this compatibility.[1] Incidentally, this is a good reason to ensure that your configuration is in version control as well as your code, because then,

1 See "Continuous Delivery" on page 483 and Chapter 25 for more information.

changes to configuration can be identified as the source of bugs as opposed to introducing random external flakiness and can be built in to large tests.

At Google, configuration changes are the number one reason for our major outages. This is an area in which we have underperformed and has led to some of our most embarrassing bugs. For example, there was a global Google outage back in 2013 due to a bad network configuration push that was never tested. Configurations tend to be written in configuration languages, not production code languages. They also often have faster production rollout cycles than binaries, and they can be more difficult to test. All of these lead to a higher likelihood of failure. But at least in this case (and others), configuration was version controlled, and we could quickly identify the culprit and mitigate the issue.

Issues that arise under load

At Google, unit tests are intended to be small and fast because they need to fit into our standard test execution infrastructure and also be run many times as part of a frictionless developer workflow. But performance, load, and stress testing often require sending large volumes of traffic to a given binary. These volumes become difficult to test in the model of a typical unit test. And our large volumes are big, often thousands or millions of queries per second (in the case of ads, real-time bidding)!

Unanticipated behaviors, inputs, and side effects

Unit tests are limited by the imagination of the engineer writing them. That is, they can only test for anticipated behaviors and inputs. However, issues that users find with a product are mostly unanticipated (otherwise it would be unlikely that they would make it to end users as issues). This fact suggests that different test techniques are needed to test for unanticipated behaviors.

Hyrum's Law is an important consideration here: even if we could test 100% for conformance to a strict, specified contract, the effective user contract applies to all visible behaviors, not just a stated contract. It is unlikely that unit tests alone test for all visible behaviors that are not specified in the public API.

Emergent behaviors and the "vacuum effect"

Unit tests are limited to the scope that they cover (especially with the widespread use of test doubles), so if behavior changes in areas outside of this scope, it cannot be detected. And because unit tests are designed to be fast and reliable, they deliberately eliminate the chaos of real dependencies, network, and data. A unit test is like a problem in theoretical physics: ensconced in a vacuum, neatly hidden from the mess of the real world, which is great for speed and reliability but misses certain defect categories.

Why Not Have Larger Tests?

In earlier chapters, we discussed many of the properties of a developer-friendly test. In particular, it needs to be as follows:

Reliable
: It must not be flaky and it must provide a useful pass/fail signal.

Fast
: It needs to be fast enough to not interrupt the developer workflow.

Scalable
: Google needs to be able to run all such useful affected tests efficiently for presubmits and for post-submits.

Good unit tests exhibit all of these properties. Larger tests often violate all of these constraints. For example, larger tests are often flakier because they use more infrastructure than does a small unit test. They are also often much slower, both to set up as well as to run. And they have trouble scaling because of the resource and time requirements, but often also because they are not isolated—these tests can collide with one another.

Additionally, larger tests present two other challenges. First, there is a challenge of ownership. A unit test is clearly owned by the engineer (and team) who owns the unit. A larger test spans multiple units and thus can span multiple owners. This presents a long-term ownership challenge: who is responsible for maintaining the test and who is responsible for diagnosing issues when the test breaks? Without clear ownership, a test rots.

The second challenge for larger tests is one of standardization (or the lack thereof). Unlike unit tests, larger tests suffer a lack of standardization in terms of the infrastructure and process by which they are written, run, and debugged. The approach to larger tests is a product of a system's architectural decisions, thus introducing variance in the type of tests required. For example, the way we build and run A-B diff regression tests in Google Ads is completely different from the way such tests are built and run in Search backends, which is different again from Drive. They use different platforms, different languages, different infrastructures, different libraries, and competing testing frameworks.

This lack of standardization has a significant impact. Because larger tests have so many ways of being run, they often are skipped during large-scale changes. (See Chapter 22.) The infrastructure does not have a standard way to run those tests, and asking the people executing LSCs to know the local particulars for testing on every team doesn't scale. Because larger tests differ in implementation from team to team, tests that actually test the integration between those teams require unifying incompatible infrastructures. And because of this lack of standardization, we cannot teach a

single approach to Nooglers (new Googlers) or even more experienced engineers, which both perpetuates the situation and also leads to a lack of understanding about the motivations of such tests.

Larger Tests at Google

When we discussed the history of testing at Google earlier (see Chapter 11), we mentioned how Google Web Server (GWS) mandated automated tests in 2003 and how this was a watershed moment. However, we actually had automated tests in use before this point, but a common practice was using automated large and enormous tests. For example, AdWords created an end-to-end test back in 2001 to validate product scenarios. Similarly, in 2002, Search wrote a similar "regression test" for its indexing code, and AdSense (which had not even publicly launched yet) created its variation on the AdWords test.

Other "larger" testing patterns also existed circa 2002. The Google search frontend relied heavily on manual QA—manual versions of end-to-end test scenarios. And Gmail got its version of a "local demo" environment—a script to bring up an end-to-end Gmail environment locally with some generated test users and mail data for local manual testing.

When C/J Build (our first continuous build framework) launched, it did not distinguish between unit tests and other tests, but there were two critical developments that led to a split. First, Google focused on unit tests because we wanted to encourage the testing pyramid and to ensure the vast majority of written tests were unit tests. Second, when TAP replaced C/J Build as our formal continuous build system, it was only able to do so for tests that met TAP's eligibility requirements: hermetic tests buildable at a single change that could run on our build/test cluster within a maximum time limit. Although most unit tests satisfied this requirement, larger tests mostly did not. However, this did not stop the need for other kinds of tests, and they have continued to fill the coverage gaps. C/J Build even stuck around for years specifically to handle these kinds of tests until newer systems replaced it.

Larger Tests and Time

Throughout this book, we have looked at the influence of time on software engineering, because Google has built software running for more than 20 years. How are larger tests influenced by the time dimension? We know that certain activities make more sense the longer the expected lifespan of code, and testing of various forms is an activity that makes sense at all levels, but the test types that are appropriate change over the expected lifetime of code.

As we pointed out before, unit tests begin to make sense for software with an expected lifespan from hours on up. At the minutes level (for small scripts), manual

testing is most common, and the SUT usually runs locally, but the local demo likely *is* production, especially for one-off scripts, demos, or experiments. At longer lifespans, manual testing continues to exist, but the SUTs usually diverge because the production instance is often cloud hosted instead of locally hosted.

The remaining larger tests all provide value for longer-lived software, but the main concern becomes the maintainability of such tests as time increases.

Incidentally, this time impact might be one reason for the development of the "ice cream cone" testing antipattern, as mentioned in the Chapter 11 and shown again in Figure 14-2.

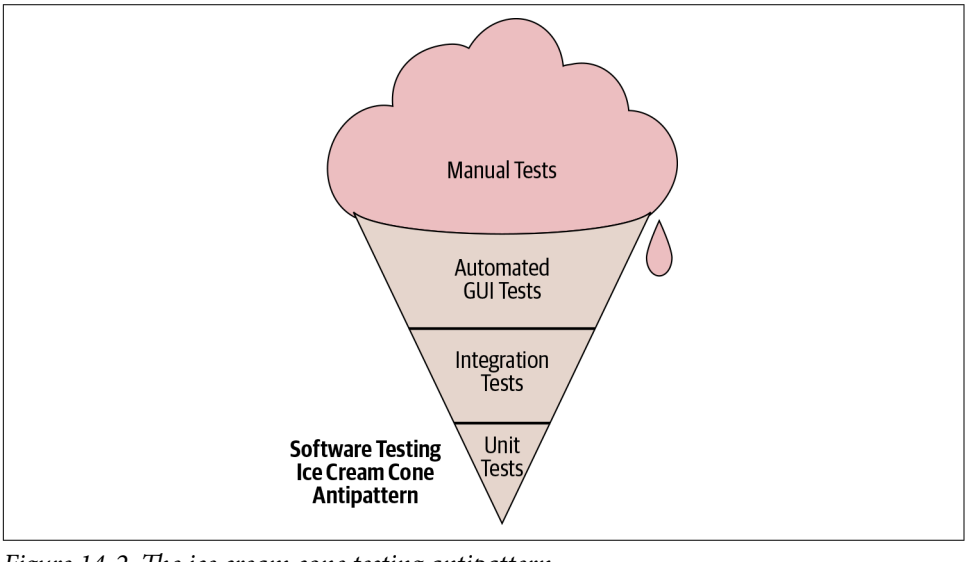

Figure 14-2. The ice cream cone testing antipattern

When development starts with manual testing (when engineers think that code is meant to last only for minutes), those manual tests accumulate and dominate the initial overall testing portfolio. For example, it's pretty typical to hack on a script or an app and test it out by running it, and then to continue to add features to it but continue to test it out by running it manually. This prototype eventually becomes functional and is shared with others, but no automated tests actually exist for it.

Even worse, if the code is difficult to unit test (because of the way it was implemented in the first place), the only automated tests that can be written are end-to-end ones, and we have inadvertently created "legacy code" within days.

It is *critical* for longer-term health to move toward the test pyramid within the first few days of development by building out unit tests, and then to top it off after that point by introducing automated integration tests and moving away from manual end-to-end tests. We succeeded by making unit tests a requirement for submission, but

covering the gap between unit tests and manual tests is necessary for long-term health.

Larger Tests at Google Scale

It would seem that larger tests should be more necessary and more appropriate at larger scales of software, but even though this is so, the complexity of authoring, running, maintaining, and debugging these tests increases with the growth in scale, even more so than with unit tests.

In a system composed of microservices or separate servers, the pattern of interconnections looks like a graph: let the number of nodes in that graph be our *N*. Every time a new node is added to this graph, there is a multiplicative effect on the number of distinct execution paths through it.

Figure 14-3 depicts an imagined SUT: this system consists of a social network with users, a social graph, a stream of posts, and some ads mixed in. The ads are created by advertisers and served in the context of the social stream. This SUT alone consists of two groups of users, two UIs, three databases, an indexing pipeline, and six servers. There are 14 edges enumerated in the graph. Testing all of the end-to-end possibilities is already difficult. Imagine if we add more services, pipelines, and databases to this mix: photos and images, machine learning photo analysis, and so on?

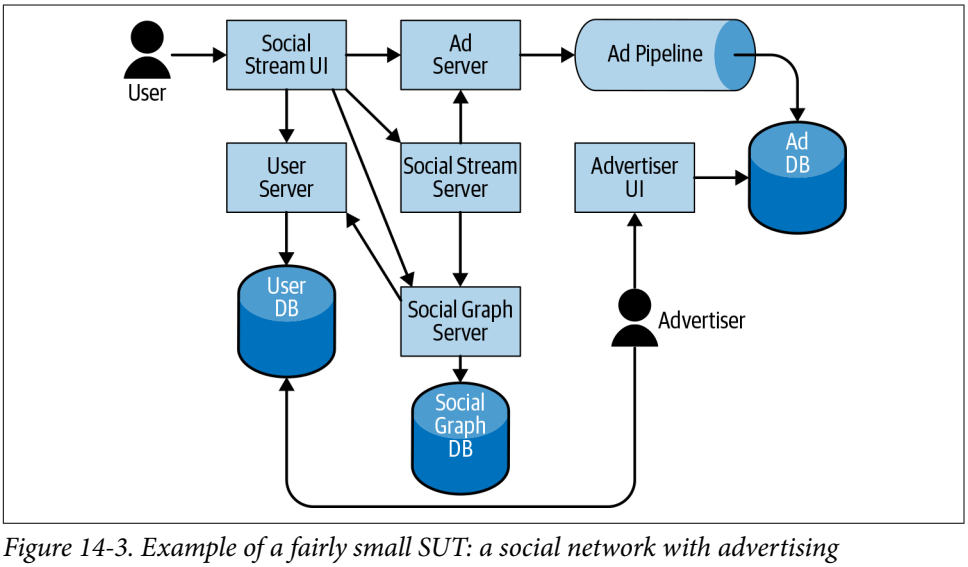

Figure 14-3. Example of a fairly small SUT: a social network with advertising

The rate of distinct scenarios to test in an end-to-end way can grow exponentially or combinatorially depending on the structure of the system under test, and that growth does not scale. Therefore, as the system grows, we must find alternative larger testing strategies to keep things manageable.

However, the value of such tests also increases because of the decisions that were necessary to achieve this scale. This is an impact of fidelity: as we move toward larger-*N* layers of software, if the service doubles are lower fidelity (1-epsilon), the chance of bugs when putting it all together is exponential in *N*. Looking at this example SUT again, if we replace the user server and ad server with doubles and those doubles are low fidelity (e.g., 10% accurate), the likelihood of a bug is 99% (1 – (0.1 ∗ 0.1)). And that's just with two low-fidelity doubles.

Therefore, it becomes critical to implement larger tests in ways that work well at this scale but maintain reasonably high fidelity.

Tip: "The Smallest Possible Test"

Even for integration tests, smaller is better—a handful of large tests is preferable to an enormous one. And, because the scope of a test is often coupled to the scope of the SUT, finding ways to make the SUT smaller help make the test smaller.

One way to achieve this test ratio when presented with a user journey that can require contributions from many internal systems is to "chain" tests, as illustrated in Figure 14-4, not specifically in their execution, but to create multiple smaller pairwise integration tests that represent the overall scenario. This is done by ensuring that the output of one test is used as the input to another test by persisting this output to a data repository.

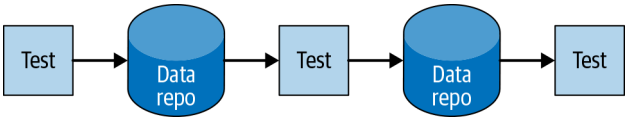

Figure 14-4. Chained tests

Structure of a Large Test

Although large tests are not bound by small test constraints and could conceivably consist of anything, most large tests exhibit common patterns. Large tests usually consist of a workflow with the following phases:

- Obtain a system under test
- Seed necessary test data
- Perform actions using the system under test
- Verify behaviors

The System Under Test

One key component of large tests is the aforementioned SUT (see Figure 14-5). A typical unit test focuses its attention on one class or module. Moreover, the test code runs in the same process (or Java Virtual Machine [JVM], in the Java case) as the code being tested. For larger tests, the SUT is often very different; one or more separate processes with test code often (but not always) in its own process.

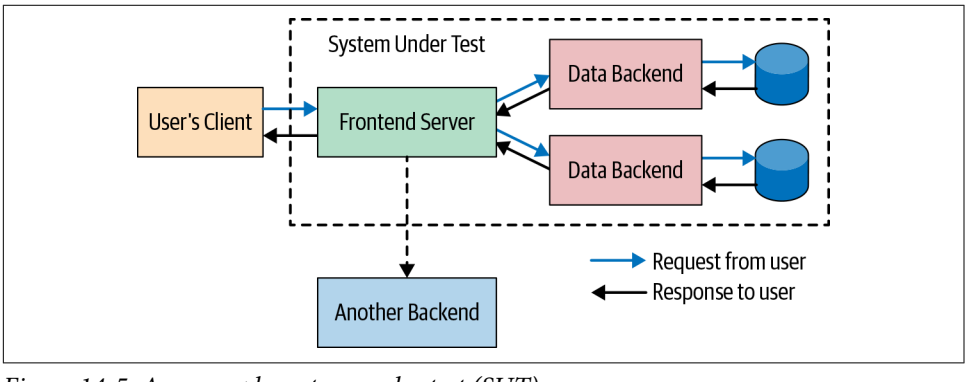

Figure 14-5. An example system under test (SUT)

At Google, we use many different forms of SUTs, and the scope of the SUT is one of the primary drivers of the scope of the large test itself (the larger the SUT, the larger the test). Each SUT form can be judged based on two primary factors:

Hermeticity
: This is the SUT's isolation from usages and interactions from other components than the test in question. An SUT with high hermeticity will have the least exposure to sources of concurrency and infrastructure flakiness.

Fidelity
: The SUT's accuracy in reflecting the production system being tested. An SUT with high fidelity will consist of binaries that resemble the production versions (rely on similar configurations, use similar infrastructures, and have a similar overall topology).

Often these two factors are in direct conflict.

Following are some examples of SUTs:

Single-process SUT
: The entire system under test is packaged into a single binary (even if in production these are multiple separate binaries). Additionally, the test code can be packaged into the same binary as the SUT. Such a test-SUT combination can be a "small" test if everything is single-threaded, but it is the least faithful to the production topology and configuration.

Single-machine SUT
: The system under test consists of one or more separate binaries (same as production) and the test is its own binary. But everything runs on one machine. This is used for "medium" tests. Ideally, we use the production launch configuration of each binary when running those binaries locally for increased fidelity.

Multimachine SUT
: The system under test is distributed across multiple machines (much like a production cloud deployment). This is even higher fidelity than the single-machine SUT, but its use makes tests "large" size and the combination is susceptible to increased network and machine flakiness.

Shared environments (staging and production)
: Instead of running a standalone SUT, the test just uses a shared environment. This has the lowest cost because these shared environments usually already exist, but the test might conflict with other simultaneous uses and one must wait for the code to be pushed to those environments. Production also increases the risk of end-user impact.

Hybrids
: Some SUTs represent a mix: it might be possible to run some of the SUT but have it interact with a shared environment. Usually the thing being tested is explicitly run but its backends are shared. For a company as expansive as Google, it is practically impossible to run multiple copies of all of Google's interconnected services, so some hybridization is required.

The benefits of hermetic SUTs

The SUT in a large test can be a major source of both unreliability and long turnaround time. For example, an in-production test uses the actual production system deployment. As mentioned earlier, this is popular because there is no extra overhead cost for the environment, but production tests cannot be run until the code reaches that environment, which means those tests cannot themselves block the release of the code to that environment—the SUT is too late, essentially.

The most common first alternative is to create a giant shared staging environment and to run tests there. This is usually done as part of some release promotion process, but it again limits test execution to only when the code is available. As an alternative, some teams will allow engineers to "reserve" time in the staging environment and to use that time window to deploy pending code and to run tests, but this does not scale with a growing number of engineers or a growing number of services, because the environment, its number of users, and the likelihood of user conflicts all quickly grow.

The next step is to support cloud-isolated or machine-hermetic SUTs. Such an environment improves the situation by avoiding the conflicts and reservation requirements for code release.

Case Study: Risks of testing in production and Webdriver Torso

We mentioned that testing in production can be risky. One humorous episode resulting from testing in production was known as the Webdriver Torso incident. We needed a way to verify that video rendering in YouTube production was working properly and so created automated scripts to generate test videos, upload them, and verify the quality of the upload. This was done in a Google-owned YouTube channel called Webdriver Torso. But this channel was public, as were most of the videos.

Subsequently, this channel was publicized in an article at Wired, which led to its spread throughout the media and subsequent efforts to solve the mystery. Finally, a blogger tied everything back to Google. Eventually, we came clean by having a bit of fun with it, including a Rickroll and an Easter Egg, so everything worked out well. But we do need to think about the possibility of end-user discovery of any test data we include in production and be prepared for it.

Reducing the size of your SUT at problem boundaries

There are particularly painful testing boundaries that might be worth avoiding. Tests that involve both frontends and backends become painful because user interface (UI) tests are notoriously unreliable and costly:

- UIs often change in look-and-feel ways that make UI tests brittle but do not actually impact the underlying behavior.
- UIs often have asynchronous behaviors that are difficult to test.

Although it is useful to have end-to-end tests of a UI of a service all the way to its backend, these tests have a multiplicative maintenance cost for both the UI and the backends. Instead, if the backend provides a public API, it is often easier to split the tests into connected tests at the UI/API boundary and to use the public API to drive the end-to-end tests. This is true whether the UI is a browser, command-line interface (CLI), desktop app, or mobile app.

Another special boundary is for third-party dependencies. Third-party systems might not have a public shared environment for testing, and in some cases, there is a cost with sending traffic to a third party. Therefore, it is not recommended to have automated tests use a real third-party API, and that dependency is an important seam at which to split tests.

To address this issue of size, we have made this SUT smaller by replacing its databases with in-memory databases and removing one of the servers outside the scope of the SUT that we actually care about, as shown in Figure 14-6. This SUT is more likely to fit on a single machine.

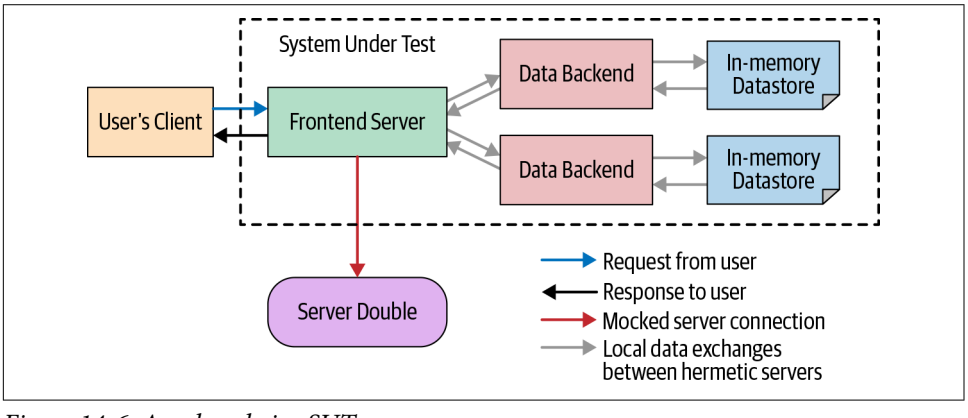

Figure 14-6. A reduced-size SUT

The key is to identify trade-offs between fidelity and cost/reliability, and to identify reasonable boundaries. If we can run a handful of binaries and a test and pack it all into the same machines that do our regular compiles, links, and unit test executions, we have the easiest and most stable "integration" tests for our engineers.

Record/replay proxies

In the previous chapter, we discussed test doubles and approaches that can be used to decouple the class under test from its difficult-to-test dependencies. We can also double entire servers and processes by using a mock, stub, or fake server or process with the equivalent API. However, there is no guarantee that the test double used actually conforms to the contract of the real thing that it is replacing.

One way of dealing with an SUT's dependent but subsidiary services is to use a test double, but how does one know that the double reflects the dependency's actual behavior? A growing approach outside of Google is to use a framework for consumer-driven contract tests. These are tests that define a contract for both the client and the provider of the service, and this contract can drive automated tests. That is, a client defines a mock of the service saying that, for these input arguments, I get a particular output. Then, the real service uses this input/output pair in a real test to ensure that it produces that output given those inputs. Two public tools for consumer-driven contract testing are Pact Contract Testing and Spring Cloud Contracts. Google's heavy dependency on protocol buffers means that we don't use these internally.

At Google, we do something a little bit different. Our most popular approach (for which there is a public API) is to use a larger test to generate a smaller one by recording the traffic to those external services when running the larger test and replaying it when running smaller tests. The larger, or "Record Mode" test runs continuously on post-submit, but its primary purpose is to generate these traffic logs (it must pass, however, for the logs to be generated). The smaller, or "Replay Mode" test is used during development and presubmit testing.

One of the interesting aspects of how record/replay works is that, because of nondeterminism, requests must be matched via a matcher to determine which response to replay. This makes them very similar to stubs and mocks in that argument matching is used to determine the resulting behavior.

What happens for new tests or tests where the client behavior changes significantly? In these cases, a request might no longer match what is in the recorded traffic file, so the test cannot pass in Replay mode. In that circumstance, the engineer must run the test in Record mode to generate new traffic, so it is important to make running Record tests easy, fast, and stable.

Test Data

A test needs data, and a large test needs two different kinds of data:

Seeded data
: Data preinitialized into the system under test reflecting the state of the SUT at the inception of the test

Test traffic
: Data sent to the system under test by the test itself during its execution

Because of the notion of the separate and larger SUT, the work to seed the SUT state is often orders of magnitude more complex than the setup work done in a unit test. For example:

Domain data
: Some databases contain data prepopulated into tables and used as configuration for the environment. Actual service binaries using such a database may fail on startup if domain data is not provided.

Realistic baseline
: For an SUT to be perceived as realistic, it might require a realistic set of base data at startup, both in terms of quality and quantity. For example, large tests of a social network likely need a realistic social graph as the base state for tests: enough test users with realistic profiles as well as enough interconnections between those users must exist for the testing to be accepted.

Seeding APIs
: The APIs by which data is seeded may be complex. It might be possible to directly write to a datastore, but doing so might bypass triggers and checks performed by the actual binaries that perform the writes.

Data can be generated in different ways, such as the following:

Handcrafted data
: Like for smaller tests, we can create test data for larger tests by hand. But it might require more work to set up data for multiple services in a large SUT, and we might need to create a lot of data for larger tests.

Copied data
: We can copy data, typically from production. For example, we might test a map of Earth by starting with a copy of our production map data to provide a baseline and then test our changes to it.

Sampled data
: Copying data can provide too much data to reasonably work with. Sampling data can reduce the volume, thus reducing test time and making it easier to reason about. "Smart sampling" consists of techniques to copy the minimum data necessary to achieve maximum coverage.

Verification

After an SUT is running and traffic is sent to it, we must still verify the behavior. There are a few different ways to do this:

Manual
: Much like when you try out your binary locally, manual verification uses humans to interact with an SUT to determine whether it functions correctly. This verification can consist of testing for regressions by performing actions as defined on a consistent test plan or it can be exploratory, working a way through different interaction paths to identify possible new failures.

  Note that manual regression testing does not scale sublinearly: the larger a system grows and the more journeys through it there are, the more human time is needed to manually test.

Assertions
: Much like with unit tests, these are explicit checks about the intended behavior of the system. For example, for an integration test of Google search of `xyzzy`, an assertion might be as follows:

```
assertThat(response.Contains("Colossal Cave"))
```

A/B comparison (differential)
: Instead of defining explicit assertions, A/B testing involves running two copies of the SUT, sending the same data, and comparing the output. The intended behavior is not explicitly defined: a human must manually go through the differences to ensure any changes are intended.

Types of Larger Tests

We can now combine these different approaches to the SUT, data, and assertions to create different kinds of large tests. Each test then has different properties as to which risks it mitigates; how much toil is required to write, maintain, and debug it; and how much it costs in terms of resources to run.

What follows is a list of different kinds of large tests that we use at Google, how they are composed, what purpose they serve, and what their limitations are:

- Functional testing of one or more binaries
- Browser and device testing
- Performance, load, and stress testing
- Deployment configuration testing
- Exploratory testing
- A/B diff (regression) testing
- User acceptance testing (UAT)
- Probers and canary analysis
- Disaster recovery and chaos engineering
- User evaluation

Given such a wide number of combinations and thus a wide range of tests, how do we manage what to do and when? Part of designing software is drafting the test plan, and a key part of the test plan is a strategic outline of what types of testing are needed and how much of each. This test strategy identifies the primary risk vectors and the necessary testing approaches to mitigate those risk vectors.

At Google, we have a specialized engineering role of "Test Engineer," and one of the things we look for in a good test engineer is the ability to outline a test strategy for our products.

Functional Testing of One or More Interacting Binaries

Tests of these type have the following characteristics:

- SUT: single-machine hermetic or cloud-deployed isolated
- Data: handcrafted
- Verification: assertions

As we have seen so far, unit tests are not capable of testing a complex system with true fidelity, simply because they are packaged in a different way than the real code is packaged. Many functional testing scenarios interact with a given binary differently than with classes inside that binary, and these functional tests require separate SUTs and thus are canonical, larger tests.

Testing the interactions of multiple binaries is, unsurprisingly, even more complicated than testing a single binary. A common use case is within microservices environments when services are deployed as many separate binaries. In this case, a functional test can cover the real interactions between the binaries by bringing up an SUT composed of all the relevant binaries and by interacting with it through a published API.

Browser and Device Testing

Testing web UIs and mobile applications is a special case of functional testing of one or more interacting binaries. It is possible to unit test the underlying code, but for the end users, the public API is the application itself. Having tests that interact with the application as a third party through its frontend provides an extra layer of coverage.

Performance, Load, and Stress testing

Tests of these type have the following characteristics:

- SUT: cloud-deployed isolated
- Data: handcrafted or multiplexed from production
- Verification: diff (performance metrics)

Although it is possible to test a small unit in terms of performance, load, and stress, often such tests require sending simultaneous traffic to an external API. That definition implies that such tests are multithreaded tests that usually test at the scope of a binary under test. However, these tests are critical for ensuring that there is no degradation in performance between versions and that the system can handle expected spikes in traffic.

As the scale of the load test grows, the scope of the input data also grows, and it eventually becomes difficult to generate the scale of load required to trigger bugs under load. Load and stress handling are "highly emergent" properties of a system; that is, these complex behaviors belong to the overall system but not the individual members. Therefore, it is important to make these tests look as close to production as possible. Each SUT requires resources akin to what production requires, and it becomes difficult to mitigate noise from the production topology.

One area of research for eliminating noise in performance tests is in modifying the deployment topology—how the various binaries are distributed across a network of machines. The machine running a binary can affect the performance characteristics; thus, if in a performance diff test, the base version runs on a fast machine (or one with a fast network) and the new version on a slow one, it can appear like a performance regression. This characteristic implies that the optimal deployment is to run both versions on the same machine. If a single machine cannot fit both versions of the binary, an alternative is to calibrate by performing multiple runs and removing peaks and valleys.

Deployment Configuration Testing

Tests of these type have the following characteristics:

- SUT: single-machine hermetic or cloud-deployed isolated
- Data: none
- Verification: assertions (doesn't crash)

Many times, it is not the code that is the source of defects but instead configuration: data files, databases, option definitions, and so on. Larger tests can test the integration of the SUT with its configuration files because these configuration files are read during the launch of the given binary.

Such a test is really a smoke test of the SUT without needing much in the way of additional data or verification. If the SUT starts successfully, the test passes. If not, the test fails.

Exploratory Testing

Tests of these type have the following characteristics:

- SUT: production or shared staging
- Data: production or a known test universe
- Verification: manual

Exploratory testing[2] is a form of manual testing that focuses not on looking for behavioral regressions by repeating known test flows, but on looking for questionable behavior by trying out new user scenarios. Trained users/testers interact with a product through its public APIs, looking for new paths through the system and for which behavior deviates from either expected or intuitive behavior, or if there are security vulnerabilities.

Exploratory testing is useful for both new and launched systems to uncover unanticipated behaviors and side effects. By having testers follow different reachable paths through the system, we can increase the system coverage and, when these testers identify bugs, capture new automated functional tests. In a sense, this is a bit like a manual "fuzz testing" version of functional integration testing.

Limitations

Manual testing does not scale sublinearly; that is, it requires human time to perform the manual tests. Any defects found by exploratory tests should be replicated with an automated test that can run much more frequently.

Bug bashes

One common approach we use for manual exploratory testing is the bug bash. A team of engineers and related personnel (managers, product managers, test engineers, anyone with familiarity with the product) schedules a "meeting," but at this session, everyone involved manually tests the product. There can be some published guidelines as to particular focus areas for the bug bash and/or starting points for using the system, but the goal is to provide enough interaction variety to document questionable product behaviors and outright bugs.

A/B Diff Regression Testing

Tests of these type have the following characteristics:

- SUT: two cloud-deployed isolated environments
- Data: usually multiplexed from production or sampled
- Verification: A/B diff comparison

Unit tests cover expected behavior paths for a small section of code. But it is impossible to predict many of the possible failure modes for a given publicly facing product. Additionally, as Hyrum's Law states, the actual public API is not the declared one but

2 James A. Whittaker, *Exploratory Software Testing: Tips, Tricks, Tours, and Techniques to Guide Test Design* (New York: Addison-Wesley Professional, 2009).

all user-visible aspects of a product. Given those two properties, it is no surprise that A/B diff tests are possibly the most common form of larger testing at Google. This approach conceptually dates back to 1998. At Google, we have been running tests based on this model since 2001 for most of our products, starting with Ads, Search, and Maps.

A/B diff tests operate by sending traffic to a public API and comparing the responses between old and new versions (especially during migrations). Any deviations in behavior must be reconciled as either anticipated or unanticipated (regressions). In this case, the SUT is composed of two sets of real binaries: one running at the candidate version and the other running at the base version. A third binary sends traffic and compares the results.

There are other variants. We use A-A testing (comparing a system to itself) to identify nondeterministic behavior, noise, and flakiness, and to help remove those from A-B diffs. We also occasionally use A-B-C testing, comparing the last production version, the baseline build, and a pending change, to make it easy at one glance to see not only the impact of an immediate change, but also the accumulated impacts of what would be the next-to-release version.

A/B diff tests are a cheap but automatable way to detect unanticipated side effects for any launched system.

Limitations

Diff testing does introduce a few challenges to solve:

Approval
: Someone must understand the results enough to know whether any differences are expected. Unlike a typical test, it is not clear whether diffs are a good or bad thing (or whether the baseline version is actually even valid), and so there is often a manual step in the process.

Noise
: For a diff test, anything that introduces unanticipated noise into the results leads to more manual investigation of the results. It becomes necessary to remediate noise, and this is a large source of complexity in building a good diff test.

Coverage
: Generating enough useful traffic for a diff test can be a challenging problem. The test data must cover enough scenarios to identify corner-case differences, but it is difficult to manually curate such data.

Setup
: Configuring and maintaining one SUT is fairly challenging. Creating two at a time can double the complexity, especially if these share interdependencies.

UAT

Tests of these type have the following characteristics:

- SUT: machine-hermetic or cloud-deployed isolated
- Data: handcrafted
- Verification: assertions

A key aspect of unit tests is that they are written by the developer writing the code under test. But that makes it quite likely that misunderstandings about the *intended* behavior of a product are reflected not only in the code, but also the unit tests. Such unit tests verify that code is "Working as implemented" instead of "Working as intended."

For cases in which there is either a specific end customer or a customer proxy (a customer committee or even a product manager), UATs are automated tests that exercise the product through public APIs to ensure the overall behavior for specific user journeys is as intended. Multiple public frameworks exist (e.g., Cucumber and RSpec) to make such tests writable/readable in a user-friendly language, often in the context of "runnable specifications."

Google does not actually do a lot of automated UAT and does not use specification languages very much. Many of Google's products historically have been created by the software engineers themselves. There has been little need for runnable specification languages because those defining the intended product behavior are often fluent in the actual coding languages themselves.

Probers and Canary Analysis

Tests of these type have the following characteristics:

- SUT: production
- Data: production
- Verification: assertions and A/B diff (of metrics)

Probers and canary analysis are ways to ensure that the production environment itself is healthy. In these respects, they are a form of production monitoring, but they are structurally very similar to other large tests.

Probers are functional tests that run encoded assertions against the production environment. Usually these tests perform well-known and deterministic read-only actions so that the assertions hold even though the production data changes over time. For example, a prober might perform a Google search at www.google.com and verify that a result is returned, but not actually verify the contents of the result. In that respect,

they are "smoke tests" of the production system, but they provide early detection of major issues.

Canary analysis is similar, except that it focuses on when a release is being pushed to the production environment. If the release is staged over time, we can run both prober assertions targeting the upgraded (canary) services as well as compare health metrics of both the canary and baseline parts of production and make sure that they are not out of line.

Probers should be used in any live system. If the production rollout process includes a phase in which the binary is deployed to a limited subset of the production machines (a canary phase), canary analysis should be used during that procedure.

Limitations

Any issues caught at this point in time (in production) are already affecting end users.

If a prober performs a mutable (write) action, it will modify the state of production. This could lead to one of three outcomes: nondeterminism and failure of the assertions, failure of the ability to write in the future, or user-visible side effects.

Disaster Recovery and Chaos Engineering

Tests of these type have the following characteristics:

- SUT: production
- Data: production and user-crafted (fault injection)
- Verification: manual and A/B diff (metrics)

These test how well your systems will react to unexpected changes or failures.

For years, Google has run an annual war game called DiRT (Disaster Recovery Testing) during which faults are injected into our infrastructure at a nearly planetary scale. We simulate everything from datacenter fires to malicious attacks. In one memorable case, we simulated an earthquake that completely isolated our headquarters in Mountain View, California, from the rest of the company. Doing so exposed not only technical shortcomings but also revealed the challenge of running a company when all the key decision makers were unreachable.[3]

The impacts of DiRT tests require a lot of coordination across the company; by contrast, chaos engineering is more of a "continuous testing" for your technical infrastructure. Made popular by Netflix, chaos engineering involves writing programs that

3 During this test, almost no one could get anything done, so many people gave up on work and went to one of our many cafes, and in doing so, we ended up creating a DDoS attack on our cafe teams!

continuously introduce a background level of faults into your systems and seeing what happens. Some of the faults can be quite large, but in most cases, chaos testing tools are designed to restore functionality before things get out of hand. The goal of chaos engineering is to help teams break assumptions of stability and reliability and help them grapple with the challenges of building resiliency in. Today, teams at Google perform thousands of chaos tests each week using our own home-grown system called Catzilla.

These kinds of fault and negative tests make sense for live production systems that have enough theoretical fault tolerance to support them and for which the costs and risks of the tests themselves are affordable.

Limitations

Any issues caught at this point in time (in production) are already affecting end users.

DiRT is quite expensive to run, and therefore we run a coordinated exercise on an infrequent scale. When we create this level of outage, we actually cause pain and negatively impact employee performance.

If a prober performs a mutable (write) action, it will modify the state of production. This could lead to either nondeterminism and failure of the assertions, failure of the ability to write in the future, or user-visible side effects.

User Evaluation

Tests of these type have the following characteristics:

- SUT: production
- Data: production
- Verification: manual and A/B diffs (of metrics)

Production-based testing makes it possible to collect a lot of data about user behavior. We have a few different ways to collect metrics about the popularity of and issues with upcoming features, which provides us with an alternative to UAT:

Dogfooding
: It's possible using limited rollouts and experiments to make features in production available to a subset of users. We do this with our own staff sometimes (eat our own dogfood), and they give us valuable feedback in the real deployment environment.

Experimentation
: A new behavior is made available as an experiment to a subset of users without their knowing. Then, the experiment group is compared to the control group at an aggregate level in terms of some desired metric. For example, in YouTube, we

had a limited experiment changing the way video upvotes worked (eliminating the downvote), and only a portion of the user base saw this change.

This is a massively important approach for Google. One of the first stories a Noogler hears upon joining the company is about the time Google launched an experiment changing the background shading color for AdWords ads in Google Search and noticed a significant increase in ad clicks for users in the experimental group versus the control group.

Rater evaluation

Human raters are presented with results for a given operation and choose which one is "better" and why. This feedback is then used to determine whether a given change is positive, neutral, or negative. For example, Google has historically used rater evaluation for search queries (we have published the guidelines we give our raters). In some cases, the feedback from this ratings data can help determine launch go/no-go for algorithm changes. Rater evaluation is critical for nondeterministic systems like machine learning systems for which there is no clear correct answer, only a notion of better or worse.

Large Tests and the Developer Workflow

We've talked about what large tests are, why to have them, when to have them, and how much to have, but we have not said much about the who. Who writes the tests? Who runs the tests and investigates the failures? Who owns the tests? And how do we make this tolerable?

Although standard unit test infrastructure might not apply, it is still critical to integrate larger tests into the developer workflow. One way of doing this is to ensure that automated mechanisms for presubmit and post-submit execution exist, even if these are different mechanisms than the unit test ones. At Google, many of these large tests do not belong in TAP. They are nonhermetic, too flaky, and/or too resource intensive. But we still need to keep them from breaking or else they provide no signal and become too difficult to triage. What we do, then, is to have a separate post-submit continuous build for these. We also encourage running these tests presubmit, because that provides feedback directly to the author.

A/B diff tests that require manual blessing of diffs can also be incorporated into such a workflow. For presubmit, it can be a code-review requirement to approve any diffs in the UI before approving the change. One such test we have files release-blocking bugs automatically if code is submitted with unresolved diffs.

In some cases, tests are so large or painful that presubmit execution adds too much developer friction. These tests still run post-submit and are also run as part of the release process. The drawback to not running these presubmit is that the taint makes it into the monorepo and we need to identify the culprit change to roll it back. But we

need to make the trade-off between developer pain and the incurred change latency and the reliability of the continuous build.

Authoring Large Tests

Although the structure of large tests is fairly standard, there is still a challenge with creating such a test, especially if it is the first time someone on the team has done so.

The best way to make it possible to write such tests is to have clear libraries, documentation, and examples. Unit tests are easy to write because of native language support (JUnit was once esoteric but is now mainstream). We reuse these assertion libraries for functional integration tests, but we also have created over time libraries for interacting with SUTs, for running A/B diffs, for seeding test data, and for orchestrating test workflows.

Larger tests are more expensive to maintain, in both resources and human time, but not all large tests are created equal. One reason that A/B diff tests are popular is that they have less human cost in maintaining the verification step. Similarly, production SUTs have less maintenance cost than isolated hermetic SUTs. And because all of this authored infrastructure and code must be maintained, the cost savings can compound.

However, this cost must be looked at holistically. If the cost of manually reconciling diffs or of supporting and safeguarding production testing outweighs the savings, it becomes ineffective.

Running Large Tests

We mentioned above how our larger tests don't fit in TAP and so we have alternate continuous builds and presubmits for them. One of the initial challenges for our engineers is how to even run nonstandard tests and how to iterate on them.

As much as possible, we have tried to make our larger tests run in ways familiar for our engineers. Our presubmit infrastructure puts a common API in front of running both these tests and running TAP tests, and our code review infrastructure shows both sets of results. But many large tests are bespoke and thus need specific documentation for how to run them on demand. This can be a source of frustration for unfamiliar engineers.

Speeding up tests

Engineers don't wait for slow tests. The slower a test is, the less frequently an engineer will run it, and the longer the wait after a failure until it is passing again.

The best way to speed up a test is often to reduce its scope or to split a large test into two smaller tests that can run in parallel. But there are some other tricks that you can do to speed up larger tests.

Some naive tests will use time-based sleeps to wait for nondeterministic action to occur, and this is quite common in larger tests. However, these tests do not have thread limitations, and real production users want to wait as little as possible, so it is best for tests to react the way real production users would. Approaches include the following:

- Polling for a state transition repeatedly over a time window for an event to complete with a frequency closer to microseconds. You can combine this with a timeout value in case a test fails to reach a stable state.
- Implementing an event handler.
- Subscribing to a notification system for an event completion.

Note that tests that rely on sleeps and timeouts will all start failing when the fleet running those tests becomes overloaded, which spirals because those tests need to be rerun more often, increasing the load further.

Lower internal system timeouts and delays
: A production system is usually configured assuming a distributed deployment topology, but an SUT might be deployed on a single machine (or at least a cluster of colocated machines). If there are hardcoded timeouts or (especially) sleep statements in the production code to account for production system delay, these should be made tunable and reduced when running tests.

Optimize test build time
: One downside of our monorepo is that all of the dependencies for a large test are built and provided as inputs, but this might not be necessary for some larger tests. If the SUT is composed of a core part that is truly the focus of the test and some other necessary peer binary dependencies, it might be possible to use prebuilt versions of those other binaries at a known good version. Our build system (based on the monorepo) does not support this model easily, but the approach is actually more reflective of production in which different services release at different versions.

Driving out flakiness

Flakiness is bad enough for unit tests, but for larger tests, it can make them unusable. A team should view eliminating flakiness of such tests as a high priority. But how can flakiness be removed from such tests?

Minimizing flakiness starts with reducing the scope of the test—a hermetic SUT will not be at risk of the kinds of multiuser and real-world flakiness of production or a shared staging environment, and a single-machine hermetic SUT will not have the network and deployment flakiness issues of a distributed SUT. But you can mitigate other flakiness issues through test design and implementation and other techniques. In some cases, you will need to balance these with test speed.

Just as making tests reactive or event driven can speed them up, it can also remove flakiness. Timed sleeps require timeout maintenance, and these timeouts can be embedded in the test code. Increasing internal system timeouts can reduce flakiness, whereas reducing internal timeouts can lead to flakiness if the system behaves in a nondeterministic way. The key here is to identify a trade-off that defines both a tolerable system behavior for end users (e.g., our maximum allowable timeout is *n* seconds) but handles flaky test execution behaviors well.

A bigger problem with internal system timeouts is that exceeding them can lead to difficult errors to triage. A production system will often try to limit end-user exposure to catastrophic failure by handling possible internal system issues gracefully. For example, if Google cannot serve an ad in a given time limit, we don't return a 500, we just don't serve an ad. But this looks to a test runner as if the ad-serving code might be broken when there is just a flaky timeout issue. It's important to make the failure mode obvious in this case and to make it easy to tune such internal timeouts for test scenarios.

Making tests understandable

A specific case for which it can be difficult to integrate tests into the developer workflow is when those tests produce results that are unintelligible to the engineer running the tests. Even unit tests can produce some confusion—if my change breaks your test, it can be difficult to understand why if I am generally unfamiliar with your code—but for larger tests, such confusion can be insurmountable. Tests that are assertive must provide a clear pass/fail signal and must provide meaningful error output to help triage the source of failure. Tests that require human investigation, like A/B diff tests, require special handling to be meaningful or else risk being skipped during presubmit.

How does this work in practice? A good large test that fails should do the following:

Have a message that clearly identifies what the failure is

The worst-case scenario is to have an error that just says "Assertion failed" and a stack trace. A good error anticipates the test runner's unfamiliarity with the code and provides a message that gives context: "In test_ReturnsOneFullPageOfSearchResultsForAPopularQuery, expected 10 search results but got 1." For a performance or A/B diff test that fails, there should be a clear explanation in the output of what is being measured and why the behavior is considered suspect.

Minimize the effort necessary to identify the root cause of the discrepancy
: A stack trace is not useful for larger tests because the call chain can span multiple process boundaries. Instead, it's necessary to produce a trace across the call chain or to invest in automation that can narrow down the culprit. The test should produce some kind of artifact to this effect. For example, Dapper is a framework used by Google to associate a single request ID with all the requests in an RPC call chain, and all of the associated logs for that request can be correlated by that ID to facilitate tracing.

Provide support and contact information.
: It should be easy for the test runner to get help by making the owners and supporters of the test easy to contact.

Owning Large Tests

Larger tests must have documented owners—engineers who can adequately review changes to the test and who can be counted on to provide support in the case of test failures. Without proper ownership, a test can fall victim to the following:

- It becomes more difficult for contributors to modify and update the test
- It takes longer to resolve test failures

And the test rots.

Integration tests of components within a particular project should be owned by the project lead. Feature-focused tests (tests that cover a particular business feature across a set of services) should be owned by a "feature owner"; in some cases, this owner might be a software engineer responsible for the feature implementation end to end; in other cases it might be a product manager or a "test engineer" who owns the description of the business scenario. Whoever owns the test must be empowered to ensure its overall health and must have both the ability to support its maintenance and the incentives to do so.

It is possible to build automation around test owners if this information is recorded in a structured way. Some approaches that we use include the following:

Regular code ownership
: In many cases, a larger test is a standalone code artifact that lives in a particular location in our codebase. In that case, we can use the OWNERS (Chapter 9) information already present in the monorepo to hint to automation that the owner(s) of a particular test are the owners of the test code.

Per-test annotations
: In some cases, multiple test methods can be added to a single test class or module, and each of these test methods can have a different feature owner. We use

per-language structured annotations to document the test owner in each of these cases so that if a particular test method fails, we can identify the owner to contact.

Conclusion

A comprehensive test suite requires larger tests, both to ensure that tests match the fidelity of the system under test and to address issues that unit tests cannot adequately cover. Because such tests are necessarily more complex and slower to run, care must be taken to ensure such larger tests are properly owned, well maintained, and run when necessary (such as before deployments to production). Overall, such larger tests must still be made as small as possible (while still retaining fidelity) to avoid developer friction. A comprehensive test strategy that identifies the risks of a system, and the larger tests that address them, is necessary for most software projects.

TL;DRs

- Larger tests cover things unit tests cannot.
- Large tests are composed of a System Under Test, Data, Action, and Verification.
- A good design includes a test strategy that identifies risks and larger tests that mitigate them.
- Extra effort must be made with larger tests to keep them from creating friction in the developer workflow.

CHAPTER 15
Deprecation

Written by Hyrum Wright
Edited by Tom Manshreck

I love deadlines. I like the whooshing sound they make as they fly by.
—Douglas Adams

All systems age. Even though software is a digital asset and the physical bits themselves don't degrade, new technologies, libraries, techniques, languages, and other environmental changes over time render existing systems obsolete. Old systems require continued maintenance, esoteric expertise, and generally more work as they diverge from the surrounding ecosystem. It's often better to invest effort in turning off obsolete systems, rather than letting them lumber along indefinitely alongside the systems that replace them. But the number of obsolete systems still running suggests that, in practice, doing so is not trivial. We refer to the process of orderly migration away from and eventual removal of obsolete systems as *deprecation*.

Deprecation is yet another topic that more accurately belongs to the discipline of software engineering than programming because it requires thinking about how to manage a system over time. For long-running software ecosystems, planning for and executing deprecation correctly reduces resource costs and improves velocity by removing the redundancy and complexity that builds up in a system over time. On the other hand, poorly deprecated systems may cost more than leaving them alone. While deprecating systems requires additional effort, it's possible to plan for deprecation during the design of the system so that it's easier to eventually decommission and remove it. Deprecations can affect systems ranging from individual function calls to entire software stacks. For concreteness, much of what follows focuses on code-level deprecations.

Unlike with most of the other topics we have discussed in this book, Google is still learning how best to deprecate and remove software systems. This chapter describes

the lessons we've learned as we've deprecated large and heavily used internal systems. Sometimes, it works as expected, and sometimes it doesn't, but the general problem of removing obsolete systems remains a difficult and evolving concern in the industry.

This chapter primarily deals with deprecating technical systems, not end-user products. The distinction is somewhat arbitrary given that an external-facing API is just another sort of product, and an internal API may have consumers that consider themselves end users. Although many of the principles apply to turning down a public product, we concern ourselves here with the technical and policy aspects of deprecating and removing obsolete systems where the system owner has visibility into its use.

Why Deprecate?

Our discussion of deprecation begins from the fundamental premise that *code is a liability, not an asset*. After all, if code were an asset, why should we even bother spending time trying to turn down and remove obsolete systems? Code has costs, some of which are borne in the process of creating a system, but many other costs are borne as a system is maintained across its lifetime. These ongoing costs, such as the operational resources required to keep a system running or the effort to continually update its codebase as surrounding ecosystems evolve, mean that it's worth evaluating the trade-offs between keeping an aging system running or working to turn it down.

The age of a system alone doesn't justify its deprecation. A system could be finely crafted over several years to be the epitome of software form and function. Some software systems, such as the LaTeX typesetting system, have been improved over the course of decades, and even though changes still happen, they are few and far between. Just because something is old, it does not follow that it is obsolete.

Deprecation is best suited for systems that are demonstrably obsolete and a replacement exists that provides comparable functionality. The new system might use resources more efficiently, have better security properties, be built in a more sustainable fashion, or just fix bugs. Having two systems to accomplish the same thing might not seem like a pressing problem, but over time, the costs of maintaining them both can grow substantially. Users may need to use the new system, but still have dependencies that use the obsolete one.

The two systems might need to interface with each other, requiring complicated transformation code. As both systems evolve, they may come to depend on each other, making eventual removal of either more difficult. In the long run, we've discovered that having multiple systems performing the same function also impedes the evolution of the newer system because it is still expected to maintain compatibility

with the old one. Spending the effort to remove the old system can pay off as the replacement system can now evolve more quickly.

> Earlier we made the assertion that "code is a liability, not an asset." If that is true, why have we spent most of this book discussing the most efficient way to build software systems that can live for decades? Why put all that effort into creating more code when it's simply going to end up on the liability side of the balance sheet?
>
> Code *itself* doesn't bring value: it is the *functionality* that it provides that brings value. That functionality is an asset if it meets a user need: the code that implements this functionality is simply a means to that end. If we could get the same functionality from a single line of maintainable, understandable code as 10,000 lines of convoluted spaghetti code, we would prefer the former. Code itself carries a cost—the simpler the code is, while maintaining the same amount of functionality, the better.
>
> Instead of focusing on how much code we can produce, or how large is our codebase, we should instead focus on how much functionality it can deliver per unit of code and try to maximize that metric. One of the easiest ways to do so isn't writing more code and hoping to get more functionality; it's removing excess code and systems that are no longer needed. Deprecation policies and procedures make this possible.

Even though deprecation is useful, we've learned at Google that organizations have a limit on the amount of deprecation work that is reasonable to undergo simultaneously, from the aspect of the teams doing the deprecation as well as the customers of those teams. For example, although everybody appreciates having freshly paved roads, if the public works department decided to close down *every* road for paving simultaneously, nobody would go anywhere. By focusing their efforts, paving crews can get specific jobs done faster while also allowing other traffic to make progress. Likewise, it's important to choose deprecation projects with care and then commit to following through on finishing them.

Why Is Deprecation So Hard?

We've mentioned Hyrum's Law elsewhere in this book, but it's worth repeating its applicability here: the more users of a system, the higher the probability that users are using it in unexpected and unforeseen ways, and the harder it will be to deprecate and remove such a system. Their usage just "happens to work" instead of being "guaranteed to work." In this context, removing a system can be thought of as the ultimate change: we aren't just changing behavior, we are removing that behavior completely! This kind of radical alteration will shake loose a number of unexpected dependents.

To further complicate matters, deprecation usually isn't an option until a newer system is available that provides the same (or better!) functionality. The new system

might be better, but it is also different: after all, if it were exactly the same as the obsolete system, it wouldn't provide any benefit to users who migrate to it (though it might benefit the team operating it). This functional difference means a one-to-one match between the old system and the new system is rare, and every use of the old system must be evaluated in the context of the new one.

Another surprising reluctance to deprecate is emotional attachment to old systems, particularly those that the deprecator had a hand in helping to create. An example of this change aversion happens when systematically removing old code at Google: we've occasionally encountered resistance of the form "I like this code!" It can be difficult to convince engineers to tear down something they've spent years building. This is an understandable response, but ultimately self-defeating: if a system is obsolete, it has a net cost on the organization and should be removed. One of the ways we've addressed concerns about keeping old code within Google is by ensuring that the source code repository isn't just searchable at trunk, but also historically. Even code that has been removed can be found again (see Chapter 17).

> There's an old joke within Google that there are two ways of doing things: the one that's deprecated, and the one that's not-yet-ready. This is usually the result of a new solution being "almost" done and is the unfortunate reality of working in a technological environment that is complex and fast-paced.
>
> Google engineers have become used to working in this environment, but it can still be disconcerting. Good documentation, plenty of signposts, and teams of experts helping with the deprecation and migration process all make it easier to know whether you should be using the old thing, with all its warts, or the new one, with all its uncertainties.

Finally, funding and executing deprecation efforts can be difficult politically; staffing a team and spending time removing obsolete systems costs real money, whereas the costs of doing nothing and letting the system lumber along unattended are not readily observable. It can be difficult to convince the relevant stakeholders that deprecation efforts are worthwhile, particularly if they negatively impact new feature development. Research techniques, such as those described in Chapter 7, can provide concrete evidence that a deprecation is worthwhile.

Given the difficulty in deprecating and removing obsolete software systems, it is often easier for users to evolve a system *in situ*, rather than completely replacing it. Incrementality doesn't avoid the deprecation process altogether, but it does break it down into smaller, more manageable chunks that can yield incremental benefits. Within Google, we've observed that migrating to entirely new systems is *extremely* expensive, and the costs are frequently underestimated. Incremental deprecation efforts

accomplished by in-place refactoring can keep existing systems running while making it easier to deliver value to users.

Deprecation During Design

Like many engineering activities, deprecation of a software system can be planned as those systems are first built. Choices of programming language, software architecture, team composition, and even company policy and culture all impact how easy it will be to eventually remove a system after it has reached the end of its useful life.

The concept of designing systems so that they can eventually be deprecated might be radical in software engineering, but it is common in other engineering disciplines. Consider the example of a nuclear power plant, which is an extremely complex piece of engineering. As part of the design of a nuclear power station, its eventual decommissioning after a lifetime of productive service must be taken into account, even going so far as to allocate funds for this purpose.[1] Many of the design choices in building a nuclear power plant are affected when engineers know that it will eventually need to be decommissioned.

Unfortunately, software systems are rarely so thoughtfully designed. Many software engineers are attracted to the task of building and launching new systems, not maintaining existing ones. The corporate culture of many companies, including Google, emphasizes building and shipping new products quickly, which often provides a disincentive for designing with deprecation in mind from the beginning. And in spite of the popular notion of software engineers as data-driven automata, it can be psychologically difficult to plan for the eventual demise of the creations we are working so hard to build.

So, what kinds of considerations should we think about when designing systems that we can more easily deprecate in the future? Here are a couple of the questions we encourage engineering teams at Google to ask:

- How easy will it be for my consumers to migrate from my product to a potential replacement?
- How can I replace parts of my system incrementally?

Many of these questions relate to how a system provides and consumes dependencies. For a more thorough discussion of how we manage these dependencies, see Chapter 16.

1 "Design and Construction of Nuclear Power Plants to Facilitate Decommissioning," Technical Reports Series No. 382, IAEA, Vienna (1997).

Finally, we should point out that the decision as to whether to support a project long term is made when an organization first decides to build the project. After a software system exists, the only remaining options are support it, carefully deprecate it, or let it stop functioning when some external event causes it to break. These are all valid options, and the trade-offs between them will be organization specific. A new startup with a single project will unceremoniously kill it when the company goes bankrupt, but a large company will need to think more closely about the impact across its portfolio and reputation as they consider removing old projects. As mentioned earlier, Google is still learning how best to make these trade-offs with our own internal and external products.

In short, don't start projects that your organization isn't committed to support for the expected lifespan of the organization. Even if the organization chooses to deprecate and remove the project, there will still be costs, but they can be mitigated through planning and investments in tools and policy.

Types of Deprecation

Deprecation isn't a single kind of process, but a continuum of them, ranging from "we'll turn this off someday, we hope" to "this system is going away tomorrow, customers better be ready for that." Broadly speaking, we divide this continuum into two separate areas: advisory and compulsory.

Advisory Deprecation

Advisory deprecations are those that don't have a deadline and aren't high priority for the organization (and for which the company isn't willing to dedicate resources). These could also be labeled *aspirational* deprecations: the team knows the system has been replaced, and although they hope clients will eventually migrate to the new system, they don't have imminent plans to either provide support to help move clients or delete the old system. This kind of deprecation often lacks enforcement: we hope that clients move, but can't force them to. As our friends in SRE will readily tell you: "Hope is not a strategy."

Advisory deprecations are a good tool for advertising the existence of a new system and encouraging early adopting users to start trying it out. Such a new system should *not* be considered in a beta period: it should be ready for production uses and loads and should be prepared to support new users indefinitely. Of course, any new system is going to experience growing pains, but after the old system has been deprecated in any way, the new system will become a critical piece of the organization's infrastructure.

One scenario we've seen at Google in which advisory deprecations have strong benefits is when the new system offers compelling benefits to its users. In these cases,

simply notifying users of this new system and providing them self-service tools to migrate to it often encourages adoption. However, the benefits cannot be simply incremental: they must be transformative. Users will be hesitant to migrate on their own for marginal benefits, and even new systems with vast improvements will not gain full adoption using only advisory deprecation efforts.

Advisory deprecation allows system authors to nudge users in the desired direction, but they should not be counted on to do the majority of migration work. It is often tempting to simply put a deprecation warning on an old system and walk away without any further effort. Our experience at Google has been that this can lead to (slightly) fewer new uses of an obsolete system, but it rarely leads to teams actively migrating away from it. Existing uses of the old system exert a sort of conceptual (or technical) pull toward it: comparatively many uses of the old system will tend to pick up a large share of new uses, no matter how much we say, "Please use the new system." The old system will continue to require maintenance and other resources unless its users are more actively encouraged to migrate.

Compulsory Deprecation

This active encouragement comes in the form of *compulsory* deprecation. This kind of deprecation usually comes with a deadline for removal of the obsolete system: if users continue to depend on it beyond that date, they will find their own systems no longer work.

Counterintuitively, the best way for compulsory deprecation efforts to scale is by localizing the expertise of migrating users to within a single team of experts—usually the team responsible for removing the old system entirely. This team has incentives to help others migrate from the obsolete system and can develop experience and tools that can then be used across the organization. Many of these migrations can be effected using the same tools discussed in Chapter 22.

For compulsory deprecation to actually work, its schedule needs to have an enforcement mechanism. This does not imply that the schedule can't change, but empower the team running the deprecation process to break noncompliant users after they have been sufficiently warned through efforts to migrate them. Without this power, it becomes easy for customer teams to ignore deprecation work in favor of features or other more pressing work.

At the same time, compulsory deprecations without staffing to do the work can come across to customer teams as mean spirited, which usually impedes completing the deprecation. Customers simply see such deprecation work as an unfunded mandate, requiring them to push aside their own priorities to do work just to keep their services running. This feels much like the "running to stay in place" phenomenon and creates friction between infrastructure maintainers and their customers. It's for this

reason that we strongly advocate that compulsory deprecations are actively staffed by a specialized team through completion.

It's also worth noting that even with the force of policy behind them, compulsory deprecations can still face political hurdles. Imagine trying to enforce a compulsory deprecation effort when the last remaining user of the old system is a critical piece of infrastructure your entire organization depends on. How willing would you be to break that infrastructure—and, transitively, everybody that depends on it—just for the sake of making an arbitrary deadline? It is hard to believe the deprecation is really compulsory if that team can veto its progress.

Google's monolithic repository and dependency graph gives us tremendous insight into how systems are used across our ecosystem. Even so, some teams might not even know they have a dependency on an obsolete system, and it can be difficult to discover these dependencies analytically. It's also possible to find them dynamically through tests of increasing frequency and duration during which the old system is turned off temporarily. These intentional changes provide a mechanism for discovering unintended dependencies by seeing what breaks, thus alerting teams to a need to prepare for the upcoming deadline. Within Google, we occasionally change the name of implementation-only symbols to see which users are depending on them unaware.

Frequently at Google, when a system is slated for deprecation and removal, the team will announce planned outages of increasing duration in the months and weeks prior to the turndown. Similar to Google's Disaster Recovery Testing (DiRT) exercises, these events often discover unknown dependencies between running systems. This incremental approach allows those dependent teams to discover and then plan for the system's eventual removal, or even work with the deprecating team to adjust their timeline. (The same principles also apply for static code dependencies, but the semantic information provided by static analysis tools is often sufficient to detect all the dependencies of the obsolete system.)

Deprecation Warnings

For both advisory and compulsory deprecations, it is often useful to have a programmatic way of marking systems as deprecated so that users are warned about their use and encouraged to move away. It's often tempting to just mark something as deprecated and hope its uses eventually disappear, but remember: "hope is not a strategy." Deprecation warnings can help prevent new uses, but rarely lead to migration of existing systems.

What usually happens in practice is that these warnings accumulate over time. If they are used in a transitive context (for example, library A depends on library B, which depends on library C, and C issues a warning, which shows up when A is built), these warnings can soon overwhelm users of a system to the point where they ignore them altogether. In health care, this phenomenon is known as "alert fatigue."

Any deprecation warning issued to a user needs to have two properties: actionability and relevance. A warning is *actionable* if the user can use the warning to actually perform some relevant action, not just in theory, but in practical terms, given the expertise in that problem area that we expect for an average engineer. For example, a tool might warn that a call to a given function should be replaced with a call to its updated counterpart, or an email might outline the steps required to move data from an old system to a new one. In each case, the warning provided the next steps that an engineer can perform to no longer depend on the deprecated system.[2]

A warning can be actionable, but still be annoying. To be useful, a deprecation warning should also be *relevant*. A warning is relevant if it surfaces at a time when a user actually performs the indicated action. Warning about the use of a deprecated function is best done while the engineer is writing code that uses that function, not after it has been checked into the repository for several weeks. Likewise, an email for data migration is best sent several months before the old system is removed rather than as an afterthought a weekend before the removal occurs.

It's important to resist the urge to put deprecation warnings on everything possible. Warnings themselves are not bad, but naive tooling often produces a quantity of warning messages that can overwhelm the unsuspecting engineer. Within Google, we are very liberal with marking old functions as deprecated but leverage tooling such as ErrorProne or clang-tidy to ensure that warnings are surfaced in targeted ways. As discussed in Chapter 20, we limit these warnings to newly changed lines as a way to warn people about new uses of the deprecated symbol. Much more intrusive warnings, such as for deprecated targets in the dependency graph, are added only for compulsory deprecations, and the team is actively moving users away. In either case, tooling plays an important role in surfacing the appropriate information to the appropriate people at the proper time, allowing more warnings to be added without fatiguing the user.

Managing the Deprecation Process

Although they can feel like different kinds of projects because we're deconstructing a system rather than building it, deprecation projects are similar to other software engineering projects in the way they are managed and run. We won't spend too much effort going over similarities between those management efforts, but it's worth pointing out the ways in which they differ.

2 See *https://abseil.io/docs/cpp/tools/api-upgrades* for an example.

Process Owners

We've learned at Google that without explicit owners, a deprecation process is unlikely to make meaningful progress, no matter how many warnings and alerts a system might generate. Having explicit project owners who are tasked with managing and running the deprecation process might seem like a poor use of resources, but the alternatives are even worse: don't ever deprecate anything, or delegate deprecation efforts to the users of the system. The second case becomes simply an advisory deprecation, which will never organically finish, and the first is a commitment to maintain every old system ad infinitum. Centralizing deprecation efforts helps better assure that expertise actually *reduces* costs by making them more transparent.

Abandoned projects often present a problem when establishing ownership and aligning incentives. Every organization of reasonable size has projects that are still actively used but that nobody clearly owns or maintains, and Google is no exception. Projects sometimes enter this state because they are deprecated: the original owners have moved on to a successor project, leaving the obsolete one chugging along in the basement, still a dependency of a critical project, and hoping it just fades away eventually.

Such projects are unlikely to fade away on their own. In spite of our best hopes, we've found that these projects still require deprecation experts to remove them and prevent their failure at inopportune times. These teams should have removal as their primary goal, not just a side project of some other work. In the case of competing priorities, deprecation work will almost always be perceived as having a lower priority and rarely receive the attention it needs. These sorts of important-not-urgent cleanup tasks are a great use of 20% time and provide engineers exposure to other parts of the codebase.

Milestones

When building a new system, project milestones are generally pretty clear: "Launch the frobnazzer features by next quarter." Following incremental development practices, teams build and deliver functionality incrementally to users, who get a win whenever they take advantage of a new feature. The end goal might be to launch the entire system, but incremental milestones help give the team a sense of progress and ensure they don't need to wait until the end of the process to generate value for the organization.

In contrast, it can often feel that the only milestone of a deprecation process is removing the obsolete system entirely. The team can feel they haven't made any progress until they've turned out the lights and gone home. Although this might be the most meaningful step for the team, if it has done its job correctly, it's often the least noticed by anyone external to the team, because by that point, the obsolete system no longer has any users. Deprecation project managers should resist the temptation to

make this the only measurable milestone, particularly given that it might not even happen in all deprecation projects.

Similar to building a new system, managing a team working on deprecation should involve concrete incremental milestones, which are measurable and deliver value to users. The metrics used to evaluate the progress of the deprecation will be different, but it is still good for morale to celebrate incremental achievements in the deprecation process. We have found it useful to recognize appropriate incremental milestones, such as deleting a key subcomponent, just as we'd recognize accomplishments in building a new product.

Deprecation Tooling

Much of the tooling used to manage the deprecation process is discussed in depth elsewhere in this book, such as the large-scale change (LSC) process (Chapter 22) or our code review tools (Chapter 19). Rather than talk about the specifics of the tools, we'll briefly outline how those tools are useful when managing the deprecation of an obsolete system. These tools can be categorized as discovery, migration, and backsliding prevention tooling.

Discovery

During the early stages of a deprecation process, and in fact during the entire process, it is useful to know *how* and *by whom* an obsolete system is being used. Much of the initial work of deprecation is determining who is using the old system—and in which unanticipated ways. Depending on the kinds of use, this process may require revisiting the deprecation decision once new information is learned. We also use these tools throughout the deprecation process to understand how the effort is progressing.

Within Google, we use tools like Code Search (see Chapter 17) and Kythe (see Chapter 23) to statically determine which customers use a given library, and often to sample existing usage to see what sorts of behaviors customers are unexpectedly depending on. Because runtime dependencies generally require some static library or thin client use, this technique yields much of the information needed to start and run a deprecation process. Logging and runtime sampling in production help discover issues with dynamic dependencies.

Finally, we treat our global test suite as an oracle to determine whether all references to an old symbol have been removed. As discussed in Chapter 11, tests are a mechanism of preventing unwanted behavioral changes to a system as the ecosystem evolves. Deprecation is a large part of that evolution, and customers are responsible for having sufficient testing to ensure that the removal of an obsolete system will not harm them.

Migration

Much of the work of doing deprecation efforts at Google is achieved by using the same set of code generation and review tooling we mentioned earlier. The LSC process and tooling are particularly useful in managing the large effort of actually updating the codebase to refer to new libraries or runtime services.

Preventing backsliding

Finally, an often overlooked piece of deprecation infrastructure is tooling for preventing the addition of new uses of the very thing being actively removed. Even for advisory deprecations, it is useful to warn users to shy away from a deprecated system in favor of a new one when they are writing new code. Without backsliding prevention, deprecation can become a game of whack-a-mole in which users constantly add new uses of a system with which they are familiar (or find examples of elsewhere in the codebase), and the deprecation team constantly migrates these new uses. This process is both counterproductive and demoralizing.

To prevent deprecation backsliding on a micro level, we use the Tricorder static analysis framework to notify users that they are adding calls into a deprecated system and give them feedback on the appropriate replacement. Owners of deprecated systems can add compiler annotations to deprecated symbols (such as the `@deprecated` Java annotation), and Tricorder surfaces new uses of these symbols at review time. These annotations give control over messaging to the teams that own the deprecated system, while at the same time automatically alerting the change author. In limited cases, the tooling also suggests a push-button fix to migrate to the suggested replacement.

On a macro level, we use visibility whitelists in our build system to ensure that new dependencies are not introduced to the deprecated system. Automated tooling periodically examines these whitelists and prunes them as dependent systems are migrated away from the obsolete system.

Conclusion

Deprecation can feel like the dirty work of cleaning up the street after the circus parade has just passed through town, yet these efforts improve the overall software ecosystem by reducing maintenance overhead and cognitive burden of engineers. Scalably maintaining complex software systems over time is more than just building and running software: we must also be able to remove systems that are obsolete or otherwise unused.

A complete deprecation process involves successfully managing social and technical challenges through policy and tooling. Deprecating in an organized and well-managed fashion is often overlooked as a source of benefit to an organization, but is essential for its long-term sustainability.

TL;DRs

- Software systems have continuing maintenance costs that should be weighed against the costs of removing them.
- Removing things is often more difficult than building them to begin with because existing users are often using the system beyond its original design.
- Evolving a system in place is usually cheaper than replacing it with a new one, when turndown costs are included.
- It is difficult to honestly evaluate the costs involved in deciding whether to deprecate: aside from the direct maintenance costs involved in keeping the old system around, there are ecosystem costs involved in having multiple similar systems to choose between and that might need to interoperate. The old system might implicitly be a drag on feature development for the new. These ecosystem costs are diffuse and difficult to measure. Deprecation and removal costs are often similarly diffuse.

PART IV

Tools

CHAPTER 16

Version Control and Branch Management

Written by Titus Winters
Edited by Lisa Carey

Perhaps no software engineering tool is quite as universally adopted throughout the industry as version control. One can hardly imagine any software organization larger than a few people that doesn't rely on a formal Version Control System (VCS) to manage its source code and coordinate activities between engineers.

In this chapter, we're going to look at why the use of version control has become such an unambiguous norm in software engineering, and we describe the various possible approaches to version control and branch management, including how we do it at scale across all of Google. We'll also examine the pros and cons of various approaches; although we believe everyone should use version control, some version control policies and processes might work better for your organization (or in general) than others. In particular, we find "trunk-based development" as popularized by DevOps[1] (one repository, no dev branches) to be a particularly scalable policy approach, and we'll provide some suggestions as to why that is.

What Is Version Control?

This section might be a little basic for many readers: use of version control is, after all, fairly ubiquitous. If you want to skip ahead, we suggest jumping to the section "Source of Truth" on page 334.

1 The DevOps Research Association, which was acquired by Google between the first draft of this chapter and publication, has published extensively on this in the annual "State of DevOps Report" and the book *Accelerate*. As near as we can tell, it popularized the terminology *trunk-based development*.

A VCS is a system that tracks revisions (versions) of files over time. A VCS maintains some metadata about the set of files being managed, and collectively a copy of the files and metadata is called a repository[2] (repo for short). A VCS helps coordinate the activities of teams by allowing multiple developers to work on the same set of files simultaneously. Early VCSs did this by granting one person at a time the right to edit a file—that style of locking is enough to establish sequencing (an agreed-upon "which is newer," an important feature of VCS). More advanced systems ensure that changes to a *collection* of files submitted at once are treated as a single unit (*atomicity* when a logical change touches multiple files). Systems like CVS (a popular VCS from the 90s) that didn't have this atomicity for a commit were subject to corruption and lost changes. Ensuring atomicity removes the chance of previous changes being overwritten unintentionally, but requires tracking which version was last synced to—at commit time, the commit is rejected if any file in the commit has been modified at head since the last time the local developer synced. Especially in such a change-tracking VCS, a developer's working copy of the managed files will therefore need metadata of its own. Depending on the design of the VCS, this copy of the repository can be a repository itself, or might contain a reduced amount of metadata—such a reduced copy is usually a "client" or "workspace."

This seems like a lot of complexity: why is a VCS necessary? What is it about this sort of tool that has allowed it to become one of the few nearly universal tools for software development and software engineering?

Imagine for a moment working without a VCS. For a (very) small group of distributed developers working on a project of limited scope without any understanding of version control, the simplest and lowest-infrastructure solution is to just pass copies of the project back and forth. This works best when edits are nonsimultaneous (people are working in different time zones, or at least with different working hours). If there's any chance for people to not know which version is the most current, we immediately have an annoying problem: tracking which version is the most up to date. Anyone who has attempted to collaborate in a non-networked environment will likely recall the horrors of copying back-and-forth files named *Presentation v5 - final - redlines - Josh's version v2*. And as we shall see, when there isn't a single agreed-upon source of truth, collaboration becomes high friction and error prone.

Introducing shared storage requires slightly more infrastructure (getting access to shared storage), but provides an easy and obvious solution. Coordinating work in a shared drive might suffice for a while with a small enough number of people but still requires out-of-band collaboration to avoid overwriting one another's work. Further, working directly in that shared storage means that any development task that doesn't

2 Although the formal idea of what is and is not a repository changes a bit depending on your choice of VCS, and the terminology will vary.

keep the build working continuously will begin to impede everyone on the team—if I'm making a change to some part of this system at the same time that you kick off a build, your build won't work. Obviously, this doesn't scale well.

In practice, lack of file locking and lack of merge tracking will inevitably lead to collisions and work being overwritten. Such a system is very likely to introduce out-of-band coordination to decide who is working on any given file. If that file-locking is encoded in software, we've begun reinventing an early-generation version control like RCS (among others). After you realize that granting write permissions a file at a time is too coarse grained and you begin wanting line-level tracking—we're definitely reinventing version control. It seems nearly inevitable that we'll want some structured mechanism to govern these collaborations. Because we seem to just be reinventing the wheel in this hypothetical, we might as well use an off-the-shelf tool.

Why Is Version Control Important?

While version control is practically ubiquitous now, this was not always the case. The very first VCSs date back to the 1970s (SCCS) and 1980s (RCS)—many years later than the first references to software engineering as a distinct discipline. Teams participated in "the multiperson development of multiversion software" before the industry had any formal notion of version control. Version control evolved as a response to the novel challenges of digital collaboration. It took decades of evolution and dissemination for reliable, consistent use of version control to evolve into the norm that it is today.[3] So how did it become so important, and, given that it seems like a self-evident solution, why might anyone resist the idea of VCS?

Recall that software engineering is programming integrated over time; we're drawing a distinction (in dimensionality) between the instantaneous production of source code and the act of maintaining that product over time. That basic distinction goes a long way to explaining the importance of, and hesitation toward, VCS: at the most fundamental level, version control is the engineer's primary tool for managing the interplay between raw source and time. We can conceptualize VCS as a way to extend a standard filesystem. A filesystem is a mapping from filename to contents. A VCS extends that to provide a mapping from (filename, time) to contents, along with the metadata necessary to track last sync points and audit history. Version control makes the consideration of time an explicit part of the operation: unnecessary in a program-

3 Indeed, I've given several public talks that use "adoption of version control" as the canonical example of how the norms of software engineering can *and do* evolve over time. In my experience, in the 1990s, version control was pretty well understood as a best practice but not universally followed. In the early 2000s, it was still common to encounter professional groups that didn't use it. Today, the use of tools like Git seems ubiquitous even among college students working on personal projects. Some of this rise in adoption is likely due to better user experience in the tools (nobody wants to go back to RCS), but the role of experience and changing norms is significant.

ming task, critical in a software engineering task. In most cases, a VCS also allows for an extra input to that mapping (a branch name) to allow for parallel mappings; thus:

```
VCS(filename, time, branch) => file contents
```

In the default usage, that branch input will have a commonly understood default: we call that "head," "default," or "trunk" to denote main branch.

The (minor) remaining hesitation toward consistent use of version control comes almost directly from conflating programming and software engineering—we teach programming, we train programmers, we interview for jobs based on programming problems and techniques. It's perfectly reasonable for a new hire, even at a place like Google, to have little or no experience with code that is worked on by more than one person or for more than a couple weeks. Given that experience and understanding of the problem, version control seems like an alien solution. Version control is solving a problem that our new hire hasn't necessarily experienced: an "undo," not for a single file but for an entire project, adding a lot of complexity for sometimes nonobvious benefits.

In some software groups, the same result plays out when management views the job of the techies as "software development" (sit down and write code) rather than "software engineering" (produce code, keep it working and useful for some extended period). With a mental model of programming as the primary task and little understanding of the interplay between code and the passage of time, it's easy to see something described as "go back to a previous version to undo a mistake" as a weird, high-overhead luxury.

In addition to allowing separate storage and reference to versions over time, version control helps us bridge the gap between single-developer and multideveloper processes. In practical terms, this is why version control is so critical to software engineering, because it allows us to scale up teams and organizations, even though we use it only infrequently as an "undo" button. Development is inherently a branch-and-merge process, both when coordinating between multiple developers or a single developer at different points in time. A VCS removes the question of "which is more recent?" Use of modern version control automates error-prone operations like tracking which set of changes have been applied. Version control is how we coordinate between multiple developers and/or multiple points in time.

Because VCS has become so thoroughly embedded in the process of software engineering, even legal and regulatory practices have caught up. VCS allows a formal record of every change to every line of code, which is increasingly necessary for satisfying audit requirements. When mixing between in-house development and appropriate use of third-party sources, VCS helps track provenance and origination for every line of code.

In addition to the technical and regulatory aspects of tracking source over time and handling sync/branch/merge operations, version control triggers some nontechnical changes in behavior. The ritual of committing to version control and producing a commit log is a trigger for a moment of reflection: what have you accomplished since your last commit? Is the source in a state that you're happy with? The moment of introspection associated with committing, writing up a summary, and marking a task complete might have value on its own for many people. The start of the commit process is a perfect time to run through a checklist, run static analyses (see Chapter 20), check test coverage, run tests and dynamic analysis, and so on.

Like any process, version control comes with some overhead: someone must configure and manage your version control system, and individual developers must use it. But make no mistake about it: these can almost always be pretty cheap. Anecdotally, most experienced software engineers will instinctively use version control for any project that lasts more than a day or two, even for a single-developer project. The consistency of that result argues that the trade-off in terms of value (including risk reduction) versus overhead must be a pretty easy one. But we've promised to acknowledge that context matters and to encourage engineering leaders to think for themselves. It is always worth considering alternatives, even on something as fundamental as version control.

In truth, it's difficult to envision any task that can be considered modern software engineering that doesn't immediately adopt a VCS. Given that you understand the value and need for version control, you are likely now asking what type of version control you need.

Centralized VCS Versus Distributed VCS

At the most simplistic level, all modern VCSs are equivalent to one another: so long as your system has a notion of atomically committing changes to a batch of files, everything else is just UI. You could build the same general semantics (not workflow) of any modern VCS out of another one and a pile of simple shell scripts. Thus, arguing about which VCS is "better" is primarily a matter of user experience—the core functionality is the same, the differences come in user experience, naming, edge-case features, and performance. Choosing a VCS is like choosing a filesystem format: when choosing among a modern-enough format, the differences are fairly minor, and the more important question by far is the content you fill that system with and the way you *use* it. However, major architectural differences in VCSs can make configuration, policy, and scaling decisions easier or more difficult, so it's important to be aware of the big architectural differences, chiefly the decision between centralized or decentralized.

Centralized VCS

In centralized VCS implementations, the model is one of a single central repository (likely stored on some shared compute resource for your organization). Although a developer can have files checked out and accessible on their local workstation, operations that interact on the version control status of those files need to be communicated to the central server (adding files, syncing, updating existing files, etc.). Any code that is committed by a developer is committed into that central repository. The first VCS implementations were all centralized VCSs.

Going back to the 1970s and early 1980s, we see that the earliest of these VCSs, such as RCS, focused on locking and preventing multiple simultaneous edits. You could copy the contents of a repository, but if you wanted to edit a file, you might need to acquire a lock, enforced by the VCS, to ensure that only you are making edits. When you've completed an edit, you release the lock. The model worked fine when any given change was a quick thing, or if there was rarely more than one person that wanted the lock for a file at any given time. Small edits like tweaking config files worked OK, as did working on a small team that either kept disjointed working hours or that rarely worked on overlapping files for extended periods. This sort of simplistic locking has inherent problems with scale: it can work fine for a few people, but has the potential to fall apart with larger groups if any of those locks become contended.[4]

As a response to this scaling problem, the VCSs that were popular through the 90s and early 2000s operated at a higher level. These more modern centralized VCSs avoid the exclusive locking but track which changes you've synced, requiring your edit to be based on the most-current version of every file in your commit. CVS wrapped and refined RCS by (mostly) operating on batches of files at a time and allowing multiple developers to check out a file at the same time: so long as your base version contained all of the changes in the repository, you're allowed to commit. Subversion advanced further by providing true atomicity for commits, version tracking, and better tracking for unusual operations (renames, use of symbolic links, etc.). The centralized repository/checked-out client model continues today within Subversion as well as most commercial VCSs.

Distributed VCS

Starting in the mid-2000s, many popular VCSs followed the Distributed Version Control System (DVCS) paradigm, seen in systems like Git and Mercurial. The

4 Anecdote: To illustrate this, I looked for information on what pending/unsubmitted edits Googlers had outstanding for a semipopular file in my most recent project. At the time of this writing, 27 changes are pending, 12 from people on my team, 5 from people on related teams, and 10 from engineers I've never met. This is basically working as expected. Technical systems or policies that require out-of-band coordination certainly don't scale to 24/7 software engineering in distributed locations.

primary conceptual difference between DVCS and more traditional centralized VCS (Subversion, CVS) is the question: "Where can you commit?" or perhaps, "Which copies of these files count as a repository?"

A DVCS world does not enforce the constraint of a central repository: if you have a copy (clone, fork) of the repository, you have a repository that you can commit to as well as all of the metadata necessary to query for information about things like revision history. A standard workflow is to clone some existing repository, make some edits, commit them locally, and then push some set of commits to another repository, which may or may not be the original source of the clone. Any notion of centrality is purely conceptual, a matter of policy, not fundamental to the technology or the underlying protocols.

The DVCS model allows for better offline operation and collaboration without inherently declaring one particular repository to be the source of truth. One repository isn't necessary "ahead" or "behind" because changes aren't inherently projected into a linear timeline. However, considering common *usage*, both the centralized and DVCS models are largely interchangeable: whereas a centralized VCS provides a clearly defined central repository through technology, most DVCS ecosystems define a central repository for a project as a matter of policy. That is, most DVCS projects are built around one conceptual source of truth (a particular repository on GitHub, for instance). DVCS models tend to assume a more distributed use case and have found particularly strong adoption in the open source world.

Generally speaking, the dominant source control system today is Git, which implements DVCS.[5] When in doubt, use that—there's some value in doing what everyone else does. If your use cases are expected to be unusual, gather some data and evaluate the trade-offs.

Google has a complex relationship with DVCS: our main repository is based on a (massive) custom in-house centralized VCS. There are periodic attempts to integrate more standard external options and to match the workflow that our engineers (especially Nooglers) have come to expect from external development. Unfortunately, those attempts to move toward more common tools like Git have been stymied by the sheer size of the codebase and userbase, to say nothing of Hyrum's Law effects tying us to a particular VCS and interface for that VCS.[6] This is perhaps not surprising: most existing tools don't scale well with 50,000 engineers and tens of millions of

5 Stack Overflow Developer Survey Results, 2018.

6 Monotonically increasing version numbers, rather than commit hashes, are particularly troublesome. Many systems and scripts have grown up in the Google developer ecosystem that assume that the numeric ordering of commits is the same as the temporal order—undoing those hidden dependencies is difficult.

commits.[7] The DVCS model, which often (but not always) includes transmission of history and metadata, requires a lot of data to spin up a repository to work out of.

In our workflow, centrality and in-the-cloud storage for the codebase seem to be critical to scaling. The DVCS model is built around the idea of downloading the entire codebase and having access to it locally. In practice, over time and as your organization scales up, any given developer is going to operate on a relatively smaller percentage of the files in a repository, and a small fraction of the versions of those files. As we grow (in file count and engineer count), that transmission becomes almost entirely waste. The only need for locality for most files occurs when building, but distributed (and reproducible) build systems seem to scale better for that task as well (see Chapter 18).

Source of Truth

Centralized VCSs (Subversion, CVS, Perforce, etc.) bake the source-of-truth notion into the very design of the system: whatever is most recently committed at trunk is the current version. When a developer goes to check out the project, by default that trunk version is what they will be presented with. Your changes are "done" when they have been recommitted on top of that version.

However, unlike centralized VCS, there is no *inherent* notion of which copy of the distributed repository is the single source of truth in DVCS systems. In theory, it's possible to pass around commit tags and PRs with no centralization or coordination, allowing disparate branches of development to propagate unchecked, and thus risking a conceptual return to the world of *Presentation v5 - final - redlines - Josh's version v2*. Because of this, DVCS requires more explicit policy and norms than a centralized VCS does.

Well-managed projects using DVCS declare one specific branch in one specific repository to be the source of truth and thus avoid the more chaotic possibilities. We see this in practice with the spread of hosted DVCS solutions like GitHub or GitLab—users can clone and fork the repository for a project, but there is still a single primary repository: things are "done" when they are in the trunk branch on that repository.

It isn't an accident that centralization and Source of Truth has crept back into the usage even in a DVCS world. To help illustrate just how important this Source of Truth idea is, let's imagine what happens when we don't have a clear source of truth.

7 For that matter, as of the publication of the Monorepo paper, the repository itself had something like 86 TB of data and metadata, ignoring release branches. Fitting that onto a developer workstation directly would be… challenging.

Scenario: no clear source of truth

Imagine that your team adheres to the DVCS philosophy enough to avoid defining a specific branch+repository as the ultimate source of truth.

In some respects, this is reminiscent of the *Presentation v5 - final - redlines - Josh's version v2* model—after you pull from a teammate's repository, it isn't necessarily clear which changes are present and which are not. In some respects, it's better than that because the DVCS model tracks the merging of individual patches at a much finer granularity than those ad hoc naming schemes, but there's a difference between the DVCS knowing *which* changes are incorporated and every engineer being sure they have *all* the past/relevant changes represented.

Consider what it takes to ensure that a release build includes all of the features that have been developed by each developer for the past few weeks. What (noncentralized, scalable) mechanisms are there to do that? Can we design policies that are fundamentally better than having everyone sign off? Are there any that require only sublinear human effort as the team scales up? Is that going to continue working as the number of developers on the team scales up? As far as we can see: probably not. Without a central Source of Truth, someone is going to keep a list of which features are potentially ready to be included in the next release. Eventually that bookkeeping is reproducing the model of having a centralized Source of Truth.

Further imagine: when a new developer joins the team, where do they get a fresh, known-good copy of the code?

DVCS enables a lot of great workflows and interesting usage models. But if you're concerned with finding a system that requires sublinear human effort to manage as the team grows, it's pretty important to have one repository (and one branch) actually defined to be the ultimate source of truth.

There is some relativity in that Source of Truth. That is, for a given project, that Source of Truth might be different for a different organization. This caveat is important: it's reasonable for engineers at Google or RedHat to have different Sources of Truth for Linux Kernel patches, still different than Linus (the Linux Kernel maintainer) himself would. DVCS works fine when organizations and their Sources of Truth are hierarchical (and invisible to those outside the organization)—that is perhaps the most practically useful effect of the DVCS model. A RedHat engineer can commit to the local Source of Truth repository, and changes can be pushed from there upstream periodically, while Linus has a completely different notion of what is the Source of Truth. So long as there is no choice or uncertainty as to where a change should be pushed, we can avoid a large class of chaotic scaling problems in the DVCS model.

In all of this thinking, we're assigning special significance to the trunk branch. But of course, "trunk" in your VCS is only the technology default, and an organization can

choose different policies on top of that. Perhaps the default branch has been abandoned and all work actually happens on some custom development branch—other than needing to provide a branch name in more operations, there's nothing inherently broken in that approach; it's just nonstandard. There's an (oft-unspoken) truth when discussing version control: the technology is only one part of it for any given organization; there is almost always an equal amount of policy and usage convention on top of that.

No topic in version control has more policy and convention than the discussion of how to use and manage branches. We look at branch management in more detail in the next section.

Version Control Versus Dependency Management

There's a lot of conceptual similarity between discussions of version control policies and dependency management (see Chapter 21). The differences are primarily in two forms: VCS policies are largely about how you manage your own code, and are usually much finer grained. Dependency management is more challenging because we primarily focus on projects managed and controlled by other organizations, at a higher granularity, and these situations mean that you don't have perfect control. We'll discuss a lot more of these high-level issues later in the book.

Branch Management

Being able to track different revisions in version control opens up a variety of different approaches for how to manage those different versions. Collectively, these different approaches fall under the term *branch management*, in contrast to a single "trunk."

Work in Progress Is Akin to a Branch

Any discussion that an organization has about branch management policies ought to at least acknowledge that every piece of work-in-progress in the organization is equivalent to a branch. This is more explicitly the case with a DVCS in which developers are more likely to make numerous local staging commits before pushing back to the upstream Source of Truth. This is still true of centralized VCSs: uncommitted local changes aren't conceptually different than committed changes on a branch, other than potentially being more difficult to find and diff against. Some centralized systems even make this explicit. For example, when using Perforce, every change is given two revision numbers: one indicating the implicit branch point where the change was created, and one indicating where it was recommitted, as illustrated in Figure 16-1. Perforce users can query to see who has outstanding changes to a given file, inspect the pending changes in other users' uncommitted changes, and more.

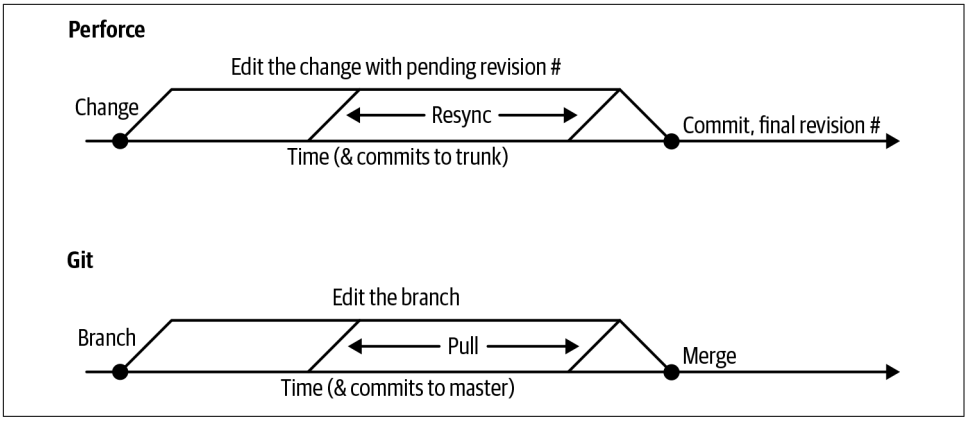

Figure 16-1. Two revision numbers in Perforce

This "uncommitted work is akin to a branch" idea is particularly relevant when thinking about refactoring tasks. Imagine a developer being told, "Go rename Widget to OldWidget." Depending on an organization's branch management policies and understanding, what counts as a branch, and which branches matter, this could have several interpretations:

- Rename Widget on the trunk branch in the Source of Truth repository
- Rename Widget on all branches in the Source of Truth repository
- Rename Widget on all branches in the Source of Truth repository, and find all devs with outstanding changes to files that reference Widget

If we were to speculate, attempting to support that "rename this everywhere, even in outstanding changes" use case is part of why commercial centralized VCSs tend to track things like "which engineers have this file open for editing?" (We don't think this is a scalable way to *perform* a refactoring task, but we understand the point of view.)

Dev Branches

In the age before consistent unit testing (see Chapter 11), when the introduction of any given change had a high risk of regressing functionality elsewhere in the system, it made sense to treat *trunk* specially. "We don't commit to trunk," your Tech Lead might say, "until new changes have gone through a full round of testing. Our team uses feature-specific development branches instead."

A development branch (usually "dev branch") is a halfway point between "this is done but not committed" and "this is what new work is based on." The problem that these are attempting to solve (instability of the product) is a legitimate one—but one that

we have found to be solved far better with more extensive use of tests, Continuous Integration (CI) (see Chapter 23), and quality enforcement practices like thorough code review.

We believe that a version control policy that makes extensive use of dev branches as a means toward product stability is inherently misguided. The same set of commits are going to be merged to trunk eventually. Small merges are easier than big ones. Merges done by the engineer who authored those changes are easier than batching unrelated changes and merging later (which will happen eventually if a team is sharing a dev branch). If presubmit testing on the merge reveals any new problems, the same argument applies: it's easier to determine whose changes are responsible for a regression if there is only one engineer involved. Merging a large dev branch implies that more changes are happening in that test run, making failures more difficult to isolate. Triaging and root-causing the problem is difficult; fixing it is even worse.

Beyond the lack of expertise and inherent problems in merging a single branch, there are significant scaling risks when relying on dev branches. This is a very common productivity drain for a software organization. When there are multiple branches being developed in isolation for long periods, coordinating merge operations becomes significantly more expensive (and possibly riskier) than they would be with trunk-based development.

How did we become addicted to dev branches?

It's easy to see how organizations fall into this trap: they see, "Merging this long-lived development branch reduced stability" and conclude, "Branch merges are risky." Rather than solve that with "Better testing" and "Don't use branch-based development strategies," they focus on slowing down and coordinating the symptom: the branch merges. Teams begin developing new branches based on other in-flight branches. Teams working on a long-lived dev branch might or might not regularly have that branch synched with the main development branch. As the organization scales up, the number of development branches grows as well, and the more effort is placed on coordinating that branch merge strategy. Increasing effort is thrown at coordination of branch merges—a task that inherently doesn't scale. Some unlucky engineer becomes the Build Master/Merge Coordinator/Content Management Engineer, focused on acting as the single point coordinator to merge all the disparate branches in the organization. Regularly scheduled meetings attempt to ensure that the organization has "worked out the merge strategy for the week."[8] The teams that aren't chosen to merge often need to re-sync and retest after each of these large merges.

8 Recent informal Twitter polling suggests about 25% of software engineers have been subjected to "regularly scheduled" merge strategy meetings.

All of that effort in merging and retesting is *pure overhead*. The alternative requires a different paradigm: trunk-based development, rely heavily on testing and CI, keep the build green, and disable incomplete/untested features at runtime. Everyone is responsible to sync to trunk and commit; no "merge strategy" meetings, no large/expensive merges. And, no heated discussions about which version of a library should be used—there can be only one. There must be a single Source of Truth. In the end, there will be a single revision used for a release: narrowing down to a single source of truth is just the "shift left" approach for identifying what is and is not being included.

Release Branches

If the period between releases (or the release lifetime) for a product is longer than a few hours, it may be sensible to create a release branch that represents the exact code that went into the release build for your product. If any critical flaws are discovered between the actual release of that product into the wild and the next release cycle, fixes can be cherry-picked (a minimal, targeted merge) from trunk to your release branch.

By comparison to dev branches, release branches are generally benign: it isn't the technology of branches that is troublesome, it's the usage. The primary difference between a dev branch and a release branch is the expected end state: a dev branch is expected to merge back to trunk, and could even be further branched by another team. A release branch is expected to be abandoned eventually.

In the highest-functioning technical organizations that Google's DevOps Research and Assessment (DORA) organization has identified, release branches are practically nonexistent. Organizations that have achieved Continuous Deployment (CD)—the ability to release from trunk many times a day—likely tend to skip release branches: it's much easier to simply add the fix and redeploy. Thus, cherry-picks and branches seem like unnecessary overhead. Obviously, this is more applicable to organizations that deploy digitally (such as web services and apps) than those that push any form of tangible release to customers; it is generally valuable to know exactly what has been pushed to customers.

That same DORA research also suggests a strong positive correlation between "trunk-based development," "no long-lived dev branches," and good technical outcomes. The underlying idea in both of those ideas seems clear: branches are a drag on productivity. In many cases we think complex branch and merge strategies are a perceived safety crutch—an attempt to keep trunk stable. As we see throughout this book, there are other ways to achieve that outcome.

Version Control at Google

At Google, the vast majority of our source is managed in a single repository (monorepo) shared among roughly 50,000 engineers. Almost all projects that are owned by Google live there, except large open source projects like Chromium and Android. This includes public-facing products like Search, Gmail, our advertising products, our Google Cloud Platform offerings, as well as the internal infrastructure necessary to support and develop all of those products.

We rely on an in-house-developed centralized VCS called Piper, built to run as a distributed microservice in our production environment. This has allowed us to use Google-standard storage, communication, and Compute as a Service technology to provide a globally available VCS storing more than 80 TB of content and metadata. The Piper monorepo is then simultaneously edited and committed to by many thousands of engineers every day. Between humans and semiautomated processes that make use of version control (or improve things checked into VCS), we'll regularly handle 60,000 to 70,000 commits to the repository per work day. Binary artifacts are fairly common because the full repository isn't transmitted and thus the normal costs of binary artifacts don't really apply. Because of the focus on Google-scale from the earliest conception, operations in this VCS ecosystem are still cheap at human scale: it takes perhaps 15 seconds total to create a new client at trunk, add a file, and commit an (unreviewed) change to Piper. This low-latency interaction and well-understood/well-designed scaling simplifies a lot of the developer experience.

By virtue of Piper being an in-house product, we have the ability to customize it and enforce whatever source control policies we choose. For instance, we have a notion of granular ownership in the monorepo: at every level of the file hierarchy, we can find OWNERS files that list the usernames of engineers that are allowed to approve commits within that subtree of the repository (in addition to the OWNERS that are listed at higher levels in the tree). In an environment with many repositories, this might have been achieved by having separate repositories with filesystem permissions enforcement controlling commit access or via a Git "commit hook" (action triggered at commit time) to do a separate permissions check. By controlling the VCS, we can make the concept of ownership and approval more explicit and enforced by the VCS during an attempted commit operation. The model is also flexible: ownership is just a text file, not tied to a physical separation of repositories, so it is trivial to update as the result of a team transfer or organization restructuring.

One Version

The incredible scaling powers of Piper alone wouldn't allow the sort of collaboration that we rely upon. As we said earlier: version control is also about policy. In addition to our VCS, one key feature of Google's version control policy is what we've come to refer to as "One Version." This extends the "Single Source of Truth" concept we

looked at earlier—ensuring that a developer knows which branch and repository is their source of truth—to something like "For every dependency in our repository, there must be only one version of that dependency to choose."[9] For third-party packages, this means that there can be only a single version of that package checked into our repository, in the steady state.[10] For internal packages, this means no forking without repackaging/renaming: it must be technologically safe to mix both the original and the fork into the same project with no special effort. This is a powerful feature for our ecosystem: there are very few packages with restrictions like "If you include this package (A), you cannot include other package (B)."

This notion of having a single copy on a single branch in a single repository as our Source of Truth is intuitive but also has some subtle depth in application. Let's investigate a scenario in which we have a monorepo (and thus arguably have fulfilled the letter of the law on Single Source of Truth), but have allowed forks of our libraries to propagate on trunk.

Scenario: Multiple Available Versions

Imagine the following scenario: some team discovers a bug in common infrastructure code (in our case, Abseil or Guava or the like). Rather than fix it in place, the team decides to fork that infrastructure and tweak it to work around the bug—without renaming the library or the symbols. It informs other teams near them, "Hey, we have an improved version of Abseil checked in over here: check it out." A few other teams build libraries that themselves rely on this new fork.

As we'll see in Chapter 21, we're now in a dangerous situation. If any project in the codebase comes to depend on both the original and the forked versions of Abseil simultaneously, in the best case, the build fails. In the worst case, we'll be subjected to difficult-to-understand runtime bugs stemming from linking in two mismatched versions of the same library. The "fork" has effectively added a coloring/partitioning property to the codebase: the transitive dependency set for any given target must include exactly one copy of this library. Any link added from the "original flavor" partition of the codebase to the "new fork" partition will likely break things. This means that in the end that something as simple as "adding a new dependency" becomes an operation that might require running all tests for the entire codebase, to ensure that we haven't violated one of these partitioning requirements. That's expensive, unfortunate, and doesn't scale well.

9 For example, during an upgrade operation, there might be two versions checked in, but if a developer is adding a new dependency on an existing package, there should be no *choice* in which version to depend upon.

10 That said, we fail at this in many cases because external packages sometimes have pinned copies of their own dependencies bundled in their source release. You can read more on how all of this goes wrong in Chapter 21.

In some cases, we might be able to hack things together in a way to allow a resulting executable to function correctly. Java, for instance, has a relatively standard practice called *shading*, which tweaks the names of the internal dependencies of a library to hide those dependencies from the rest of the application. When dealing with functions, this is technically sound, even if it is theoretically a bit of a hack. When dealing with types that can be passed from one package to another, shading solutions work neither in theory nor in practice. As far as we know, any technological trickery that allows multiple isolated versions of a library to function in the same binary share this limitation: that approach will work for functions, but there is no good (efficient) solution to shading types—multiple versions for any library that provides a vocabulary type (or any higher-level construct) will fail. Shading and related approaches are patching over the underlying issue: multiple versions of the same dependency are needed. (We'll discuss how to minimize that in general in Chapter 21.)

Any policy system that allows for multiple versions in the same codebase is allowing for the possibility of these costly incompatibilities. It's possible that you'll get away with it for a while (we certainly have a number of small violations of this policy), but in general, any multiple-version situation has a very real possibility of leading to big problems.

The "One-Version" Rule

With that example in mind, on top of the Single Source of Truth model, we can hopefully understand the depth of this seemingly simple rule for source control and branch management:

> Developers must never have a choice of "What version of this component should I depend upon?"

Colloquially, this becomes something like a "One-Version Rule." In practice, "One-Version" is not hard and fast,[11] but phrasing this around limiting the versions that can be *chosen* when adding a new dependency conveys a very powerful understanding.

For an individual developer, lack of choice can seem like an arbitrary impediment. Yet we see again and again that for an organization, it's a critical component in efficient scaling. Consistency has a profound importance at all levels in an organization. From one perspective, this is a direct side effect of discussions about consistency and ensuring the ability to leverage consistent "choke points."

11 For instance, if there are external/third-party libraries that are periodically updated, it might be infeasible to update that library and update all use of it in a single atomic change. As such, it is often necessary to add a new version of that library, prevent new users from adding dependencies on the old one, and incrementally switch usage from old to new.

(Nearly) No Long-Lived Branches

There are several deeper ideas and policies implicit in our One-Version Rule; foremost among them: development branches should be minimal, or at best be very short lived. This follows from a lot of published work over the past 20 years, from Agile processes to DORA research results on trunk-based development and even Phoenix Project[12] lessons on "reducing work-in-progress." When we include the idea of pending work as akin to a dev branch, this further reinforces that work should be done in small increments against trunk, committed regularly.

As a counterexample: in a development community that depends heavily on long-lived development branches, it isn't difficult to imagine opportunity for choice creeping back in.

Imagine this scenario: some infrastructure team is working on a new Widget, better than the old one. Excitement grows. Other newly started projects ask, "Can we depend on your new Widget?" Obviously, this can be handled if you've invested in codebase visibility policies, but the deep problem happens when the new Widget is "allowed" but only exists in a parallel branch. Remember: new development must not have a choice when adding a dependency. That new Widget should be committed to trunk, disabled from the runtime until it's ready, and hidden from other developers by visibility if possible—or the two Widget options should be designed such that they can coexist, linked into the same programs.

Interestingly, there is already evidence of this being important in the industry. In Accelerate and the most recent State of DevOps reports, DORA points out that there is a predictive relationship between trunk-based development and high-performing software organizations. Google is not the only organization to have discovered this—nor did we necessarily have expected outcomes in mind when these policies evolved—it just seemed like nothing else worked. DORA's result certainly matches our experience.

Our policies and tools for large-scale changes (LSCs; see Chapter 22) put additional weight on the importance of trunk-based development: broad/shallow changes that are applied across the codebase are already a massive (often tedious) undertaking when modifying everything checked in to the trunk branch. Having an unbounded number of additional dev branches that might need to be refactored at the same time would be an awfully large tax on executing those types of changes, finding an ever-expanding set of hidden branches. In a DVCS model, it might not even be possible to identify all of those branches.

12 Kevin Behr, Gene Kim, and George Spafford, *The Phoenix Project* (Portland: IT Revolution Press, 2018).

Of course, our experience is not universal. You might find yourself in unusual situations that require longer-lived dev branches in parallel to (and regularly merged with) trunk.

Those scenarios should be rare, and should be understood to be expensive. Across the roughly 1,000 teams that work in the Google monorepo, there are only a couple that have such a dev branch.[13] Usually these exist for a very specific (and very unusual) reason. Most of those reasons boil down to some variation of "We have an unusual requirement for compatibility over time." Oftentimes this is a matter of ensuring compatibility for data at rest across versions: readers and writers of some file format need to agree on that format over time even if the reader or writer implementations are modified. Other times, long-lived dev branches might come from promising API compatibility over time—when One Version isn't enough and we need to promise that an older version of a microservice client still works with a newer server (or vice versa). That can be a very challenging requirement, something that you should not promise lightly for an actively evolving API, and something you should treat carefully to ensure that period of time doesn't accidentally begin to grow. Dependency across time in any form is far more costly and complicated than code that is time invariant. Internally, Google production services make relatively few promises of that form.[14] We also benefit greatly from a cap on potential version skew imposed by our "build horizon": every job in production needs to be rebuilt and redeployed every six months, maximum. (Usually it is far more frequent than that.)

We're sure there are other situations that might necessitate long-lived dev branches. Just make sure to keep them rare. If you adopt other tools and practices discussed in this book, many will tend to exert pressure against long-lived dev branches. Automation and tooling that works great at trunk and fails (or takes more effort) for a dev branch can help encourage developers to stay current.

What About Release Branches?

Many Google teams use release branches, with limited cherry picks. If you're going to put out a monthly release and continue working toward the next release, it's perfectly reasonable to make a release branch. Similarly, if you're going to ship devices to customers, it's valuable to know exactly what version is out "in the field." Use caution and reason, keep cherry picks to a minimum, and don't plan to remerge with trunk. Our various teams have all sorts of policies about release branches given that relatively few teams have arrived at the sort of rapid release cadence promised by CD (see Chapter 24) that obviates the need or desire for a release branch. Generally speaking,

13 It's difficult to get a precise count, but the number of such teams is almost certainly fewer than 10.

14 Cloud interfaces are a different story.

release branches don't cause any widespread cost in our experience. Or, at least, no noticeable cost above and beyond the additional inherent cost to the VCS.

Monorepos

In 2016, we published a (highly cited, much discussed) paper on Google's monorepo approach.[15] The monorepo approach has some inherent benefits, and chief among them is that adhering to One Version is trivial: it's usually more difficult to violate One Version than it would be to do the right thing. There's no process of deciding which versions of anything are official, or discovering which repositories are important. Building tools to understand the state of the build (see Chapter 23) doesn't also require discovering where important repositories exist. Consistency helps scale up the impact of introducing new tools and optimizations. By and large, engineers can see what everyone else is doing and use that to inform their own choices in code and system design. These are all very good things.

Given all of that and our belief in the merits of the One-Version Rule, it is reasonable to ask whether a monorepo is the One True Way. By comparison, the open source community seems to work just fine with a "manyrepo" approach built on a seemingly infinite number of noncoordinating and nonsynchronized project repositories.

In short: no, we don't think the monorepo approach as we've described it is the perfect answer for everyone. Continuing the parallel between filesystem format and VCS, it's easy to imagine deciding between using 10 drives to provide one very large logical filesystem or 10 smaller filesystems accessed separately. In a filesystem world, there are pros and cons to both. Technical issues when evaluating filesystem choice would range from outage resilience, size constraints, performance characteristics, and so on. Usability issues would likely focus more on the ability to reference files across filesystem boundaries, add symlinks, and synchronize files.

A very similar set of issues governs whether to prefer a monorepo or a collection of finer-grained repositories. The specific decisions of how to store your source code (or store your files, for that matter) are easily debatable, and in some cases, the particulars of your organization and your workflow are going to matter more than others. These are decisions you'll need to make yourself.

What is important is not whether we focus on monorepo; it's to adhere to the One-Version principle to the greatest extent possible: developers must not have a *choice* when adding a dependency onto some library that is already in use in the organization. Choice violations of the One-Version Rule lead to merge strategy discussions, diamond dependencies, lost work, and wasted effort.

15 Rachel Potvin and Josh Levenberg, "Why Google stores billions of lines of code in a single repository," *Communications of the ACM*, 59 No. 7 (2016): 78-87.

Software engineering tools including both VCS and build systems are increasingly providing mechanisms to smartly blend between fine-grained repositories and monorepos to provide an experience akin to the monorepo—an agreed-upon ordering of commits and understanding of the dependency graph. Git submodules, Bazel with external dependencies, and CMake subprojects all allow modern developers to synthesize something weakly approximating monorepo behavior without the costs and downsides of a monorepo.[16] For instance, fine-grained repositories are easier to deal with in terms of scale (Git often has performance issues after a few million commits and tends to be slow to clone when repositories include large binary artifacts) and storage (VCS metadata can add up, especially if you have binary artifacts in your version control system). Fine-grained repositories in a federated/virtual-monorepo (VMR)–style repository can make it easier to isolate experimental or top-secret projects while still holding to One Version and allowing access to common utilities.

To put it another way: if every project in your organization has the same secrecy, legal, privacy, and security requirements,[17] a true monorepo is a fine way to go. Otherwise, *aim* for the functionality of a monorepo, but allow yourself the flexibility of implementing that experience in a different fashion. If you can manage with disjoint repositories and adhere to One Version or your workload is all disconnected enough to allow truly separate repositories, great. Otherwise, synthesizing something like a VMR in some fashion may represent the best of both worlds.

After all, your choice of filesystem format really doesn't matter as much as what you write to it.

Future of Version Control

Google isn't the only organization to publicly discuss the benefits of a monorepo approach. Microsoft, Facebook, Netflix, and Uber have also publicly mentioned their reliance on the approach. DORA has published about it extensively. It's vaguely possible that all of these successful, long-lived companies are misguided, or at least that their situations are sufficiently different as to be inapplicable to the average smaller organization. Although it's possible, we think it is unlikely.

Most arguments against monorepos focus on the technical limitations of having a single large repository. If cloning a repository from upstream is quick and cheap, developers are more likely to keep changes small and isolated (and to avoid making

16 We don't think we've seen anything do this particularly smoothly, but the interrepository dependencies/virtual monorepo idea is clearly in the air.

17 Or you have the willingness and capability to customize your VCS—and maintain that customization for the lifetime of your codebase/organization. Then again, maybe don't plan on that as an option; that is a lot of overhead.

mistakes with committing to the wrong work-in-progress branch). If cloning a repository (or doing some other common VCS operation) takes hours of wasted developer time, you can easily see why an organization would shy away from reliance on such a large repository/operation. We luckily avoided this pitfall by focusing on providing a VCS that scales massively.

Looking at the past few years of major improvements to Git, there's clearly a lot of work being done to support larger repositories: shallow clones, sparse branches, better optimization, and more. We expect this to continue and the importance of "but we need to keep the repository small" to diminish.

The other major argument against monorepos is that it doesn't match how development happens in the Open Source Software (OSS) world. Although true, many of the practices in the OSS world come (rightly) from prioritizing freedom, lack of coordination, and lack of computing resources. Separate projects in the OSS world are effectively separate organizations that happen to be able to see one another's code. Within the boundaries of an organization, we can make more assumptions: we can assume the availability of compute resources, we can assume coordination, and we can assume that there is some amount of centralized authority.

A less common but perhaps more legitimate concern with the monorepo approach is that as your organization scales up, it is less and less likely that every piece of code is subject to exactly the same legal, compliance, regulatory, secrecy, and privacy requirements. One native advantage of a manyrepo approach is that separate repositories are obviously capable of having different sets of authorized developers, visibility, permissions, and so on. Stitching that feature into a monorepo can be done but implies some ongoing carrying costs in terms of customization and maintenance.

At the same time, the industry seems to be inventing lightweight interrepository linkage over and over again. Sometimes, this is in the VCS (Git submodules) or the build system. So long as a collection of repositories have a consistent understanding of "what is trunk," "which change happened first," and mechanisms to describe dependencies, we can easily imagine stitching together a disparate collection of physical repositories into one larger VMR. Even though Piper has done very well for us, investing in a highly scaling VMR and tools to manage it and relying on off-the-shelf customization for per-repository policy requirements could have been a better investment.

As soon as someone builds a sufficiently large nugget of compatible and interdependent projects in the OSS community and publishes a VMR view of those packages, we suspect that OSS developer practices will begin to change. We see glimpses of this in the tools that *could* synthesize a virtual monorepo as well as in the work done by (for instance) large Linux distributions discovering and publishing mutually compatible revisions of thousands of packages. With unit tests, CI, and automatic version bumping for new submissions to one of those revisions, enabling a package owner to

update trunk for their package (in nonbreaking fashion, of course), we think that model will catch on in the open source world. It is just a matter of efficiency, after all: a (virtual) monorepo approach with a One-Version Rule cuts down the complexity of software development by a whole (difficult) dimension: time.

We expect version control and dependency management to evolve in this direction in the next 10 to 20 years: VCSs will focus on *allowing* larger repositories with better performance scaling, but also removing the need for larger repositories by providing better mechanisms to stitch them together across project and organizational boundaries. Someone, perhaps the existing package management groups or Linux distributors, will catalyze a de facto standard virtual monorepo. Depending on the utilities in that monorepo will provide easy access to a compatible set of dependencies as one unit. We'll more generally recognize that version numbers are timestamps, and that allowing version skew adds a dimensionality complexity (time) that costs a lot—and that we can learn to avoid. It starts with something logically like a monorepo.

Conclusion

Version control systems are a natural extension of the collaboration challenges and opportunities provided by technology, especially shared compute resources and computer networks. They have historically evolved in lockstep with the norms of software engineering as we understand them at the time.

Early systems provided simplistic file-granularity locking. As typical software engineering projects and teams grew larger, the scaling problems with that approach became apparent, and our understanding of version control changed to match those challenges. Then, as development increasingly moved toward an OSS model with distributed contributors, VCSs became more decentralized. We expect a shift in VCS technology that assumes constant network availability, focusing more on storage and build in the cloud to avoid transmitting unnecessary files and artifacts. This is increasingly critical for large, long-lived software engineering projects, even if it means a change in approach compared to simple single-dev/single-machine programming projects. This shift to cloud will make concrete what has emerged with DVCS approaches: even if we allow distributed development, something must still be centrally recognized as the Source of Truth.

The current DVCS decentralization is a sensible reaction of the technology to the needs of the industry (especially the open source community). However, DVCS configuration needs to be tightly controlled and coupled with branch management policies that make sense for your organization. It also can often introduce unexpected scaling problems: perfect fidelity offline operation requires a lot more local data. Failure to rein in the potential complexity of a branching free-for-all can lead to a potentially unbounded amount of overhead between developers and deployment of that code. However, complex technology doesn't need to be used in a complex fashion: as

we see in monorepo and trunk-based development models, keeping branch policies simple generally leads to better engineering outcomes.

Choice leads to costs here. We highly endorse the One-Version Rule presented here: developers within an organization must not have a choice where to commit, or which version of an existing component to depend upon. There are few policies we're aware of that can have such an impact on the organization: although it might be annoying for individual developers, in the aggregate, the end result is far better.

TL;DRs

- Use version control for any software development project larger than "toy project with only one developer that will never be updated."
- There's an inherent scaling problem when there are choices in "which version of this should I depend upon?"
- One-Version Rules are surprisingly important for organizational efficiency. Removing choices in where to commit or what to depend upon can result in significant simplification.
- In some languages, you might be able to spend some effort to dodge this with technical approaches like shading, separate compilation, linker hiding, and so on. The work to get those approaches working is entirely lost labor—your software engineers aren't producing anything, they're just working around technical debts.
- Previous research (DORA/State of DevOps/Accelerate) has shown that trunk-based development is a predictive factor in high-performing development organizations. Long-lived dev branches are not a good default plan.
- Use whatever version control system makes sense for you. If your organization wants to prioritize separate repositories for separate projects, it's still probably wise for interrepository dependencies to be unpinned/"at head"/"trunk based." There are an increasing number of VCS and build system facilities that allow you to have both small, fine-grained repositories as well as a consistent "virtual" head/trunk notion for the whole organization.